

SAPIENZA
UNIVERSITÀ DI ROMA

Experimental Evaluation of a Multi-Layer Abstraction Approach for RL on Non-Markovian Tasks

Faculty of Information Engineering, Informatics, and Statistics
Department of Computer, Control and Management Engineering "Antonio Ruberti"
Master of Science in Engineering in Computer Science

Matteo Ventali
ID number 1985026

Advisor
Prof. Fabio Patrizi

Academic Year 2025/2026

To my mother, my guiding light in life.
A mia mamma, la mia guida nella vita.

Abstract

Applying Reinforcement Learning to areas involving sparse rewards and objectives that extend over time could be prohibitive. Useful feedback can be delayed and the present state of the environment may not be enough to show how far along the task is. Until now, in order to reduce the complexity of the learning problem, several methods based on Hierarchical Reinforcement Learning and state-abstraction mechanisms have already been proposed in the literature.
In this thesis we look at a multi-level abstraction framework that makes use of a hierarchy of ever coarser Markov Decision Processes in order to obtain value-based shaping signals which guide learning at the lower levels. Starting from this architecture, the framework is extended to handle non-Markovian tasks that are given in LTL_f and are automatically converted into Deterministic Finite Automata, thereby allowing temporal progress towards the task to be explicitly represented across the entire abstraction hierarchy.
Then, the full implementation of the proposed approach is tested on a complex domain considering non-Markovian task specifications. In particular, the experimental study focuses on episodic tasks that involve increasing temporal complexity, various abstraction configurations, and cyclic temporal behaviours. The complex domain considered suitable to carry out this empirical evaluation is the well-known Gymnasium *Lunar Lander*. The results are employed in order to evaluate the effectiveness of abstraction-based guidance in this context and to determine the main practical limitations that arise when the framework is applied to complex non-Markovian learning problems.

Contents

1 Introduction **1**
1.1 Problem Statement 1
1.2 Objectives and Contributions 2
1.3 Thesis Organization 3

2 Background **4**
2.1 Reinforcement Learning 4
2.1.1 Markov Decision Processes 5
2.1.2 Policies, Returns, and Value Functions 6
2.1.3 Bellman Equations and Value Iteration 7
2.1.4 Value-Based Methods 8
2.2 Non-Markovianity and Temporally Extended Tasks 10
2.2.1 Markovian and Non-Markovian Tasks 10
2.2.2 Linear Temporal Logic on Finite Traces 12
2.2.3 Deterministic Finite Automata 12
2.2.4 From LTL_f Specifications to DFA 13
2.2.5 Automata-Based Approaches for Non-Markovian Problems . 14
2.3 Reward Design in Reinforcement Learning 15
2.3.1 Goal-Oriented Tasks and Sparse Rewards 16
2.3.2 Reward Shaping and PBRS 17

3 Multi-Layer Abstraction Framework **19**
3.1 State Abstraction and Abstract Guidance 19
3.1.1 State Abstraction mechanism 19
3.1.2 Guidance from Abstract Models 20
3.2 Reward Shaping and Learning Architecture 20
3.2.1 Non Return-Invariant Reward Shaping 21
3.2.2 Biased and Unbiased Learning 21
3.3 End-to-End Framework 22
3.4 Implementation and Design Choices 23
3.4.1 Framework Architecture 23
3.4.2 Temporal Task Specification 24
3.4.3 Configuration Interface and Automated Pipeline 25

4 Experimental Evaluation **27**
4.1 Evaluation Goals and Research Questions . . . 27
4.2 Experimental Setting . . . 28
4.2.1 Environment and Abstractions . . . 28
4.2.2 Temporal Tasks and Predicates . . . 30
4.2.3 Learning and Evaluation Setup . . . 32
4.3 RQ1: Direct Applicability of the Framework . . . 34
4.3.1 Biased-Unbiased Interaction . . . 34
4.3.2 Shaping Guidance . . . 37
4.3.3 Discussion . . . 39
4.4 RQ2: Performance under Increasing Temporal Complexity . . . 40
4.4.1 2-Phase Trajectory . . . 41
4.4.2 3-Phase Trajectory . . . 43
4.4.3 4-Phase Trajectory . . . 48
4.4.4 5-Phase Trajectory . . . 53
4.4.5 Discussion . . . 56
4.5 RQ3: Learning and Propagation of Abstract Guidance . . . 57
4.5.1 Experimental Design . . . 57
4.5.2 Learning on Abstract Levels . . . 58
4.5.3 Extraction and Analysis of the Guidance Signal . . . 60
4.5.4 Impact on Ground-Level Learning . . . 63
4.5.5 Discussion . . . 65
4.6 RQ4: Effectiveness on Cyclic Temporal Tasks . . . 65
4.6.1 Experimental Design . . . 65
4.6.2 Results . . . 66
4.6.3 Discussion . . . 69
4.7 Additional Case Studies . . . 70
4.7.1 Trajectory with Spatial Constraints . . . 70
4.7.2 Landing-Oriented Tasks . . . 72
4.7.3 Discussion . . . 75

5 Conclusions and Future Works **77**
5.1 Overall Findings . . . 77
5.2 Limitations . . . 78
5.3 Future Research Directions . . . 79
5.4 Final Remarks . . . 80

A Framework User Manual **81**
A.1 Project Structure . . . 81
A.2 Configuration Files . . . 82
A.2.1 Abstraction Configuration . . . 82
A.2.2 Task Configuration . . . 83
A.3 Source Code Organization . . . 85
A.3.1 Experiment Orchestration . . . 85
A.3.2 Learning Agents . . . 86
A.3.3 Abstraction Configuration . . . 86
A.3.4 Task Automata and Abstract MDPs . . . 87

A.3.5 Spatial Propositions 87
A.3.6 Other Supporting Modules 88
A.4 Running an Experiment 88
A.4.1 Experiment Output Directory Structure 88

Bibliography **90**

Chapter 1

Introduction

Reinforcement Learning is a general framework for learning sequential decision-making strategies through direct interaction with an environment. However, its applicability becomes increasingly challenging when we are in a sparse-reward setting and we consider complex task specifications.
In sparse-reward settings, the agent receives informative feedback only after reaching specific goal conditions. Although these task specifications are very simple, the main consequence is that successful trajectories may have to be discovered almost entirely through exploration. This difficulty becomes more relevant when the task requires several intermediate phases to be completed before any reward is obtained.
A further source of complexity that limits the applicability of RL arises when the objective cannot be characterized only through the current physical state of the environment. It is very common that several tasks require events to occur in a specific temporal order, or depend on what has already happened during the interaction. In these cases, we are referring to non-Markovian goal specifications and the combination of the latter with RL techniques is one of the most interesting research topics in this field. Nevertheless, the environment dynamics may still be Markovian, while the task specification introduces a dependence on the interaction history. Temporally extended objectives can be expressed through formal specifications and finite-state structures that keep track of task progress. At the same time, abstraction techniques can provide simplified representations of the original decision problem. Solving these simpler models can make additional information available to the learning agent and provide intermediate guidance even when the original task reward is sparse.
This motivates the study of approaches that combine temporal task representations, state abstraction, and additional reward signal techniques. In particular, this thesis will investigate whether abstract information can be exploited to guide RL algorithms on non-Markovian tasks, and which practical limitations emerge when this approach is applied to more complex task structures and domains.

1.1 Problem Statement

The starting point of this thesis is a multi-layer abstraction framework in which a sequence of simplified MDPs is built above the ground decision problem [4].
This architecture provides a promising way to address sparse-reward problems, since

simpler abstract models can generate intermediate information that is not directly available from the original environment. However, extending this approach to temporally extended non-Markovian tasks introduces additional challenges.
First, the temporal progress must be represented consistently across the abstraction hierarchy. When task satisfaction depends on the order in which events occur, the physical state alone is not sufficient to determine the current phase of the task. Therefore, the task-related information must be explicitly represented and synchronized across all abstraction levels.
Second, the effectiveness of the framework depends on the quality of the information propagated through the hierarchy. In principle an abstract decision problem could be solved with two main techniques: planning and learning. The latter is particularly suitable when the decision problem is complex enough and planning becomes infeasible.
Another difficulty arises from the relationship between the abstraction and the ground dynamics. An abstract representation may preserve information that is useful to describe the task while discarding other quantities that are relevant to its execution. As a consequence, even an additional signal that is informative at the abstract level may provide incomplete guidance when transferred to the ground environment.
The problem addressed in this thesis is therefore to understand whether a multi-layer abstraction framework can be effectively extended and applied to non-Markovian reinforcement-learning tasks, and which practical limitations emerge when the theoretical architecture is adopted in a complex domain.

1.2 Objectives and Contributions

The main objective of this thesis is to extend and experimentally evaluate the multi-layer abstraction framework in the presence of sparse rewards and temporally extended non-Markovian tasks.
In particular, an important target of the work is to make the framework as configurable and automated as possible, reducing the amount of problem-dependent adaptation required from the user. More precisely, we want to introduce support for LTL_f to express temporal objectives declaratively, rather than by manually constructing the corresponding automaton.
The same principle is adopted for the abstraction hierarchy. Its structure will be defined externally through independent configuration files, allowing the number and characteristics of the abstract levels to be modified without changing the underlying learning procedure. Together with the task specification mechanism, this results in a configurable pipeline in which both the temporal objective and the abstraction hierarchy can be described separately from the algorithmic implementation. Starting from this extended architecture, the thesis will also investigate its practical behavior in a complex ground domain.
The experimental analysis will be focused on four main aspects:

- the direct applicability of the original learning architecture;
- the behavior of the framework under increasing temporal complexity;

- the introduction of learning at abstract levels and the propagation of the resulting guidance information;
- the application of abstraction-based guidance to cyclic temporal behaviors.

Hence, the thesis contributes both an implementation-oriented extension of the reference framework and an empirical analysis of its practical capabilities and limitations. The experiments will be designed to highlight conditions under which abstract guidance is effective, as well as issues related to the learning architecture, temporal mismatch between levels and simplified abstract models that omit relevant physical information.

1.3 Thesis Organization

The remainder of this thesis is organized as follows:

- Chapter 2 introduces the theoretical background required throughout the work. It reviews the main concepts related to RL, MDP, value-based learning methods, temporally extended and non-Markovian tasks, LTL_f and DFA, and reward shaping;
- Chapter 3 describes the multi-layer abstraction framework introduced by Cipollone et al. [4], together with the extensions and implementation introduced in this thesis. In particular, it describes the abstraction hierarchy, the biased–unbiased learning architecture, the integration of temporal task specifications, and the mechanisms used to propagate the additional information signal across abstraction levels;
- Chapter 4 reports the experimental evaluation of the framework on the Gymnasium *Lunar Lander* domain. The analysis examines the aspects mentioned in the previous section;
- Chapter 5 summarizes the main findings of the work, discusses its limitations and contributions, and outlines possible directions for future works.

Chapter 2

Background

2.1 Reinforcement Learning

Within the Machine Learning area, a general distinction is made between supervised, unsupervised, and Reinforcement Learning. Reinforcement Learning (RL) is a branch of Machine Learning (ML) concerned with learning how to make sequential decisions through interaction with an environment.
In *supervised learning*, an algorithm is trained on a labeled dataset containing examples, with the objective of learning a mapping that generalizes to samples not belonging to it. On the other hand, *unsupervised learning* works on unlabeled data and aims to identify underlying structures and patterns within the available observations [15].
Reinforcement Learning differs from both paradigms because the learner is not provided with explicit examples of the correct action to perform. Instead, an agent interacts with an (initially) unknown environment, selects actions, observes their consequences, and receives a reward signal that provides feedback about its behavior. Therefore, the goal of the agent is to learn a policy that maximizes the expected cumulative reward over time.
This type of interaction describes a *sequential decision-making problem*. Unlike prediction problems in which individual outputs can often be considered independently, in RL the consequences of a decision may extend beyond the immediate transition. An action selected at a given time may influence the state reached by the agent, the actions that will subsequently become available, and the rewards that can be obtained in the future. So, the quality of a decision cannot generally be evaluated only in terms of its immediate outcome, but must also account for its long-term consequences. In fact, in RL we often refer to trajectories (sequences of states and actions) generated by the agent's behavior during its interaction with the environment.
RL can be viewed as a framework for sequential decision-making under uncertainty, where an agent must learn how its actions influence both immediate and future outcomes. To reason formally about these interactions, a mathematical model of the decision process is required. The standard framework adopted for this purpose is the Markov Decision Process (MDP), which provides a formal description of states, actions, transition dynamics, and rewards. MDPs are introduced in the following

section and will be the basis for the RL methods discussed throughout this chapter.

2.1.1 Markov Decision Processes

The mathematical framework adopted for modeling sequential decision-making problems is the Markov Decision Process (MDP). An MDP is defined as a tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle \tag{2.1}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the transition probability function, R is the reward function, and $\gamma \in [0, 1]$ is the discount factor [15].
At each discrete time step t, the environment is in a state $S_t \in \mathcal{S}$. Based on this state, the agent selects an action $A_t \in \mathcal{A}$. The execution of the action triggers the environment to transition into a new state S_{t+1} and produces a scalar reward R_{t+1}.

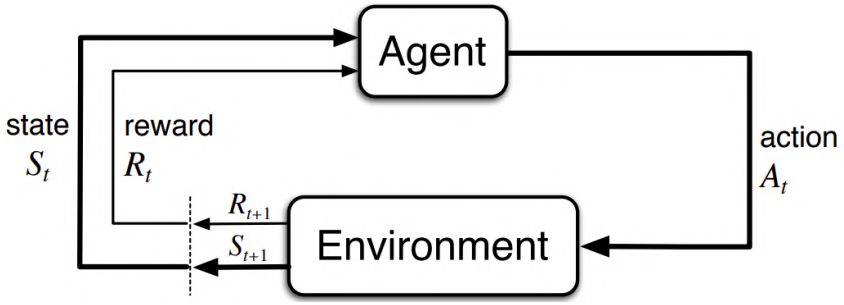


Figure 2.1. Interaction between the agent and the environment in an MDP [15].

The transition dynamics are characterized by

$$P(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) \tag{2.2}$$

which denotes the probability of reaching state s' after executing action a in state s. Looking at the P definition, it is immediate to notice that s' strictly depends only on the immediate previous state s and the action a. Therefore, with this definition, we are implicitly assuming the so-called *Markov property*: conditioned on the current state and action, the distribution of the next state is independent of the preceding interaction history.
Formally, it can be expressed as follows:

$$\Pr(s' \mid s, a, S_{t-1}, A_{t-1}, \ldots, S_0) = \Pr(s' \mid s, a) \tag{2.3}$$

The current state must contain all the information relevant to predicting the future evolution of the process [15]. If additional information from the interaction history is required, the chosen state representation is *not Markovian* with respect to the problem under consideration.
The reward function assigns a numerical signal to the agent's interaction with the environment. The most general formulation corresponds to the function $R(s, a, s')$ which provides immediate feedback, whereas the agent's objective generally depends on rewards accumulated over multiple time steps. The distinction between immediate and long-term consequences is therefore central to sequential decision-making.

The discount factor γ determines the relative importance of future rewards. Values close to zero emphasize immediate outcomes, whereas values close to one assign greater importance to rewards received further in the future. The accumulation of discounted rewards will be formalized through the concept of return in Section 2.1.2. A sequence generated through interaction between the agent and the environment can be written as

$$s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots \tag{2.4}$$

and is referred to as a *trajectory*. The probability of a trajectory is determined jointly by the transition dynamics of the MDP and the actions chosen by the agent based on its own *policy*.

2.1.2 Policies, Returns, and Value Functions

A *policy* specifies how the agent selects actions in each state. A stochastic policy π assigns a probability to every action $a \in \mathcal{A}$ given a state $s \in \mathcal{S}$:

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s) \tag{2.5}$$

For every state, these probabilities satisfy $\pi(a \mid s) \geq 0$ and $\sum_{a \in \mathcal{A}} \pi(a \mid s) = 1$. A deterministic policy is the special case in which one action is selected with probability one and can therefore be represented as a mapping $\pi : \mathcal{S} \to \mathcal{A}$.
The immediate reward obtained from a single transition is generally insufficient to evaluate an action, since its consequences may affect rewards received many steps later. The long-term outcome from time step t is represented by the *return*, defined as the discounted sum of future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \tag{2.6}$$

In episodic tasks, this sum terminates when a terminal state is reached. The discount factor γ controls the relative importance of immediate and delayed rewards. For continuing tasks with bounded rewards, choosing $\gamma < 1$ also ensures that the infinite sum remains finite.
Value functions measure the expected return obtained when the agent follows a given policy. The *state-value function* evaluates a state, whereas the *action-value function* evaluates the selection of a particular action in that state:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] \tag{2.7}$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \tag{2.8}$$

The expectation accounts for both the actions selected according to π and the stochastic transitions of the environment. The state-value and action-value functions are related by

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s)\, Q^\pi(s, a) \tag{2.9}$$

An optimal policy π^* maximizes the expected return from every state. The corresponding optimal value functions are

$$V^*(s) = \max_\pi V^\pi(s) \tag{2.10}$$

$$Q^*(s,a) = \max_\pi Q^\pi(s,a) \tag{2.11}$$

Once Q^* is known, an optimal deterministic policy can be obtained by selecting an action with maximum value:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} Q^*(s,a) \tag{2.12}$$

Policies and value functions therefore provide the principal quantities used to describe and compare decision-making strategies. Their recursive structure is captured by the Bellman equations [15].

2.1.3 Bellman Equations and Value Iteration

The return in Equation (2.6) can be decomposed into the immediate reward and the discounted return from the next time step:

$$G_t = R_{t+1} + \gamma G_{t+1} \tag{2.13}$$

This recursive relation leads to the *Bellman expectation equation*. For a policy π, the value of a state equals the expected immediate reward plus the discounted value of the successor state:

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^\pi(s') \right] \tag{2.14}$$

The corresponding equation for the action-value function is

$$Q^\pi(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q^\pi(s',a') \right] \tag{2.15}$$

These equations express a consistency condition: the value assigned to the current state or action must agree with the values assigned to its possible successors. For an optimal policy, the expectation over the next action is replaced by a maximization, yielding the Bellman optimality equations

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^*(s') \right] \tag{2.16}$$

$$Q^*(s,a) = \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a,s') + \gamma \max_{a' \in \mathcal{A}} Q^*(s',a') \right] \tag{2.17}$$

When the transition and reward models are known, the optimal state-value function can be computed through *value iteration*. Starting from an arbitrary estimate V_0, the Bellman optimality backup is applied repeatedly:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V_k(s') \right] \tag{2.18}$$

For a finite discounted MDP, the sequence of estimates converges to V^*. An optimal policy can then be extracted by acting greedily with respect to the converged values:

$$\pi^*(s) \in \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^*(s') \right] \tag{2.19}$$

Value iteration is a model-based planning method because it requires explicit knowledge of P and R. In RL, these quantities are unknown, motivating the adoption of model-free methods like Q-learning [15].

Policy Evaluation

Given a fixed policy π, the associated V^π can be computed by applying the Bellman expectation equation in an iterative way. Starting from a random initialization V_0, the update is

$$V^\pi_{k+1}(s) = \sum_{s' \in \mathcal{S}} P(s' \mid s, \pi(s)) \left[R(s, \pi(s), s') + \gamma V^\pi_k(s') \right] \tag{2.20}$$

for a deterministic policy π. The procedure is repeated until the value function converges. For finite discounted MDPs, iterative policy evaluation converges to V^π.

2.1.4 Value-Based Methods

Value-based RL methods compute an optimal policy by estimating value functions rather than representing the policy π directly. In particular, action-value methods learn the expected return associated with each state-action pair and derive a policy by selecting actions adopting a greedy strategy with respect to these estimates. These algorithms can be classified in two different categories:

- *on-policy*: the algorithm learns the values of the current policy. In other words, the data used for the policy update are generated adopting the same policy the algorithm is trying to improve.
- *off-policy*: the algorithm learns the values adopting a different policy with respect to the one that is trying to improve. We usually refer to the first one as *Behavior Policy*, which controls directly the exploration and interaction with the environment during the learning phase.

Q-learning

Q-learning is a model-free and off-policy algorithm that estimates the optimal action-value function directly from observed transitions [20]. After observing the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, it applies the temporal-difference update

$$Q_{t+1}(S_t, A_t) = Q_t(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q_t(S_{t+1}, a') - Q_t(S_t, A_t) \right] \tag{2.21}$$

where α is the learning rate hyper-parameter. The maximization defines a greedy target policy, while experience can be collected through an exploratory behavior policy such as ϵ-greedy. The most straightforward implementation of it is Tabular Q-learning which is particularly effective for small discrete MDPs, but it becomes impractical when the state space is large because it maintains a separate estimate for every (s, a) pair [15].

Deep Q-Networks

Deep Q-Networks (DQN) address the limitation of Q-learning by approximating the action-value function with a neural network $Q(s, a; \theta)$. To improve training stability, DQN stores transitions in a replay memory and learns from randomly sampled mini-batches. It also maintains a target network with parameters θ^-, which are periodically synchronized with the online parameters θ. For a transition (s, a, r, s', d), the target is

$$y^{\mathrm{DQN}} = r + \gamma(1 - d) \max_{a' \in \mathcal{A}} Q(s', a'; \theta^-) \tag{2.22}$$

where d indicates whether the transition is terminal. The online network is trained by minimizing the squared difference between this target and $Q(s, a; \theta)$ [12].

Double Deep Q-Networks

The maximization in the DQN target can lead to overestimated action values because the same estimates are used both to select and evaluate the next action. Double Deep Q-Networks (DDQN) reduce this bias by using the online network for action selection and the target network for action evaluation:

$$y^{\mathrm{DDQN}} = r + \gamma(1 - d) Q\left(s', \arg\max_{a' \in \mathcal{A}} Q(s', a'; \theta); \theta^-\right) \tag{2.23}$$

DDQN retains the replay memory, target network, and optimization procedure of DQN while changing only the construction of the temporal-difference target. This generally produces more reliable value estimates [18].

Dueling Deep Q-Networks

The Dueling Network architecture divides the estimation of the V-function from the relative advantage associated with each action [19]. More precisely, the advantage function is defined as

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \tag{2.24}$$

Then, the Q-function can be re-written as

$$Q(s, a) = V(s) + A(s, a) \tag{2.25}$$

In the neural architecture, these two quantities are estimated through separate streams. A direct sum of the two estimates is however not identifiable, since the

same constant can be added to both terms without changing the resulting Q-value. For this reason, the dueling architecture combines the two streams as

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s,a';\theta,\alpha) \right) \tag{2.26}$$

where θ denotes the parameters shared by the common feature representation, while α and β denote the parameters of the advantage and value streams, respectively. This decomposition allows the network to estimate the value of a state independently from the relative relevance of the available actions. The dueling architecture can be combined with Double DQN, yielding the so-called Dueling-DDQN algorithm.

2.2 Non-Markovianity and Temporally Extended Tasks

In several situations, a problem specification cannot be expressed as an objective that depends only on the last state of the environment. Specifically, in these scenarios, a successful behavior could depend on the order in which events occur, on whether certain conditions have already been satisfied, or on whether specific events are eventually reached during the execution.
For instance, consider a task requiring an agent to first visit a region (A) and subsequently reach a region (B). Reaching (B) cannot be considered sufficient to determine task completion unless we have already reached region (A) in the past. Therefore, two trajectories ending in the region (B) may correspond to different levels of progress with respect to the task.
These types of objective specifications are commonly referred to as *temporally extended tasks*, since their satisfaction is defined over sequences of states or observations rather than on individual states only. When the information required to evaluate the task is not completely represented by the current environment state, the task introduces a form of *non-Markovian* dependence on the interaction history (the *Markov property* is violated).
Representing such objectives requires a formalism capable of expressing temporal relationships between events. In this thesis, we choose to express these goal specifications through Linear Temporal Logic on finite traces (LTL_f), a logical language which allows temporal properties to be specified over finite executions. Then, LTL_f specifications can in turn be converted into a deterministic finite automaton (DFA), whose states provide a way to monitor the task progress.

2.2.1 Markovian and Non-Markovian Tasks

As introduced in Section 2.1.1, the Markov property requires the current state to contain all the information needed to characterize the future evolution of the environment [15].
In an MDP, the probability distribution $P(s' \mid s, a)$ of the next state depends only on the current state and action, and not on the complete interaction history.
A similar distinction can be made with respect to the task or reward structure.
In a *Markovian task*, the relevance of a transition can be determined from the current

state, action, and successor state. So, under this assumption, a reward function can be expressed as $R(s, a, s')$.

However, there are some objectives that cannot be evaluated using only the current transition. In particular, their satisfaction depends on events that have occurred earlier in the execution. Let

$$h_t = s_0, a_0, s_1, a_1, \ldots, s_t \tag{2.27}$$

denote the interaction history up to time t. When the reward or the task satisfaction depends on h_t, we are referring to a *Non-Markovian task specification.*
This distinction is crucial because the environment itself may still satisfy the Markov property even when the task specification does not. In other words, non-Markovianity may arise from the specification of the objective rather than from the dynamics of the environment. The physical state can therefore be sufficient to predict the evolution of the environment while being insufficient to determine the progress made toward the task.

This concept is formalized by the *Non-Markovian Reward Decision Process* (NMRDP) definition introduced in [2].
An NMRDP is defined as a tuple:

$$\mathcal{M}_{\mathrm{NM}} = \left\langle \mathcal{S}, \mathcal{A}, P, \overline{R}, \gamma \right\rangle \tag{2.28}$$

where $\mathcal{S}$, $\mathcal{A}$, P, and γ have the same meaning as in an MDP, while the reward function is defined over finite sequences of state-action pairs:

$$\overline{R} : (\mathcal{S} \times \mathcal{A})^* \to \mathbb{R} \tag{2.29}$$

Let

$$\tau = \langle (s_0, a_0), (s_1, a_1), \ldots, (s_{n-1}, a_{n-1}) \rangle \tag{2.30}$$

be a finite trace, and let $\tau_{\leq i}$ denote its prefix containing the first i state-action pairs. The value of τ is defined as

$$V(\tau) = \sum_{i=1}^{|\tau|} \gamma^{i-1} \overline{R}(\tau_{\leq i}) \tag{2.31}$$

Unlike a Markovian policy, whose action depends only on the current state, a policy for an NMRDP may depend on the complete sequence of states observed so far:

$$\overline{\pi} : \mathcal{S}^* \to \mathcal{A} \tag{2.32}$$

Therefore, both the reward and the policy may depend on the interaction history rather than exclusively on the current state.

2.2.2 Linear Temporal Logic on Finite Traces

Linear Temporal Logic (LTL) [14] is a formal language used to describe properties of sequences that evolve over time. Unlike propositional logic, which evaluates formulas with respect to a single state (the *interpretation*), temporal logic allows the specification of relationships between events occurring at different points of an execution.
LTL is interpreted over infinite traces. However, in episodic RL, interactions between agent and environment are naturally represented by finite sequences, since an episode terminates after a finite number of steps. Therefore, in this thesis, we mainly consider LTL_f which is a variant of LTL evaluated on finite traces [5].

Let P be a finite set of atomic propositions describing relevant properties of the environment. At each time step, a subset of these propositions is true. A finite trace over P can therefore be represented as

$$\rho = \rho_0, \rho_1, \ldots, \rho_{n-1} \tag{2.33}$$

where

$$\rho_i \in 2^P \tag{2.34}$$

denotes the set of atomic propositions that hold at position i.
An LTL_f formula is built from atomic propositions using boolean and temporal operators. Formally,

$$\varphi ::= A \mid (\neg\varphi) \mid (\varphi_1 \wedge \varphi_2) \mid (\circ\varphi) \mid (\varphi_1 \, U \, \varphi_2) \mid (\Diamond\varphi) \mid (\Box\varphi) \tag{2.35}$$

where $A \in P$, $\circ$ is the *next* operator, U is the *until* operator, $\Diamond$ is the *eventually* operator and $\Box$ is the *globally* operator.
The satisfaction of an LTL_f formula is defined with respect to a position of a finite trace. For an atomic proposition p,

$$\rho, i \models p \quad \Longleftrightarrow \quad p \in \rho_i \tag{2.36}$$

A finite trace ρ satisfies an LTL_f formula φ, written as

$$\rho \models \varphi \tag{2.37}$$

when the formula is satisfied at the initial position of the trace. Hence, every LTL_f formula defines a set of finite traces that satisfy the corresponding temporal specification.

2.2.3 Deterministic Finite Automata

A Deterministic Finite Automaton (DFA) is a finite-state machine useful to recognize languages defined over a finite alphabet. Formally, a DFA is defined as a tuple

$$\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle \tag{2.38}$$

where Q is a finite set of states, Σ is a finite input alphabet, δ is the transition function

$$\delta : Q \times \Sigma \to Q \tag{2.39}$$

$q_0 \in Q$ is the initial state, and $F \subseteq Q$ is the set of accepting states [1].
The automaton processes an input word one symbol at a time. Since the transition function is deterministic, for every state $q \in Q$ and input symbol $\sigma \in \Sigma$, there exists a unique successor state

$$q' = \delta(q, \sigma) \tag{2.40}$$

Given a finite word

$$w = \sigma_0 \sigma_1 \dots \sigma_{n-1} \in \Sigma^* \tag{2.41}$$

the execution, or run, of the automaton over w is the sequence of states

$$q_0, q_1, \dots, q_n \tag{2.42}$$

such that

$$q_{i+1} = \delta(q_i, \sigma_i) \qquad 0 \leq i < n \tag{2.43}$$

The word w is accepted by the automaton if the state reached after processing the complete word belongs to the accepting set:

$$q_n \in F \tag{2.44}$$

The set of all finite words accepted by $\mathcal{A}$ defines the language recognized by the automaton, and it is denoted by

$$L(\mathcal{A}) = \{w \in \Sigma^* \mid \mathcal{A} \text{ accepts } w\} \tag{2.45}$$

2.2.4 From LTL$_f$ Specifications to DFA

An LTL$_f$ formula can be represented through an equivalent DFA. As described in [5], this property is achieved thanks to the fact that every LTL$_f$ formula defines a regular language over finite traces.
Formally, let's consider the LTL$_f$ formula φ defined over a set P of atomic propositions. We can define the DFA $\mathcal{A}_\varphi = \langle Q, 2^P, \delta, q_0, F \rangle$ such that, for every finite trace ρ, the following holds:

$$\rho \models \varphi \quad \iff \quad \rho \in L(\mathcal{A}_\varphi) \tag{2.46}$$

Therefore, the LTL$_f$ formula and the corresponding DFA provide two different representations of the same temporal specification. In particular, the former provides a declarative way to describe a temporal specification, whereas the latter provides an operational representation useful for the integration of the formula in an algorithm.

The DFA state summarizes the information contained in the observed trace that is relevant to the satisfaction of the temporal specification. Hence, we can interpret it as a finite-memory representation of the progress made toward the task.
This representation is particularly useful for temporally extended tasks. If we consider different trajectories that lead the environment to the same physical state at a certain time, we will be able to identify the current correct stage of the temporal objective in each trajectory. Hence, the environment state and the automaton state can be considered jointly through an augmented representation

$$\tilde{s}_t = (s_t, q_t) \tag{2.47}$$

where s_t represents the physical configuration of the environment and q_t corresponds to the logical progress with respect to the LTL_f specification. This representation makes the task-relevant history explicitly available to the learning process and it is the basis for the treatment of temporally extended objectives adopted in the framework presented in Chapter 3.

This construction has been formalized in the literature on NMRDP. In particular, in [3] it is shown that $\text{LTL}_f/\text{LDL}_f$ non-Markovian rewards can be compiled into an equivalent Markovian MDP by augmenting the environment state with the states of the corresponding automata.

2.2.5 Automata-Based Approaches for Non-Markovian Problems

In the RL literature different approaches exploit finite-state representation for non-Markovian objectives. Two relevant examples are Reward Machines and Restraining Bolts.

Reward Machines

Reward Machines [16] provide an explicit finite-state representation of the internal structure of a reward function. Formally, given a set of propositional symbols $\mathcal{P}$, an environment state space $\mathcal{S}$, and an action space $\mathcal{A}$, a Reward Machine can be defined as

$$\mathcal{R} = \langle U, u_0, F, \delta_u, \delta_r \rangle$$

where U is a finite set of machine states, $u_0 \in U$ is the initial state, F is a set of terminal states,

$$\delta_u : U \times 2^{\mathcal{P}} \to U \cup F$$

is the transition function, and

$$\delta_r : U \to [\mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}]$$

associates each machine state with the reward function that must be used at that stage of the task. Furthermore, a labeling function

$$L : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to 2^{\mathcal{P}}$$

maps environment transitions to the high-level events processed by the machine.

Restraining Bolts

Restraining Bolts [8] tackle the problem from a different perspective. They introduce a logical specification describing temporal requirements and constraints that the RL agent should satisfy. The restraining bolt is defined independently by the agent as

$$RB = \langle \mathcal{L}, \{(\varphi_i, r_i)\}_{i=1}^{m} \rangle$$

where each φ_i is an $\text{LTL}_f/\text{LDL}_f$ formula defined over the high-level features available to the restraining bolt and r_i is the reward associated with its satisfaction. For each temporal specification φ_i, an equivalent DFA

$$\mathcal{A}_{\varphi_i} = \langle Q_i, \Sigma_i, \delta_i, q_{i0}, F_i \rangle$$

is obtained. The current states of these automata are then made available to the learning agent, augmenting its state representation from S to

$$S \times Q_1 \times \cdots \times Q_m$$

The most important aspect of this tool is that the features used by the restraining bolt may be different from the ones exposed by the original environment. Hence, the logical specification is defined externally and only the resulting automaton-state needs to be exposed to the learner.

Relation with the Framework Adopted in this Thesis

The framework considered in this thesis shares the general state-augmentation approach exploited by both techniques. In particular, similarly to restraining bolts, the temporal objective is specified declaratively through an LTL_f formula and then it is automatically translated into a DFA. But, at the same time, the overall construction is also related to reward machines since the automaton state separates the different temporal phases of the task. However, the DFAs used in this thesis do not directly encode a dense reward function.

2.3 Reward Design in Reinforcement Learning

The reward design task plays a central role in an RL setting. As depicted in Figure 2.1, the reward received by the agent provides feedback on its actions. Therefore, the reward function can be considered as an encoding of which actions are desirable with respect to the task objective required by the problem. Unfortunately, there is no general method for defining a reward function. Each reward function must be suitable and informative for the problem under consideration. In a common scenario, the most intuitive reward function corresponds to feeding the agent with a reward only at the end of the task. Although this formulation is trivial, it makes the learning process more difficult. On the other hand, defining a reward function that generates also intermediate feedback could be cumbersome and not straightforward.

In this section, we report some notions about reward functions with a particular focus on sparse-reward settings. Then, we describe the Reward Shaping mechanism useful to enrich in some way the reward signal that comes out from the reward function.

2.3.1 Goal-Oriented Tasks and Sparse Rewards

Many RL problems are naturally formulated in terms of achieving a specific objective. In these scenarios, the agent receives a reward signal only when it reaches the final goal, and therefore it is not guided through a reward signal that continuously assesses every intermediate action. Such problems can be modeled as goal-oriented MDPs, in which the task objective is explicitly associated with a goal condition.

Let g denote a goal and let $\mathcal{G}_g \subseteq \mathcal{S}$ be the set of states that satisfy it. A simple sparse formulation assigns a positive reward only when a transition reaches a goal state:

$$R_g(s, a, s') = \begin{cases} r_g & \text{if } s' \in \mathcal{G}_g \\ 0 & \text{otherwise} \end{cases} \tag{2.48}$$

where $r_g > 0$ is the reward associated with successful goal achievement.
Depending on the setting, reaching a goal state may also terminate the episode.

A reward function is described as *sparse* when the agent receives informative feedback only on a small fraction of the transitions it experiences. On the other hand, when the agent receives a high frequency reward signal from the environment, we refer to a *dense* reward function.

A notable sparse-reward setting is the *Goal MDP*: an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ is a *Goal MDP* if and only if there exists a set $\mathcal{G} \subseteq \mathcal{S}$ of goal states for which the reward function satisfies

$$R(s, a, s') = \begin{cases} 1 & \text{if } s \notin \mathcal{G} \text{ and } s' \in \mathcal{G} \\ 0 & \text{otherwise} \end{cases} \tag{2.49}$$

In addition, goal states have zero optimal continuation value:

$$V^*(s) = 0, \qquad \forall s \in \mathcal{G} \tag{2.50}$$

Under these conditions, the agent receives a unit reward only when it enters the goal set from a non-goal state. Once a goal state has been reached, no further return can be accumulated from that state.
Sparse rewards provide a direct representation of the task objective because they reward behavior that actually achieves the final goal. However, they also make learning more difficult [15, 4]. Before encountering a successful trajectory, the agent may receive the same reward for many different actions and states, with no direct indication of whether its behavior is moving toward or away from the objective. This difficulty is closely related to exploration. Without informative intermediate feedback, the agent must discover successful behavior largely through interaction with the environment. If the goal can be reached only after a long sequence of appropriate decisions, the probability of finding a successful trajectory without the reward signal's guidance may be very low.
Sparse-reward problems will be central to the framework described in Chapter 3.

2.3.2 Reward Shaping and PBRS

To address the consequences of sparse-reward settings, in the literature it has been introduced a technique called Reward Shaping. The core idea behind reward shaping is to augment the reward signal generated by the reward function of the original MDP with an additional signal [13]. This additional signal provides intermediate feedback to the agent during the learning process.

Formally, given an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, a shaping function F can be used to define a modified reward function:

$$R'(s, a, s') = R(s, a, s') + F(s, a, s') \tag{2.51}$$

The introduction of F transforms the original MDP M to a modified version $\mathcal{M}' = \langle \mathcal{S}, \mathcal{A}, P, R', \gamma \rangle$. This transformation could modify the desirability of trajectories and the optimal policy induced by the original reward function.

Potential Based Reward Shaping

One of the most adopted solution to design the shaping function F is Potential Based Reward Shaping (PBRS) [13]. Formally,

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s) \tag{2.52}$$

From the definition, it is immediate to see that the function $F(s', a, s)$ does not depend on the action a and therefore it could be defined simply as $F(s, s')$. Moreover, there is the introduction of $\Phi(s)$ which is the *Potential Function*. Specifically,

$$\Phi : S \to \mathbb{R} \tag{2.53}$$

assigns a scalar potential value to each state. Despite the difference in meaning between Φ and V-function, they share a structural similarity in their definition. In fact, the V-function is a practical and natural choice when the reward shaping technique is adopted.

Policy Invariance of PBRS

The introduction of the shaping term according to the PBRS formulation does not alter the optimal policies of the original MDP, if we assume the following facts:

- the discount factor $\gamma \in [0, 1)$;
- the potential function Φ is bounded.

This result relies on the fact that the shaping contribution accumulated along a trajectory assumes a telescoping structure. Considering a trajectory

$$s_0, s_1, \ldots, s_T$$

the discounted cumulative shaping reward is

$$\sum_{t=0}^{T-1} \gamma^t \left(\gamma\Phi(s_{t+1}) - \Phi(s_t)\right) = -\Phi(s_0) + \gamma^T \Phi(s_T) \tag{2.54}$$

Under the assumptions:

$$\lim_{T\to\infty} \gamma^T \Phi(s_T) = 0$$

and therefore

$$\sum_{t=0}^{\infty} \gamma^t F(s_t, s_{t+1}) = -\Phi(s_0) \tag{2.55}$$

For a fixed initial state s_0, the shaped return differs from the original one only for $-\Phi(s_0)$. Hence, the relative ordering of policies is preserved, and any optimal policy in the original MDP remains optimal in the shaped MDP [13].
When we are in episodic finite-horizon settings, additional care is required because the terminal term $\gamma^T \Phi(s_T) \neq 0$. A common sufficient condition is to assign zero potential to terminal states:

$$\Phi(s) = 0, \qquad \forall s \in \mathcal{S}_{\text{terminal}} \tag{2.56}$$

so that, the cumulative shaping contribution again reduces to $-\Phi(s_0)$ [9]. Under this condition, the shaping transformation preserves the original task objective also in the episodic case.

Chapter 3

Multi-Layer Abstraction Framework

The foundations of this thesis lie in the multi-level framework introduced in [4]. The framework exploits abstract representations of the original decision process in order to tackle the difficulties associated with a sparse-reward learning problem.

In this chapter, we firstly analyze the main components of the reference framework including the abstraction hierarchy and the learning architecture adopted. Then, we move on to presenting the resulting end-to-end algorithm. In the end, we describe its implementation and the integration of non-Markovian goal specifications through LTL_f.

3.1 State Abstraction and Abstract Guidance

The core idea behind the framework corresponds to the linear hierarchy of MDPs built over the ground domain. In particular, each level of the stack can be seen as a simplification of the one immediately below. The purpose of this hierarchical organization is to exploit simpler representations of the original decision problem in order to derive additional information that can guide the learning process in a more efficient way.
Let

$$\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_L$$

denote the sequence of MDPs forming the hierarchy, where $\mathcal{M}_0$ represents the ground MDP. Therefore, the hierarchy can be conceptually represented as

$$\mathcal{M}_L \to \mathcal{M}_{L-1} \to \cdots \to \mathcal{M}_0$$

where the information flows according to the arrows.

3.1.1 State Abstraction mechanism

Each component of the hierarchy is a complete MDP $\mathcal{M}_i = \langle \mathcal{S}_i, \mathcal{A}_i, P_i, R_i, \gamma_i \rangle$. Given two consecutive levels of the hierarchy, $\mathcal{M}_i$ and $\mathcal{M}_{i+1}$, the latter represents a simplified model of the former. They are connected through an abstraction mapping,

which is a function that maps the states of $\mathcal{M}_i$ into the states of $\mathcal{M}_{i+1}$. Formally the mapping function is

$$\varphi_i : \mathcal{S}_i \to \mathcal{S}_{i+1} \tag{3.1}$$

and the pair $\langle \mathcal{M}_{i+1}, \varphi_i \rangle$ is the abstraction built over $\mathcal{M}_i$. Unlike other HRL frameworks, this one does not impose any constraint about the mapping between $\mathcal{A}_i$ and $\mathcal{A}_{i+1}$. This feature leaves freedom and flexibility to the designer during the definition of the abstractions. Therefore, each MDP can be equipped with an appropriate set of actions suitable for each level of the problem description. The only assumptions made in [4] are:

- $\mathcal{M}_0$ is a Goal MDP;
- if $\mathcal{M}_i$ is a Goal MDP with goal states $\mathcal{G}_i$ and $\langle \mathcal{M}_{i+1}, \varphi_i \rangle$ is the abstraction built over it, then $\mathcal{M}_{i+1}$ is a Goal MDP such that:

$$\mathcal{G}_{i+1} = \bigcup_{s \in \mathcal{G}_i} \varphi_i(s) \tag{3.2}$$

3.1.2 Guidance from Abstract Models

Once the abstraction hierarchy has been defined, the framework exploits the higher-level models to derive guidance for the learning process at lower levels. The main idea is to solve an abstract MDP and use the resulting information in some way to speed-up the learning phase at the lower level.

Let $\mathcal{M}_{i+1}$ be an MDP of the hierarchy. We can solve it using planning or learning techniques. In particular, we can adopt the first solution when the MDP is fully-known, including the transition function and the reward function. This is not always true, especially when we are considering a rich and complex abstraction over the real problem. For instance, an abstraction that includes in its dynamics physics relations and quantities.
In the framework, the information extracted at level $(i + 1)$ corresponds to V_{i+1}. Then, this term is plugged inside the PBRS formulation described in Section 2.3.2:

$$F_i(s, a, s') = \gamma V_{i+1}(s') - V_{i+1}(s) \tag{3.3}$$

$F_i(s, a, s')$ corresponds to the shaping term exploited to enrich the reward function at level i. As a consequence of this choice, transitions in $\mathcal{M}_i$ that lead towards states mapped to abstract ones with higher potential values receive an additional positive feedback in the reward signal. On the other hand, transitions towards states associated with lower potential in the abstraction can receive a negative shaping contribution.

3.2 Reward Shaping and Learning Architecture

Although the framework adopts a potential-based shaping formulation, its shaping mechanism differs from the standard return-invariant setting.

Episodic tasks are common settings to be faced in RL. In these situations, preserving the return-invariance property of PBRS requires a particular treatment of terminal

states, as discussed in Section 2.3.2. While this property guarantees that the shaped and original MDPs share the same optimal policies, it may also limit the ability of the shaping signal to provide persistent exploration guidance.

3.2.1 Non Return-Invariant Reward Shaping

The return-invariance property of PBRS can be preserved by assigning a null potential to terminal states. Under this condition, for a trajectory $s_0, s_1, \ldots, s_T$, the cumulative discounted shaping contribution is

$$G = -V(s_0) + \gamma^T V(s_T) \tag{3.4}$$

and since $V(s_T) = 0$, the previous expression reduces to $-V(s_0)$.

It is immediate to notice that, for trajectories originating from the same initial state (s_0 is the same), the cumulative shaping reward is independent of the intermediate states visited. Hence, in a sparse reward problem like a Goal MDP, all finite trajectories that terminate without reaching the objective will receive the same shaped return contribution, regardless of whether they end close to or far from the desired goal.

To overcome this limitation, the authors of [4] introduce a modified version of PBRS called *non return-invariant Reward Shaping*. In practice, the constraint about the final state value is relaxed.
Now, two unsuccessful trajectories ρ_1 and ρ_2, originating from the same state, can satisfy

$$G(\rho_1) > G(\rho_2)$$

retaining useful information about the quality of the trajectories, with respect to the distance between the final state and the final goal. In other words, adopting this version, the shaping mechanism provides a persistent bias toward states associated with higher abstract values.

3.2.2 Biased and Unbiased Learning

The non return-invariant shaping mechanism loses the guarantee that the optimal policy of the shaped MDP corresponds to the optimal one in the original MDP.
From this consideration, it's now important to consider two MDPs at each level in the hierarchy. Precisely, we have:

- $\mathcal{M}^u = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ named *unbiased* MDP. The reward function R is not influenced by the shaping signal;
- $\mathcal{M}^b = \langle \mathcal{S}, \mathcal{A}, P, R', \gamma \rangle$ named *biased* MDP. Here, the reward function R' involves also the shaping signal according to the non return-invariant shaping formulation.

At this point, the task is to learn an optimal policy in $\mathcal{M}^u$ exploiting the information coming from the learning process performed on $\mathcal{M}^b$. Hence, from now on, we have two learning procedures over the two different MDPs.

The learning procedures are coupled together so that we can exploit the advantage coming from the shaping signal without losing the guarantee of reaching the optimal policy in the unbiased MDP. In particular, the biased learner is responsible for interacting with the environment and therefore it establishes the behavior policy used during data collection. The key idea supporting this architecture is to exploit the trajectories generated by the biased learner in $\mathcal{M}^b$ to update also the unbiased one.
Formally, for each transition generated during the interaction with the environment

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

both learning processes are updated using the same experience, but with different reward signals. The biased learner uses

$$r'_{t+1} = r_{t+1} + F(s_t, a, s_{t+1})$$

while the unbiased learner uses only the original reward r_{t+1}. In this way, the unbiased learner is able to see trajectories toward the final goal allowing it to learn the optimal desired policy.
It is worth mentioning that this architecture requires the adoption of an off-policy algorithm to implement the unbiased learner. This concept is obvious considering that transitions come from a different algorithm (the biased learner).

3.3 End-to-End Framework

The overall procedure described in the previous sections is summarized in the following algorithm extracted and adapted from [4].

Algorithm 1: Full learning architecture

Input: An off-policy learning algorithm $\mathcal{L}$
Input: The hierarchy $\mathcal{M}_0, \ldots, \mathcal{M}_L$ and the abstraction mappings $\varphi_0, \ldots, \varphi_{L-1}$
Output: An estimate $\widehat{\pi}_0^*$ of the optimal policy for the ground MDP

$\widehat{V}^*_{L+1}(s) \leftarrow 0$;
$\varphi_L(s) \leftarrow s$;
for $i \leftarrow L, L-1, \ldots, 0$ **do**
  $F_i \leftarrow \text{Shaping}\left(\gamma_i, \varphi_i, \widehat{V}^*_{i+1}\right)$;
  $\text{Learner}_i^u \leftarrow \mathcal{L}(\mathcal{M}_i)$;
  $\text{Learner}_i^b \leftarrow \mathcal{L}(\mathcal{M}_i)$;
  while $\neg\, \text{Learner}_i^b.\text{Stop}()$ **do**
    $s \leftarrow \mathcal{M}_i.\text{State}()$;
    $a \leftarrow \text{Learner}_i^b.\text{Action}(s)$;
    $(r, s') \leftarrow \mathcal{M}_i.\text{Act}(a)$;
    $r^b \leftarrow r + F_i(s, a, s')$;
    $\text{Learner}_i^b.\text{Update}(s, a, r^b, s')$;
    $\text{Learner}_i^u.\text{Update}(s, a, r, s')$;
  $\widehat{\pi}_i^* \leftarrow \text{Learner}_i^u.\text{Output}()$;
  $\widehat{V}_i^* \leftarrow \text{ComputeValue}(\widehat{\pi}_i^*, \mathcal{M}_i)$;

The algorithm highlights two important features of the learning architecture:

1. after completing the learning process at level i, the value function used to construct the shaping signal for the level immediately below is derived from the unbiased learner. In particular, the policy produced by the unbiased learner is used to compute the corresponding value function V_i^*, which then serves as the source of guidance for level $i-1$;

2. the final result of the procedure is the policy $\widehat{\pi}_0^*$ learned by the unbiased learner at the ground level.

Hence, it is clear that the biased policies are used only as intermediate mechanisms to improve the exploration process for the unbiased learner.

3.4 Implementation and Design Choices

The framework described in the previous sections represents the foundation of this thesis. Starting from the architecture and the algorithm introduced in [4], a full implementation of the multi-level framework has been developed with the aim of experimentally evaluating its effectiveness against temporally extended non-Markovian task specifications in a complex domain.
Until now, the original framework has been evaluated on relatively simple tasks and domains. Moreover, when a temporally extended behavior was required, there was a need to manually define the corresponding DFA for the task [4].
In this thesis, we address this limitation by explicitly introducing support for task specifications expressed in LTL_f. Furthermore, the whole training pipeline has been fully automated so that it is transparent and *ready to use* for the user.

3.4.1 Framework Architecture

The implementation developed in this thesis follows the multi-level architecture described in the previous sections, while extending it with an explicit representation of temporally extended tasks. The temporal objective is specified through an LTL_f formula, which is automatically translated into a DFA taking advantage of the *ltlf2dfa* Python library [7]. The state q of the automaton is shared across all levels in the hierarchy, so that all abstraction levels are synchronized on the same temporal objective over different representations of the environment.

The hierarchy is processed from the highest abstraction level toward the ground MDP. The highest-level MDP is solved without receiving shaping guidance. When its transition and reward models are explicitly available and the state space is sufficiently small, we can also adopt a Value Iteration algorithm to solve the abstract task. Once a level has been solved, the resulting V-function is propagated to the level immediately below and used to construct its shaping signal according to the mechanism described in Algorithm 1. This procedure is repeated throughout the hierarchy until the ground level is reached.

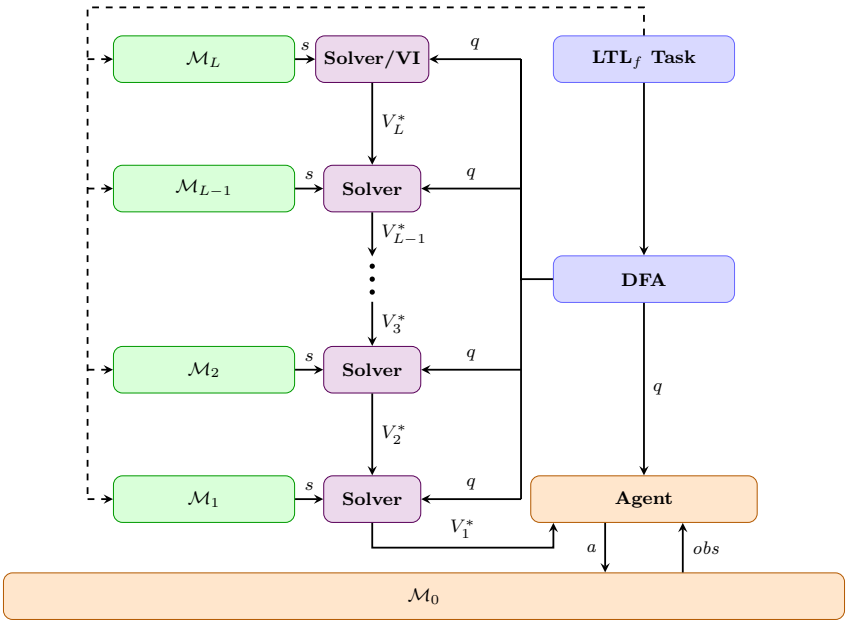


Figure 3.1. Full framework architecture.

At each level, the V-function propagated to the level immediately below is derived from the solution produced by the unbiased learner. The framework supports two different ways to extract the unbiased V-function.
The first possibility consists of extracting the greedy policy induced by the unbiased Q-function following

$$\pi^u(s) = \arg\max_{a \in \mathcal{A}} Q^u(s, a) \tag{3.5}$$

Then, V^{π^u} is computed through the iterative Policy Evaluation procedure, as described in Section 2.1.3. When this solution is adopted, the numerical magnitudes and errors of Q^u are not propagated directly.
Alternatively, the V-function can be obtained by estimating it following

$$\hat{V}^u(s) = \max_{a \in \mathcal{A}} Q^u(s, a) \tag{3.6}$$

Therefore, the magnitudes and estimation errors are propagated through the shaping mechanism. A comparative study of the two solutions will be presented in Chapter 4.

3.4.2 Temporal Task Specification

The implementation supports two different classes of temporally extended tasks:

- episodic tasks: they are expressed through an LTL$_f$ formula;
- cyclic tasks: they are expressed through a dedicated automaton whose transitions encode the phases of the recurring task.

Episodic tasks

For episodic tasks, reaching an accepting state of the DFA corresponds to the completion of the temporal objective. The associated goal reward is generated and the current interaction episode terminates. At the beginning of a new episode, both the environment and the automaton state are reinitialized.

Cyclic tasks

For cyclic tasks, reaching the end of one temporal sequence does not terminate the overall objective. Instead, the task progress is redirected to the initial phase of the cycle, allowing the same temporal behavior to be executed again during the same episode. Hence, LTL_f is not suitable for expressing this type of recurring task. However, the corresponding automaton is designed to contain transitions that explicitly represent this recurrence.

3.4.3 Configuration Interface and Automated Pipeline

The complete framework is configurable through two external JSON files, one describing the abstraction hierarchy and one specifying the temporal task. This design separates the experimental configuration from the implementation of the learning procedure, allowing different tasks and abstraction settings to be tested without modifying the underlying source code.

The abstraction hierarchy is defined through an `abstraction.json` file. This configuration specifies the number of abstraction levels and the parameters required to instantiate and solve the corresponding abstract MDPs. A possible configuration is the following one:

```
{
    "levels": [
        {
            "name": "level1", "grid_w": 360, "grid_h": 360,
            "learning": {
                "episodes": 1000000,
                "max_steps": 600,
                "alpha": 0.1,
                "epsilon_start": 1.0,
                "epsilon_min": 0.05,
                "epsilon_decay": 0.999995,
                "gamma_shaping": 1.0
            }
        },
        {
            "name": "level2", "grid_w": 180, "grid_h": 180,
            "algorithm": "value_iteration"
        }
    ]
}
```

Listing 3.1. Example of an abstraction hierarchy configuration.

The temporal task is specified through a separate configuration file `trajectory.json`. In the episodic case, the configuration contains the LTL_f formula together with the definitions required to evaluate the atomic propositions appearing in it.
In the cyclic case, the recurring temporal structure is explicitly encoded and from that specification the automaton is automatically built.

A possible episodic task configuration is the following one:

```
{
    "formula": "F(wp1 & X(F(low & g1)))",
    "goal_reward": 10000,
    "regions": {
        "wp1": {
            "center": [-0.5, 1.0],
            "radius": 0.1
        },
        "g1": {
            "center": [0.5, 0.5],
            "radius": 0.1
        }
    },
    "predicates": {
        "low": {
            "type": "half_plane",
            "axis": "y",
            "operator": "<",
            "threshold": 0.7
        }
    }
}
```

Listing 3.2. Example of an LTL_f task configuration.

A possible cyclic task configuration is the following one:

```
{
    "task_type": "cyclic_waypoints",
    "goal_reward": 10000,
    "waypoint_cycle": ["g1", "g2", "g3", "g4"],
    "regions": {
        "g1": {"center": [-0.75, 1.0625], "radius": 0.12},
        "g2": {"center": [0.583333, 1.0625], "radius": 0.12},
        "g3": {"center": [0.583333, 0.4375], "radius": 0.12},
        "g4": {"center": [-0.75, 0.4375], "radius": 0.12}
    }
}
```

Listing 3.3. Example of a cyclic task configuration.

The predicates shown in these examples are related to the specific ground domain used for the experiments. For our use cases, the definition of the relevant predicates will be provided in Chapter 4.

Chapter 4

Experimental Evaluation

In this chapter, we present the experimental evaluation of the implemented framework applied to a complex domain with temporally extended non-Markovian task specifications.
The experiments have been designed to investigate the effectiveness and applicability of the learning architecture under different task and abstraction configurations.
Throughout this chapter, we will firstly evaluate the framework on episodic tasks of increasing temporal complexity, then we will study the effect of introducing learning when multiple abstraction levels have been adopted, and finally we will consider its application on cyclic temporal objectives.

4.1 Evaluation Goals and Research Questions

The target of this experimental analysis is to identify possible limitations of the framework and to better understand its potential when increasingly complex tasks must be fulfilled. The experimental evaluation pipeline is organized around four key research questions, each one investigating a different aspect of the framework.

Our research questions are:

- **RQ1** To what extent can the theoretical framework be directly applied to a complex domain without requiring additional adaptations?
- **RQ2** How does the multi-level framework perform on episodic non-Markovian tasks with a growing temporal complexity?
- **RQ3** How does the introduction of learning on abstraction levels affect the guidance signal and the resulting learning process?
- **RQ4** Can the framework be applied to learn cyclic temporal behaviors that are not limited to purely episodic task structures?

These research questions guide the experimental analysis presented in the following sections, covering the practical limitations of the learning architecture, the effect of temporal-task complexity, the impact of learning on multi-level abstraction, and the extension to cyclic task structures.

4.2 Experimental Setting

4.2.1 Environment and Abstractions

To carry out the experimental evaluation we have chosen the Gymnasium *Lunar Lander* environment [17]. On top of it, the framework supports the possibility to equip different abstract grids used to generate the shaping guidance.

Gymnasium *Lunar Lander*

This environment is a classic rocket trajectory optimization problem. The agent is a lander with the goal to correctly land on the ground in between of a specified region. The agent is equipped with three engines that can be turned on/off during the interaction with the environment [6].

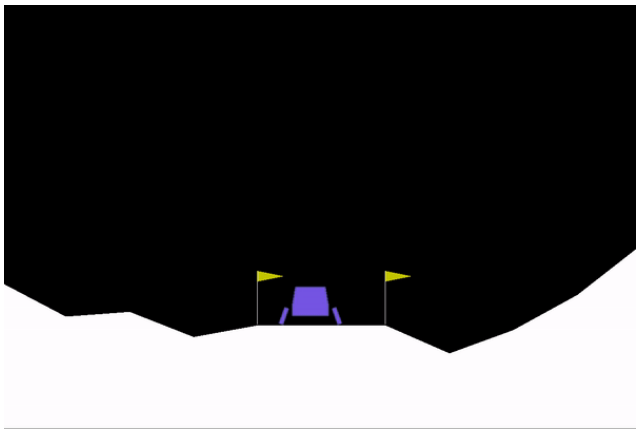


Figure 4.1. The Gymnasium *Lunar Lander* environment.

The state space is represented by an eight-dimensional observation vector where:

- x and y denote the horizontal and vertical position of the lander;
- v_x and v_y denote the linear velocities of the lander;
- θ denotes the lander orientation;
- ω denotes the angular velocity of the lander;
- c_l and c_r denote whether the left and right legs are in contact with the ground.

The action space is discrete and provides four different actions:

- 0 do nothing;
- 1 fire left orientation engine;
- 2 fire main engine;
- 3 fire right orientation engine.

The original *Lunar Lander* environment is characterized by a complex and dense reward function. For the purposes of this thesis, the original reward function is not used. Instead, the reward structure is replaced by a sparse goal-based formulation, so that the ground environment can be treated as a Goal MDP with respect to the temporal objective considered in each experiment.
For this reason, the details of the original *Lunar Lander* reward function are intentionally omitted. Moreover, the original landing objective is not considered: the goals adopted throughout the experimental evaluation are defined by the temporal task specifications described in the following sections. This requirement will be re-introduced only for the additional case studies.

Abstractions Grids

The abstract environments supported by the framework are represented through 2D discrete grids. Each grid provides a simplified representation of the ground *Lunar Lander* state space considering only its positional components. In particular, the abstraction is defined over the x and y components. The other physical quantities are intentionally discarded. The purpose of the abstraction is not to reproduce the complete dynamics of *Lunar Lander*, but to provide a simpler state representation from which task-related guidance can be automatically derived.

Formally, an abstraction grid is an MDP $\mathcal{M}_{\text{grid}} = \langle \mathcal{S}_{\text{grid}}, \mathcal{A}_{\text{grid}}, P_{\text{grid}}, R_{\text{grid}}, \gamma \rangle$ where:

- $\mathcal{S}_{\text{grid}} = \{0, \dots, W-1\} \times \{0, \dots, H-1\}$;
- $\mathcal{A}_{\text{grid}} = \{\uparrow, \downarrow, \leftarrow, \rightarrow \nwarrow, \nearrow, \swarrow, \searrow, \mathit{Done}\}$;
- $P_{\text{grid}} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \{0, 1\}$ is the deterministic transition function, defined as

$$P_{\text{grid}}(s' \mid s, a) = \begin{cases} 1 & \text{if } s' = T(s, a) \\ 0 & \text{otherwise} \end{cases}$$

  where T denotes the successor state obtained by applying action a to state s;
- $R_{\text{grid}} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the classical reward function used in a Goal MDP setting, as described in Section 2.3.1;
- $\gamma \in [0, 1]$ is the discount factor.

State Abstraction Mapping

The ground MDP and the abstraction built over it are linked together through several mapping functions φ_i, as described in Section 3.1.1.

In particular, two different mapping function types are required to correctly implement the hierarchy. The first mapping connects the continuous ground environment to the first abstract grid. It corresponds to a classical projection operation of a point in the continuous space to a cell in the abstract grid. The second mapping is instead defined between two consecutive abstract grid levels. Now, since both source and target states correspond to grid cells, the mapping is performed by projecting the center of the source cell to the target grid.

For the ground-to-grid mapping, let the continuous spatial domain be bounded by $x \in [x_{\min}, x_{\max}]$ and $y \in [y_{\min}, y_{\max}]$. Let the first abstract grid have width W and height H. Given a continuous position (x, y), the corresponding abstract cell (i, j) is computed as

$$i = clip\left(\left\lfloor \frac{x - x_{\min}}{x_{\max} - x_{\min}} W \right\rfloor, W - 1\right) \tag{4.1}$$

$$j = clip\left(\left\lfloor \frac{y - y_{\min}}{y_{\max} - y_{\min}} H \right\rfloor, H - 1\right) \tag{4.2}$$

In the experiments $x \in [-1, 1]$ and $y \in [0, 1.5]$. These intervals do not correspond to those of the real environment, but they correctly describe the visible area.

For the mapping between two consecutive abstraction grids, let the source grid have size $W_s \times H_s$ and the target grid size $W_t \times H_t$. Given a source cell (x, y), the center of the cell is projected onto the target grid according to

$$x' = clip\left(\left\lfloor \frac{x + 0.5}{W_s} W_t \right\rfloor, W_t - 1\right) \tag{4.3}$$

$$y' = clip\left(\left\lfloor \frac{y + 0.5}{H_s} H_t \right\rfloor, H_t - 1\right) \tag{4.4}$$

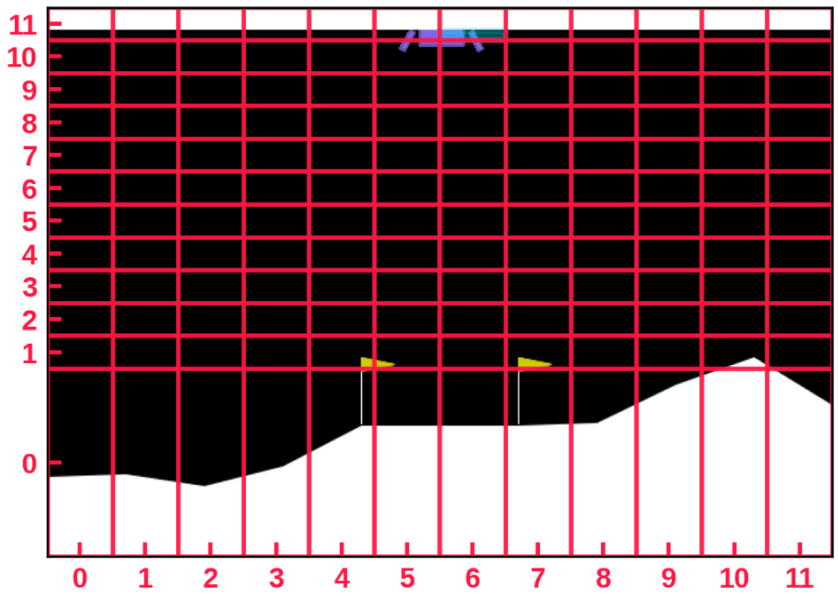


Figure 4.2. Example of a grid abstraction 12x12 over the *Lunar Lander* environment.

4.2.2 Temporal Tasks and Predicates

The temporal tasks considered in the experiments are defined over a set of spatial predicates, which are evaluated on the spatial coordinates (x, y) of the *Lunar Lander* environment. The set of atomic propositions used to define a task is named AP.

Supported Predicates

The framework supports three different predicate types:

- *circular region*: a circular predicate is defined specifying a center (c_x, c_y) and a radius r. The predicate is satisfied when

$$(x - c_x)^2 + (y - c_y)^2 \leq r^2$$

- *rectangular region*: a rectangular region is defined specifying the bounds $x_{\min}$, $x_{\max}$, $y_{\min}$, and $y_{\max}$. The predicate is satisfied when

$$x_{\min} \leq x \leq x_{\max} \quad \wedge \quad y_{\min} \leq y \leq y_{\max}$$

- *half planes*: a half-plane corresponds to a spatial condition with respect to a certain threshold along a coordinate axis. The framework supports half-planes along both axes.

Predicate mapping on Abstraction Grids

The predicates involved in the task specification are defined over the continuous ground state space. This fact guarantees that the task specification is completely independent from whatever abstraction grids have been adopted to solve the problem. In order to evaluate the same task specification over different levels in the hierarchy, each predicate needs to be rasterized onto the corresponding grid.

Formally, let R_p denote the continuous region associated with predicate p and let C_{ij} denote the rectangular area represented by the grid cell (i, j). The predicate is assigned to the cell whenever:

$$C_{ij} \cap R_p \neq \emptyset \tag{4.5}$$

This criterion corresponds to an over-approximation of the original continuous region, since a predicate is considered true in the whole cell even when only part of the cell intersects the corresponding region. This is done to test the reliability of the framework when the abstractions are not fully precise. Moreover, the rasterization task is performed for each level in the hierarchy starting from the continuous domain. This choice has been intentionally adopted to avoid the over-approximation error propagation between abstraction levels.

At level k, the result of the rasterization is encoded through a labeling function

$$L_k : \mathcal{S}_k \to 2^{AP} \tag{4.6}$$

defined as

$$L_k(i, j) = \left\{ p \in AP \mid C_{ij}^{(k)} \cap R_p \neq \emptyset \right\} \tag{4.7}$$

Augmented state representation

The spatial state alone is not sufficient to represent the progress of a non-Markovian temporal task. For this reason, at each level in the hierarchy, we need to consider the augmented version of the state space, as described in Section 2.2.4.

After each transition, the predicates that hold in the successor state are evaluated through the corresponding labeling function. Let $\sigma \subseteq AP$ denote the set of atomic propositions that are true after the transition. The automaton state is then updated according to

$$q' = \delta(q, \sigma) \tag{4.8}$$

Therefore, a transition at abstraction level i can be represented as

$$(x_i, y_i, q) \xrightarrow{a} (x_i', y_i', q') \tag{4.9}$$

with

$$q' = \delta\left(q, L_i(x_i', y_i')\right) \tag{4.10}$$

At this point, we need to consider also an extension to the mapping function φ_i. We know that the logical task progress must remain aligned across all levels. Hence, we can extend the mapping function as follows:

$$\tilde{\varphi}_i(x_i, y_i, q) = (\varphi(x_i, y_i), q) \tag{4.11}$$

Furthermore, each time a new episode starts we label the initial state s_0 before the first action is executed. In this way, the DFA can account for predicates that are already satisfied at the beginning of the interaction.

4.2.3 Learning and Evaluation Setup

The experiments presented throughout this chapter are carried out adopting a common architecture, while the specific hyperparameters may vary depending on the considered experimental scenario.

Learning Algorithms

The framework is equipped with a set of learning algorithms: DDQN, Dueling-DDQN and Tabular Q-Learning. All of them have been described in Section 2.1.4.

The ground MDP is characterized by a continuous observation space and therefore requires a neural approach to estimate the action-value function. For this reason the choice was DDQN.
Instead, at the abstract levels, the state spaces are discrete and significantly smaller. For this reason, Tabular Q-Learning has been adopted.
The two learning procedures are configured independently, since the ground and abstract MDPs differ substantially in terms of state-space representation and complexity. The main hyperparameters and configurations adopted for DDQN and Tabular Q-Learning are reported in the following tables.

Parameter	Value
Network architecture	MLP $(128, 128)$
Hidden activation	ReLU
Input dimension	$8 + \|Q\|$
Output dimension	4
Optimizer	Adam [10]
Learning rate	10^{-3}
Loss function	MSE
Batch size	64
Replay-buffer capacity	300 000 transitions
Replay sampling	Uniform
Discount factor γ	0.99
Target-network update	Polyak averaging
Polyak coefficient τ	0.005

Table 4.1. DDQN configuration for learning on ground MDP

Parameter	Value
Learning rate α	0.1
Discount factor γ	0.99
Q-value initialization	0
State initialization	Uniform random restart
Update frequency	One update per transition

Table 4.2. Q-learning configuration for learning on abstract levels.

During the learning episodes, in both cases, the classical ϵ-greedy strategy has been adopted. Its configuration may depend on the specific experiment. Furthermore, in all the experiments, the reward assigned when a task is completed is set to 10000. From now on, this reward is referred to as *task reward* or *environment reward*.

Evaluation Procedure and Metrics

During the learning procedures, at each level, several training and evaluation metrics have been monitored. In particular, the performance of the agent is monitored during training through periodic greedy evaluations, in which exploration is disabled. At the end of each learning procedure the results are two different policies:

- **best policy** which is selected among all the checkpoints evaluated during the training task;
- **last policy** which corresponds to the policy resulting after executing the whole training task.

The main metrics adopted throughout the experimental evaluation are:

- **task reward**: cumulative reward associated with the temporal task during an episode. It does not involve the shaping signal;

- **shaping reward**: cumulative contribution generated by the abstraction-based shaping signal.
- **success rate**: fraction of evaluation episodes in which the temporal objective is successfully completed. For cyclic tasks this means that at least one cycle has been completed during the interaction.
- **episode length**: number of interaction steps performed before termination;
- **DFA progression**: statistics related to the automaton states and transitions visited during the interaction.

When multiple seeds are considered, results are aggregated across runs by reporting the mean value together with a variability band representing the standard deviation. Furthermore, to select the best policy among the periodically evaluated checkpoints, a hierarchical criterion is adopted:

1. highest success rate;
2. highest mean task reward. For cyclic tasks, the latter corresponds to the highest mean number of completed cycles;
3. lowest mean episode length;
4. earliest checkpoint.

Setup

All experiments are conducted on a machine running Ubuntu 24.04.1 LTS equipped with an AMD Ryzen 9 7950X processor, 124 GiB of RAM, and two NVIDIA GeForce RTX 4090 GPUs, each providing 24 GB of VRAM. The experiments were executed within a Docker container with Python 3.11, PyTorch 2.7.1 with CUDA 12.8 support, Gymnasium 1.2.1, NumPy 2.2.6, pandas 2.3.2, and Matplotlib 3.10.6.

4.3 RQ1: Direct Applicability of the Framework

This section investigates whether the framework is directly applicable to the considered ground domain. These experiments are organized to analyze two important aspects of the implemented learning architecture. The first aspect studied is the interaction between the biased and unbiased learners. Then, the second one is aimed to study the real impact of the shaping guidance on the learning process.

4.3.1 Biased-Unbiased Interaction

We recall from Section 3.2.2 that the architecture employs two learners with different objectives. The biased learner interacts with the environment using the shaped reward (task reward augmented by the shaping signal), while the unbiased learner is updated on the same experience generated by the biased one, but considering only the original task reward.

To isolate the effect of this coupling, the settings considered are:

- *1-phase trajectory* with unbiased standalone learner;
- *1-phase trajectory* with coupled learner architecture;
- *2-phase trajectory* with unbiased standalone learner;
- *2-phase trajectory* with coupled learner architecture.

1-Phase Trajectory

This task corresponds to a trajectory composed of only one phase: reaching a specific region of the environment. The corresponding LTL_f formula is:

$$\varphi = \Diamond g_1$$

where g_1 is the predicate associated with the target region.

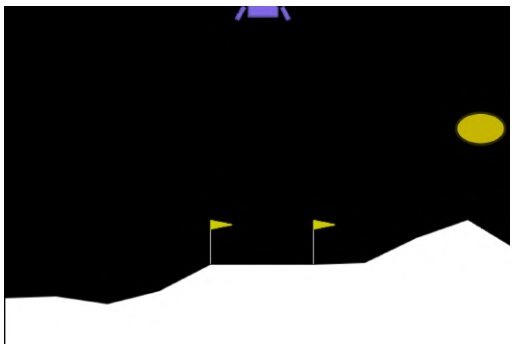


Figure 4.3. 1-phase task.

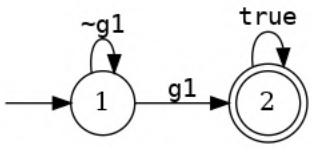


Figure 4.4. DFA for the 1-phase task.

2-Phase Trajectory

This task corresponds to a trajectory composed of two phases: reaching a first waypoint and then the target region of the environment. The corresponding LTL_f formula is:

$$\varphi = \Diamond\left(\text{wp}_1 \wedge \circ\left(\Diamond g_1\right)\right)$$

where wp_1 and g_1 are the predicates associated with the waypoint and the target region, respectively.

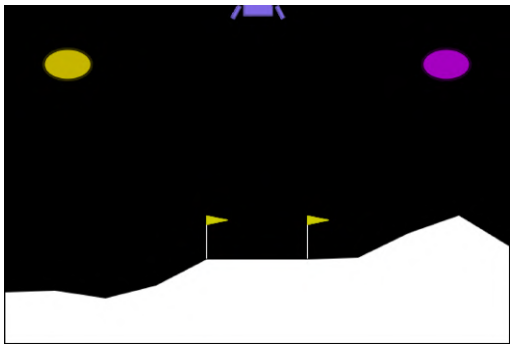


Figure 4.5. 2-phase task. In yellow wp_1 and in purple g_1.

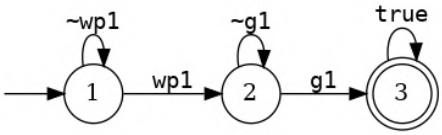


Figure 4.6. DFA for the 2-phase task.

Hyperparameters

Here, the common hyperparameters adopted for these experiments are reported.

Parameter	Value
Episodes	10000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	5
Grid Dimension	12×12
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes	1000

Table 4.3. Hyperparameters adopted in the experiments.

The grid resolution affects only the experiments with the coupled learner.

Results and Interpretations

We evaluate the outcome of these experiments by considering the evaluation performance of the various learners during the training procedure.

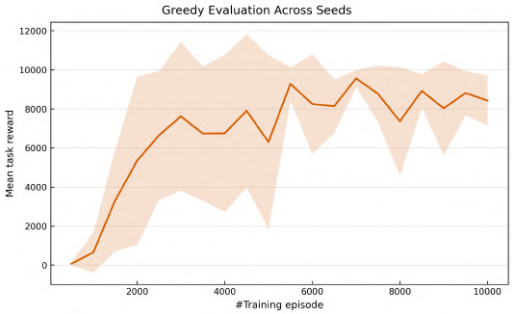


Figure 4.7. 1-phase task. Periodically greedy evaluation of the standalone unbiased policy.

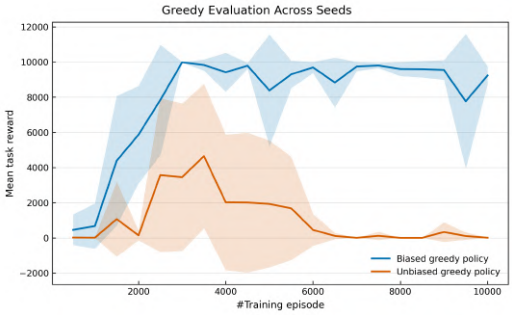


Figure 4.8. 1-phase task. Periodically greedy evaluation of the biased and unbiased policies.

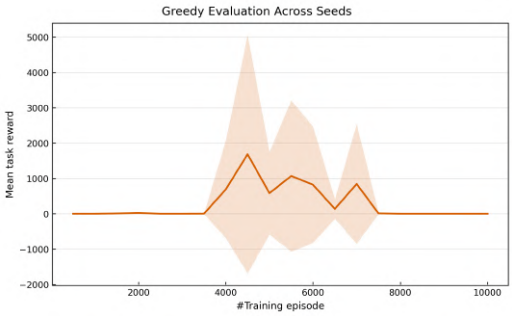


Figure 4.9. 2-phase task. Periodically greedy evaluation of the standalone unbiased policy.

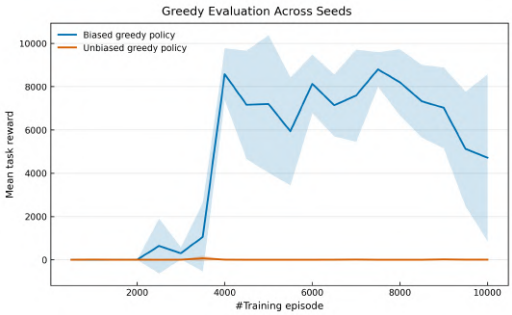


Figure 4.10. 2-phase task. Periodically greedy evaluation of the biased and unbiased policies.

The results obtained provide a first indication of a practical limitation in the coupled learning architecture. Specifically, in the 1-phase setting, the standalone unbiased learner was able to learn a policy that fulfills the task. On the other hand, in the coupled architecture, the unbiased learner does not maintain a stable and effective policy throughout training, even though the biased learner successfully solves the task.

More precisely, during the early stages of training, the unbiased learner was able to discover a useful policy but it didn't manage to preserve it until the end. Since the main difference with respect to the standalone case lies in the source of the collected experience, this is a strong suggestion that the biased learner strongly influences the data distribution seen by the unbiased one and the introduced effect is catastrophic.

Moving on to the 2-phase setting, we immediately notice that the required task is too difficult to be learned by the standalone unbiased learner. The only learner that was able to build a strong and useful policy is the biased one. Therefore, we can deduce that the shaping information extracted from the 12×12 grid was crucial to solve the task.

4.3.2 Shaping Guidance

We recall the shaping formulation used in the framework, from Section 3.1.2:

$$F(s, a, s') = \gamma V(s') - V(s)$$

From now on, we will refer to the γ factor involved in the formula as γ_s.

To analyze the shaping guidance quality, we focus on the impact of γ_s. In particular, the tasks considered are:

- *2-phase trajectory* with $\gamma_s = 0.99$;
- *2-phase trajectory* with $\gamma_s = 1$.

The *2-phase trajectory* considered is the one already studied in the previous section.

Hyperparameters

Here, the common hyperparameters adopted for these experiments are reported.

Parameter	Value
Episodes	10000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	5
Grid Dimension	12×12
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes	1000

Table 4.4. Hyperparameters adopted in the experiments.

Results and Interpretations

The analysis has been carried out on the metrics related to the biased learner's performance. We already know that the unbiased learner is not able to learn this task.

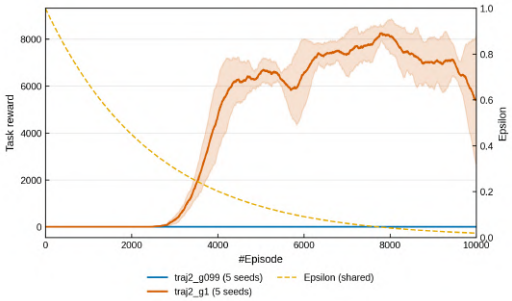


Figure 4.11. 2-phase task. Training task reward of the biased learner with $\gamma_s = 0.99$ and $\gamma_s = 1$. The dashed curve shows ϵ.

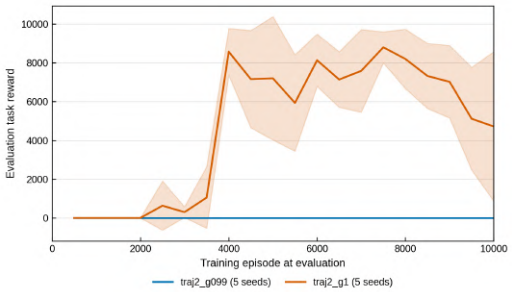


Figure 4.12. 2-phase task. Periodic greedy evaluation of the biased policy with $\gamma_s = 0.99$ and $\gamma_s = 1$.

γ_s	Best success rate (%)	Same-state fraction (%)	Same-state reward per transition	Same-state reward per episode
0.99	0.00 ± 0.00	82.39 ± 0.96	-89.51 ± 0.05	-5000.60 ± 53.21
1.00	96.94 ± 1.91	91.48 ± 0.22	0.00 ± 0.00	0.00 ± 0.00

Table 4.5. Comparison of shaping discounts on the 2-phase task: best-policy success rate and same-state transition statistics.

The results show a clear difference between the two shaping configurations. With $\gamma_s = 0.99$, none of the five runs is able to learn a successful policy, whereas setting $\gamma_s = 1$ leads to a best evaluation success rate of $96.94\% \pm 1.91\%$. Furthermore, in both configurations, we notice that most of the time (expressed in transitions) the agent performs actions remaining in states mapped to the same abstract one. This fact is strictly related to the ground environment dynamics and the abstraction adopted over it. A grid abstraction does not capture any detail about the physics of the lander, intentionally. Hence, after each transition, with high probability the agent remains in the same abstract cell.

Let's consider the situation in which the abstract cell does not change after a transition. Formally, the state s and s' involved in the shaping formula are the same. Then:

$$F(s, a, s') = (\gamma_s - 1)V(s)$$

Therefore, every transition receives a negative shaping signal whenever $V(s) > 0$ and $\gamma_s < 1$. This generates a high cumulative penalty over a single episode when $\gamma_s = 0.99$. Instead, with $\gamma_s = 1$, we neutralize this effect on the reward signal.

Overall, this behavior can be interpreted as a temporal misalignment between the ground MDP and the abstract model.

Ground-level transitions occur at a much finer temporal resolution than transitions that effectively change the abstract state. Consequently, several consecutive ground interactions may correspond to the same abstract cell and therefore to the same potential value. This effect leads to a non-informative shaping guidance for the agent.

4.3.3 Discussion

The experiments presented in this section highlight two practical aspects of the framework related to its application on a complex domain.

First, the coupled biased-unbiased learner architecture does not reliably transfer the benefits obtained by the biased policy to the unbiased one. The results suggest that this behavior is strictly related to the data distribution induced by the biased learner. In fact, on a very simple task like the *1-phase trajectory*, the unbiased standalone learner was able to learn a successful policy. Hence, for this task, we can exclude that the problem was the sparse-reward setting. However, for more complex tasks, the shaping guidance is still necessary.

The second issue is about the temporal misalignment between the ground domain and the abstract one. More generally, this issue can happen between two consecutive layers in the hierarchy. Intentionally, in this thesis, we have adopted a very trivial abstraction over a complex environment for the following reasons:

- at this moment, we do not have any automatic mechanism to design the abstraction levels;
- in this setting, it was not clear how to integrate the complex physics of *Lunar Lander* into the abstraction levels.

In particular, the temporal misalignment emerges when the changing state frequency between two consecutive levels is very different. As happened in the current setting, this could lead to the generation of artificial penalties received by the agent, also due to the mathematical structure of the shaping formula.

From these considerations, we continue the experimental evaluation process adopting the following choices:

- we will evaluate the performance over the various tasks considering the biased learner at the ground MDP.
  In [4], this choice was already adopted when the authors studied the framework on a robotic domain. In particular, they say: "*In practice, in the considered robot control problems (modeled as Goal MDPs), the main challenge is to compute a policy achieving the goal with high sample efficiency, rather than finding an optimal one.*"[1];
- we will set $\gamma_s = 1$ to neutralize the consequences of temporal misalignment.

[1] Extracted from [4], Section 7, page 7.

Theoretical consequences of the assumptions

First of all, by considering the biased learner as the final ground-level policy, we accept that the learned policy may differ from the optimal one induced by the original reward function. We can accept the risk because we have guarantee that the abstractions provided are synchronized and coherent with the task specification on the ground MDP.

Furthermore, adopting $\gamma_s = 1$, we lose the PBRS policy invariance property. This guarantee was already relaxed in [4]. However, this choice is asymmetrical, in the sense that the discount factor γ is different from γ_s.
Hence, it is no longer true that

$$G = \sum_{t=0}^{T-1} \gamma^t \left(\gamma V(s_{t+1}) - V(s_t)\right) = -V(s_0) + \gamma^T V(s_T)$$

because the new formulation is

$$G = \sum_{t=0}^{T-1} \gamma^t \left(\gamma_s V(s_{t+1}) - V(s_t)\right) = \sum_{t=0}^{T-1} \gamma^t \left(V(s_{t+1}) - V(s_t)\right)$$

and the intermediate potential terms no longer cancel in the sum.
Therefore, the cumulative shaping return depends also on the intermediate potential values visited along the trajectory.

4.4 RQ2: Performance under Increasing Temporal Complexity

The experiments shown in this section investigate how the framework behaves when the temporal complexity of the task increases.

The *2-phase trajectory* introduced in the previous section represents the starting point of this analysis. From this task specification, we increase the task complexity by requiring the satisfaction of longer and more structured trajectories.
More precisely, the task specifications considered in this section are:

- *2-phase trajectory*;
- *3-phase trajectory* in three different versions: simple, hard and extreme;
- *4-phase trajectory*;
- *5-phase trajectory*.

Each task will be tested using a single abstraction level over the ground MDP, solved with Value Iteration. In particular, the grids used in these experiments are:

$$6 \times 6 \quad 12 \times 12 \quad 24 \times 24 \quad 48 \times 48 \quad 180 \times 180 \quad 360 \times 360$$

These settings allow us to investigate if there exist a sort of correlation between the grid precision and the task complexity. In other words, we are investigating if more precise grids provide a better guidance useful to fulfill more difficult task.

4.4.1 2-Phase Trajectory

The first task considered is the *2-phase trajectory*, already presented in Section 4.3.1.

Hyperparameters

Here, the common hyperparameters adopted for these experiments are reported.

Parameter	Value
Episodes	8000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	10
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes during learning	1000
Eval. episodes after learning	1000

Table 4.6. Hyperparameters adopted in the experiments.

Results and Interpretations

We evaluate these experiments by considering the learning curves and evaluation performance. In particular, we will consider performance during both the training phase and post-training, using different evaluation seeds.

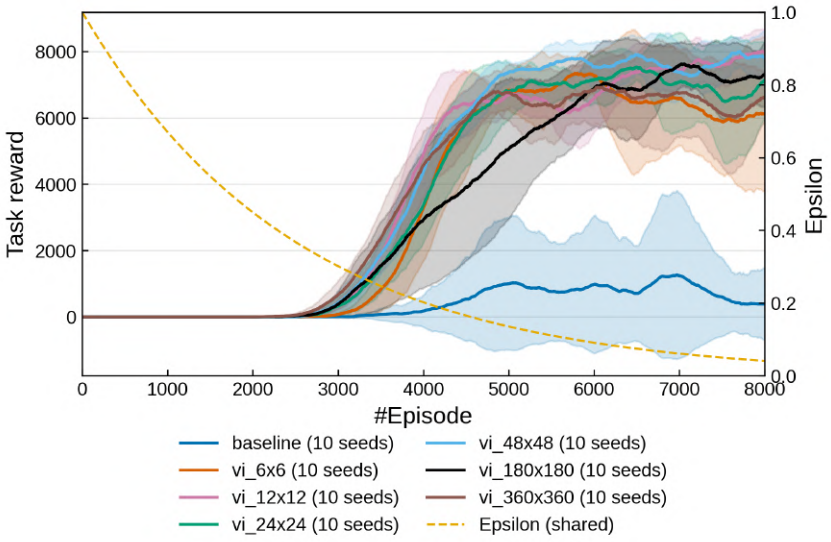


Figure 4.13. 2-phase trajectory. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

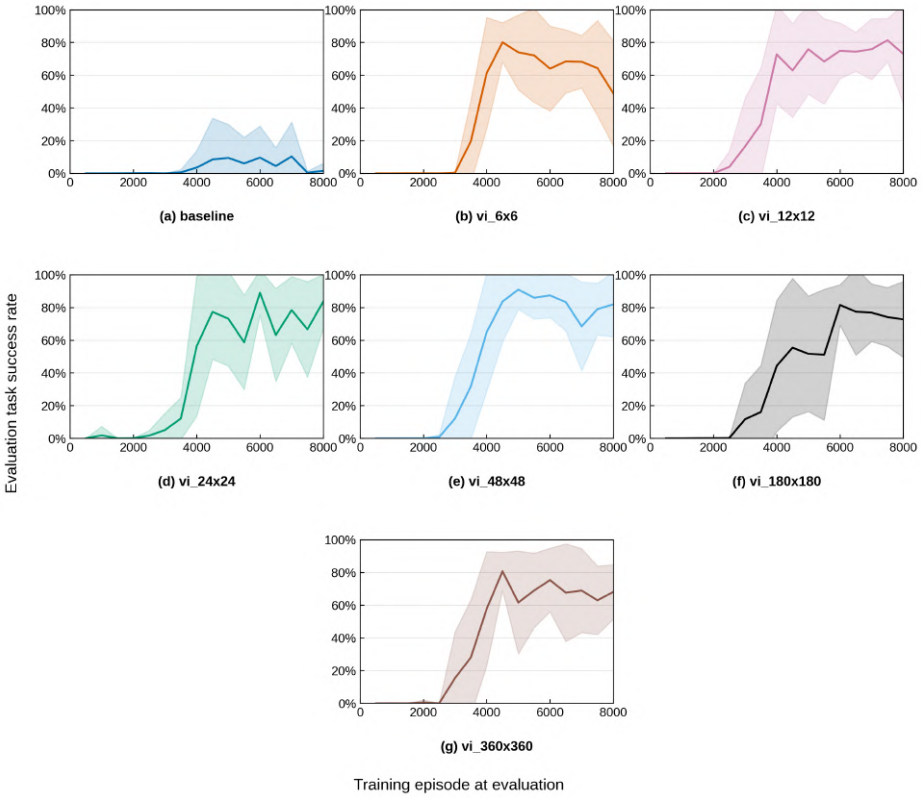


Figure 4.14. 2-phase trajectory. Periodic evaluation success rate during training for the six abstraction grid resolutions.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$16.10 \pm 30.52\,\%$	$1.99 \pm 5.97\,\%$
6×6	$91.89 \pm 10.15\,\%$	$49.20 \pm 31.81\,\%$
12×12	$97.42 \pm 1.83\,\%$	$73.53 \pm 29.99\,\%$
24×24	$96.81 \pm 3.71\,\%$	$84.33 \pm 15.18\,\%$
48×48	$98.26 \pm 0.99\,\%$	$82.55 \pm 19.05\,\%$
180×180	$95.08 \pm 7.10\,\%$	$73.07 \pm 23.50\,\%$
360×360	$97.01 \pm 2.25\,\%$	$69.38 \pm 17.28\,\%$

Table 4.7. Post-learning evaluation for Trajectory 2. Success rates are reported as mean $\pm$ standard deviation across 10 seeds, with each policy evaluated over 1000 episodes.

The learning curves highlight the importance of the abstraction-based guidance. The baseline configuration, without any shaping signal, is not able to reliably learn a successful policy that solves the task. On the other hand, all other configurations were able to find a policy.

Furthermore, Figure 4.13 exhibits a largely comparable learning dynamic across different grid resolutions. At this moment, no clear relationship emerges between grid precision and quality of the obtained policy. This fact is also confirmed by the post-learning evaluation, in which all abstraction-based best policies reach a success

rate greater than 90%.

Analyzing the variance of the periodic evaluations performed during the training phase and noticing the differences between the performance of the best and last policies, we have empirical evidence that the best-policy selection mechanism is crucial. In each setting, the last policy performs worse than the corresponding best policy.

4.4.2 3-Phase Trajectory

This task corresponds to a trajectory composed of three phases: reaching the first waypoint, then the second waypoint, and finally the target region. The LTL_f formula is:

$$\varphi = \Diamond\left(\mathrm{wp}_1 \wedge \circ\left(\Diamond\left(\mathrm{wp}_2 \wedge \circ\left(\Diamond g_1\right)\right)\right)\right)$$

where wp_1 and wp_2 are the waypoint predicates, and g_1 is the final target predicate.

The corresponding automaton is the one depicted in the following figure.

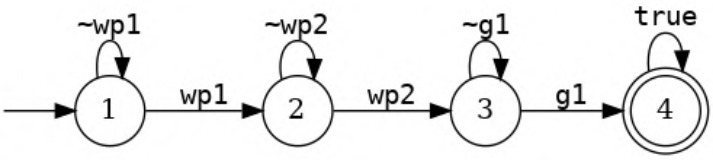


Figure 4.15. DFA for the 3-phase trajectory.

Task Variants

The same temporal specification is tested with three different spatial configurations. In particular, the differences between these versions correspond to the second waypoint's spatial position, as depicted in the following figures.

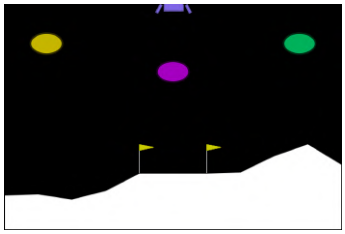


Figure 4.16. 3-phase trajectory: Simple.

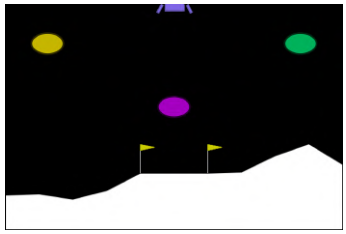


Figure 4.17. 3-phase trajectory: Hard.

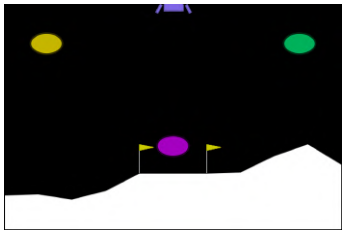


Figure 4.18. 3-phase trajectory: Extreme.

As we can see, the idea is to increase the difficulty associated with the required trajectory in order to stress the framework evaluating the resulting policies.

Hyperparameters

The hyperparameters adopted for these experiments are the same as those reported in Table 4.6. They are shared for all three variants of the task.

Training Results – Simple

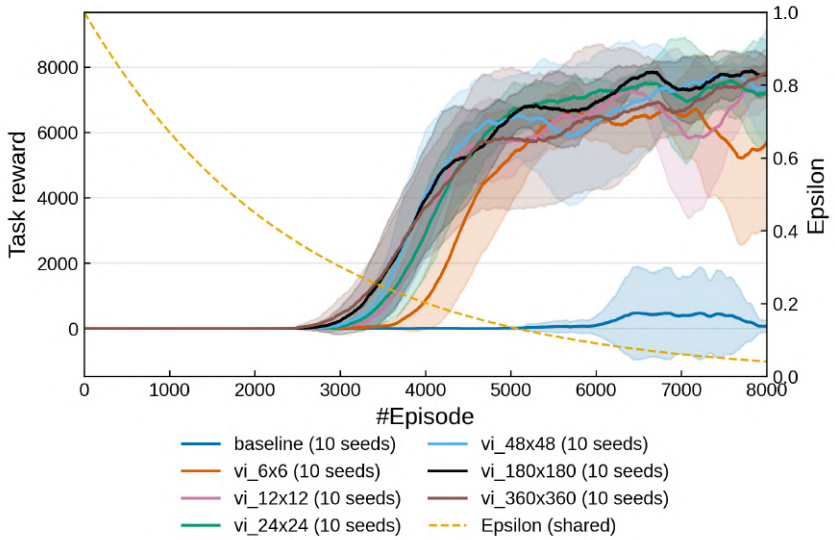


Figure 4.19. 3-phase trajectory – Simple. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

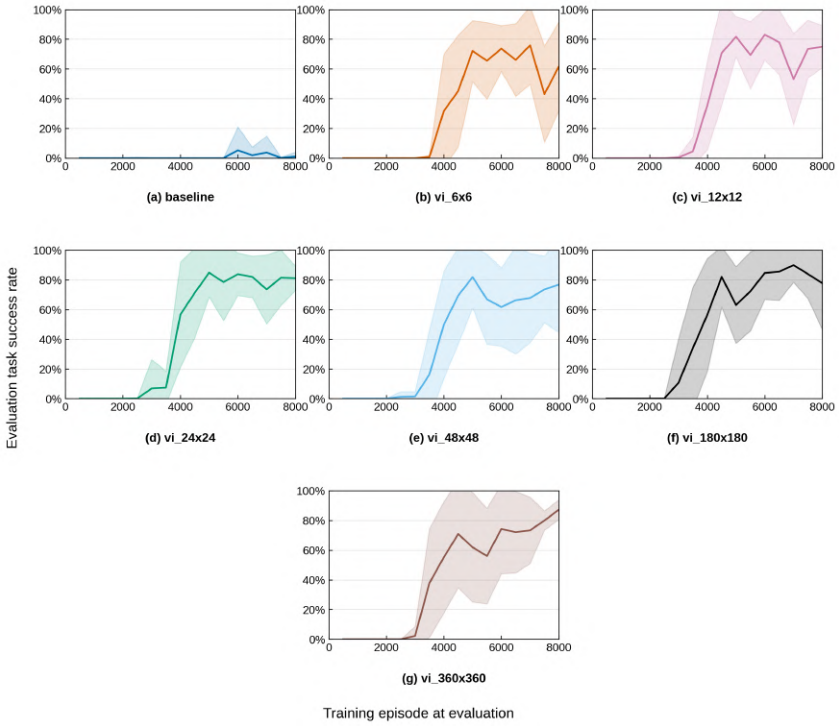


Figure 4.20. 3-phase trajectory – Simple. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Training Results – Hard

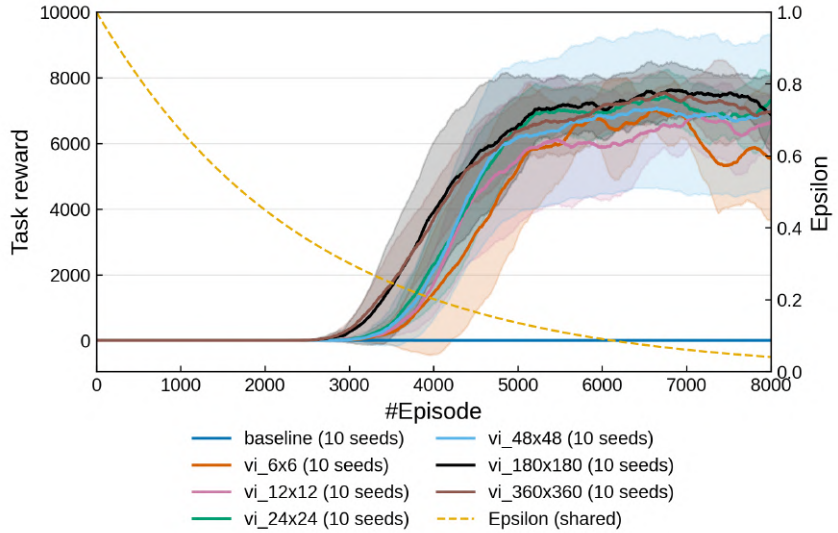


Figure 4.21. 3-phase trajectory – Hard. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

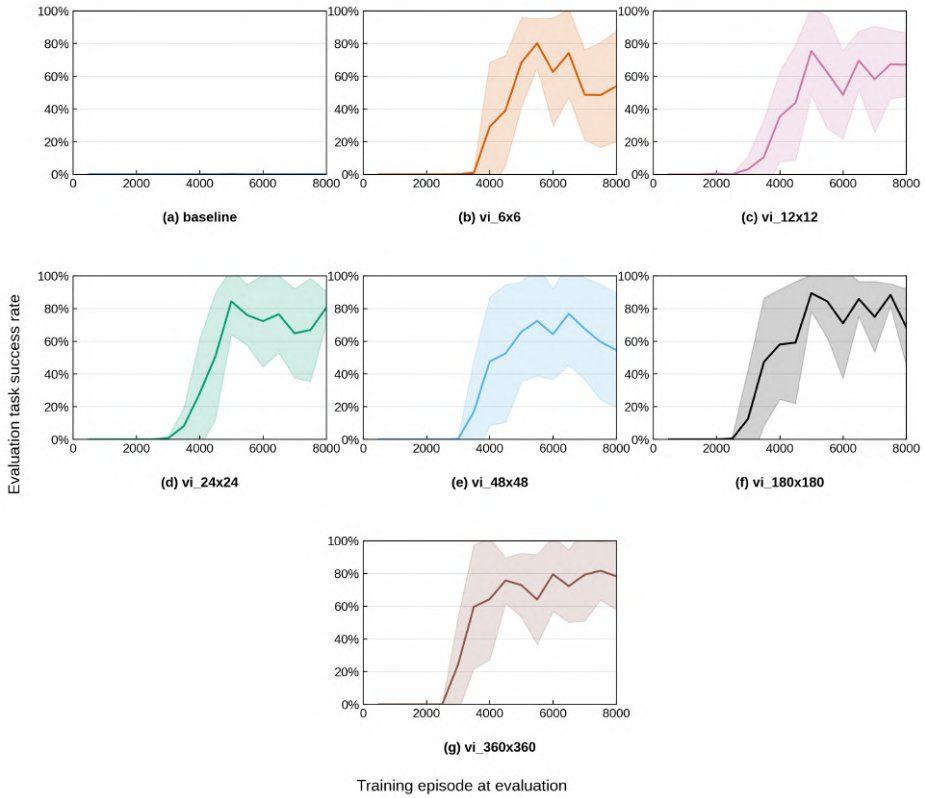


Figure 4.22. 3-phase trajectory – Hard. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Training Results – Extreme

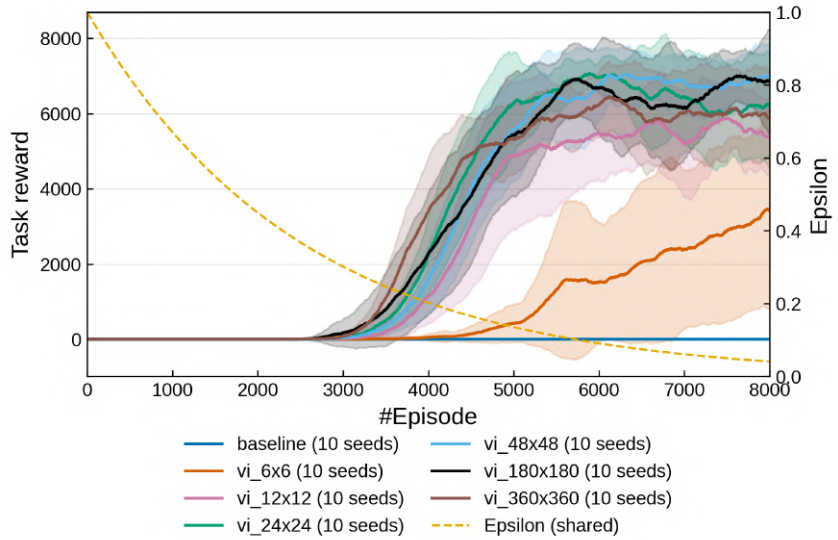


Figure 4.23. 3-phase trajectory – Extreme. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

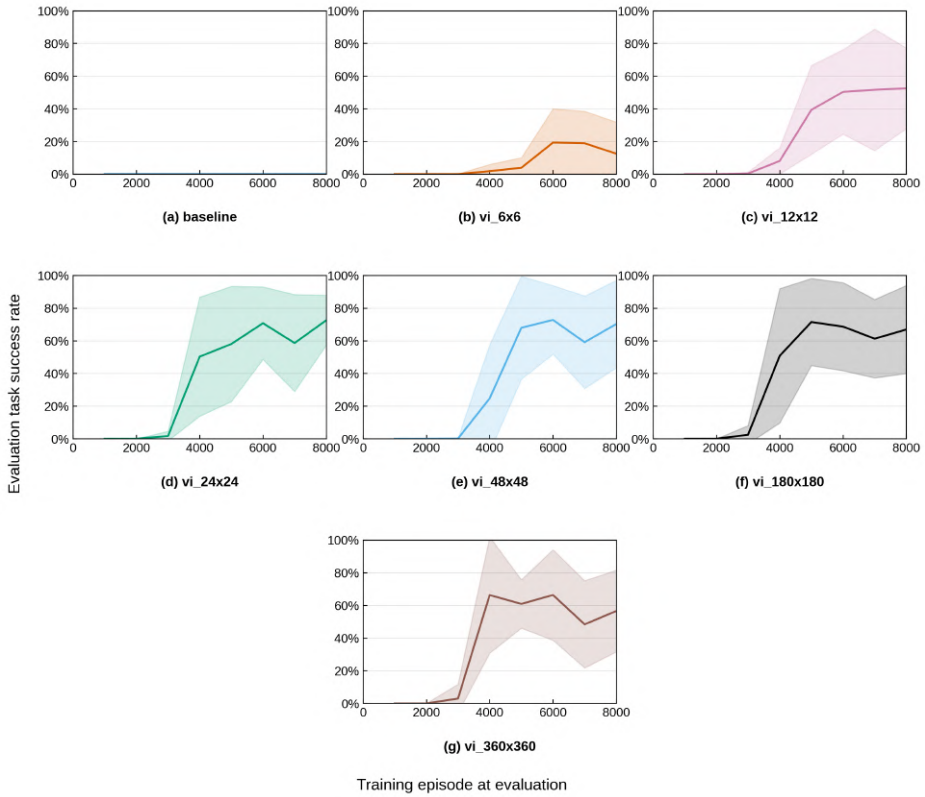


Figure 4.24. 3-phase trajectory – Extreme. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Post-Training Evaluation

Abstraction	Best policy success rate	Last policy success rate
Baseline	$5.3 \pm 15.9\,\%$	$0.9 \pm 2.7\,\%$
6×6	$95.0 \pm 2.3\,\%$	$62.7 \pm 30.5\,\%$
12×12	$94.1 \pm 5.6\,\%$	$75.3 \pm 14.6\,\%$
24×24	$98.2 \pm 1.3\,\%$	$82.4 \pm 8.0\,\%$
48×48	$98.3 \pm 1.6\,\%$	$76.9 \pm 32.5\,\%$
180×180	$98.8 \pm 1.8\,\%$	$78.2 \pm 31.4\,\%$
360×360	$96.5 \pm 2.1\,\%$	$87.5 \pm 5.8\,\%$

Table 4.8. Post-training evaluation results on the Simple variant. Values report the mean success rate and standard deviation across 10 training seeds, using 1000 evaluation episodes per policy.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$0.1 \pm 0.4\,\%$	$0.0 \pm 0.0\,\%$
6×6	$95.0 \pm 6.1\,\%$	$54.6 \pm 34.0\,\%$
12×12	$93.1 \pm 4.3\,\%$	$67.7 \pm 19.4\,\%$
24×24	$97.0 \pm 2.5\,\%$	$81.3 \pm 9.1\,\%$
48×48	$88.0 \pm 28.6\,\%$	$55.0 \pm 35.1\,\%$
180×180	$98.6 \pm 2.0\,\%$	$68.9 \pm 22.9\,\%$
360×360	$97.1 \pm 2.9\,\%$	$78.2 \pm 20.7\,\%$

Table 4.9. Post-training evaluation results on the Hard variant. Values report the mean success rate and standard deviation across 10 training seeds, using 1000 evaluation episodes per policy.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$0.0 \pm 0.0\,\%$	$0.0 \pm 0.0\,\%$
6×6	$28.3 \pm 23.9\,\%$	$12.4 \pm 19.7\,\%$
12×12	$77.8 \pm 11.3\,\%$	$53.1 \pm 24.9\,\%$
24×24	$93.2 \pm 5.3\,\%$	$73.7 \pm 15.1\,\%$
48×48	$87.6 \pm 13.0\,\%$	$70.1 \pm 26.9\,\%$
180×180	$90.1 \pm 11.2\,\%$	$67.3 \pm 26.9\,\%$
360×360	$91.6 \pm 5.3\,\%$	$58.2 \pm 25.1\,\%$

Table 4.10. Post-training evaluation results on the Extreme variant. Values report the mean success rate and standard deviation across 10 training seeds, using 1000 evaluation episodes per policy.

Results Interpretation

The results show that the abstraction-based guidance is still effective for all the spatial configurations tested, consistently outperforming the baseline.

For the *Simple* and *Hard* variants, most grid resolutions lead to similar final performance. Nevertheless, from the learning curves, we notice that the ones associated with a finer abstraction tend to reach the plateau earlier during the training phase. The trend becomes more evident in the *Extreme* variant, where the coarsest resolutions like the 6×6 and 12×12 learn slowly and achieve lower performance. This is also confirmed by the periodically greedy evaluations performed during training.

Analyzing the post-training evaluation results for the *Hard* variant in greater depth, we find that the worst configuration (apart from the baseline) involves the 48×48 abstraction grid. In reality, the results shown for this configuration are strongly influenced by a single seed in which the learner completely failed to discover a successful policy. The other runs achieve performance comparable to that obtained with other abstraction grids. This outlier raises concerns about whether the agent can reliably discover a useful policy. The abstraction built over the ground MDP cannot provide us with full certainty that the agent will discover the required trajectory.

Since the three variants share the same temporal specification, the observed differences cannot be linked to the temporal complexity itself, but rather to the increasing spatial difficulty of the required trajectory. Overall, the results suggest that there is a benefit of using a more precise abstraction and this becomes more relevant when finer spatial information is required to complete the task. Nevertheless, the relation between grid precision and policy quality does not appear monotonic. Rather, the results suggest the existence of a task-dependent minimum resolution. Below this threshold, the abstraction may be too coarse to provide sufficiently informative guidance.

4.4.3 4-Phase Trajectory

This task corresponds to a trajectory composed of four phases: reaching the first waypoint, then the second waypoint, then the third waypoint and finally the target region. The LTL_f formula is:

$$\varphi = \Diamond\left(\mathrm{wp}_1 \wedge \circ\left(\Diamond\left(\mathrm{wp}_2 \wedge \circ\left(\Diamond\left(\mathrm{wp}_3 \wedge \circ\left(\Diamond g_1\right)\right)\right)\right)\right)\right)$$

where wp_1, wp_2 and wp_3 are the waypoint predicates, and g_1 is the final target predicate.

The corresponding automaton is the one depicted in the following figure

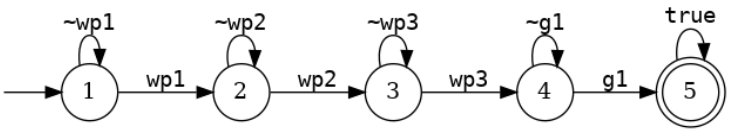


Figure 4.25. DFA for the 4-phase task.

and the required trajectory is:

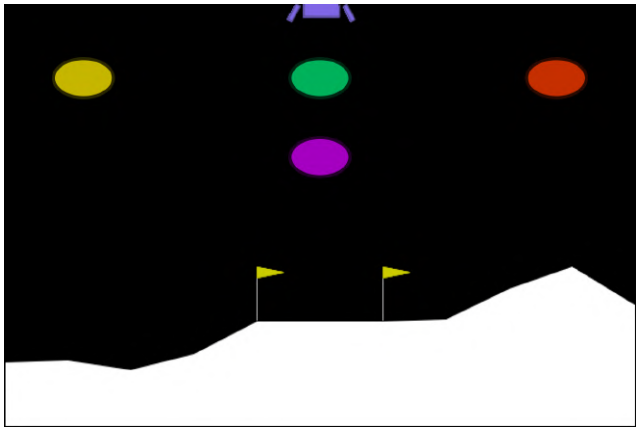


Figure 4.26. 4-phase task: yellow → purple → green → red.

Hyperparameters

Here, the common hyperparameters adopted for these experiments are reported.

Parameter	Value
Episodes	12000
ϵ decay	0.9998
ϵ minimum	0.01
Seeds	10
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes during learning	1000
Eval. episodes after learning	1000

Table 4.11. Hyperparameters adopted in the experiments.

Results and Interpretations

As a first stage, we evaluate these experiments by considering the learning curves and the evaluation performance. In particular, we consider performance during both the training phase and post-training, using different evaluation seeds. Then, we move on to analyzing the results for each abstraction grid in greater depth, given the presence of outlier seeds.

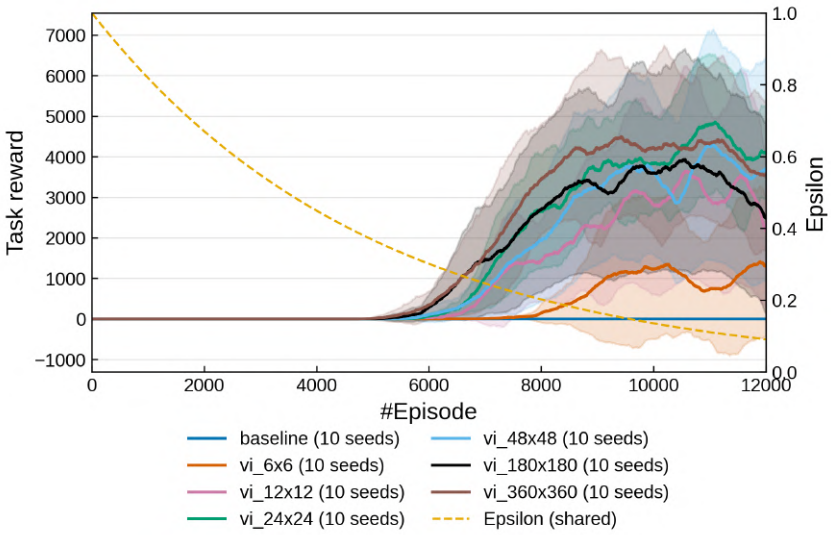


Figure 4.27. 4-phase trajectory. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

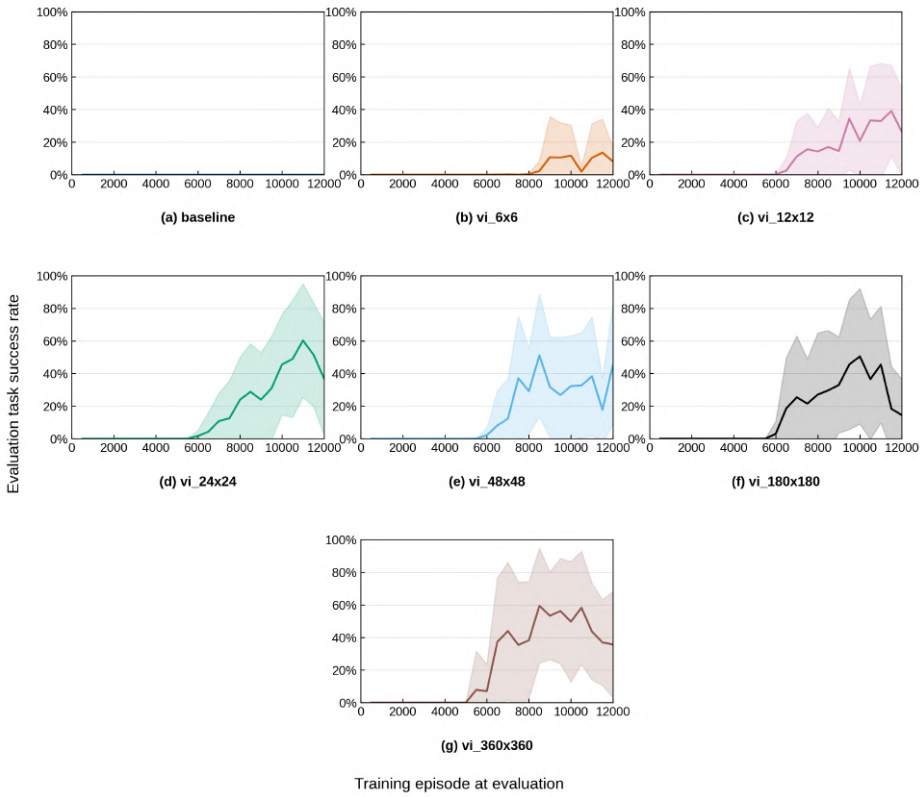


Figure 4.28. 4-phase trajectory. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$0.0 \pm 0.0\,\%$	$0.0 \pm 0.0\,\%$
6×6	$24.8 \pm 31.7\,\%$	$8.1 \pm 9.2\,\%$
12×12	$53.5 \pm 37.3\,\%$	$26.1 \pm 26.1\,\%$
24×24	$83.6 \pm 15.0\,\%$	$36.8 \pm 34.5\,\%$
48×48	$74.9 \pm 37.6\,\%$	$45.9 \pm 37.6\,\%$
180×180	$64.2 \pm 42.3\,\%$	$14.7 \pm 21.6\,\%$
360×360	$85.1 \pm 28.6\,\%$	$35.7 \pm 32.8\,\%$

Table 4.12. Four-phase trajectory. Post-training evaluation results for the best and last policies over ten training seeds and 1000 evaluation episodes per policy.

The results presented so far reveal a significant increase in the variance characterizing the performance of the best policies, as shown in Table 4.12. A deep analysis of the individual runs shows that the variance is strongly influenced by the presence of outlier seeds. The presence of such cases is not completely unexpected, considering that they were also observed in the *3-phase* experiments and that the new task is more difficult compared with the previous one. For this reason, the mean success rate alone is no longer enough to correctly characterize the performances of the policies obtained. Hence, we continue the analysis by studying the performance obtained after excluding the identified outliers and investigating the differences between successful and failed training runs.

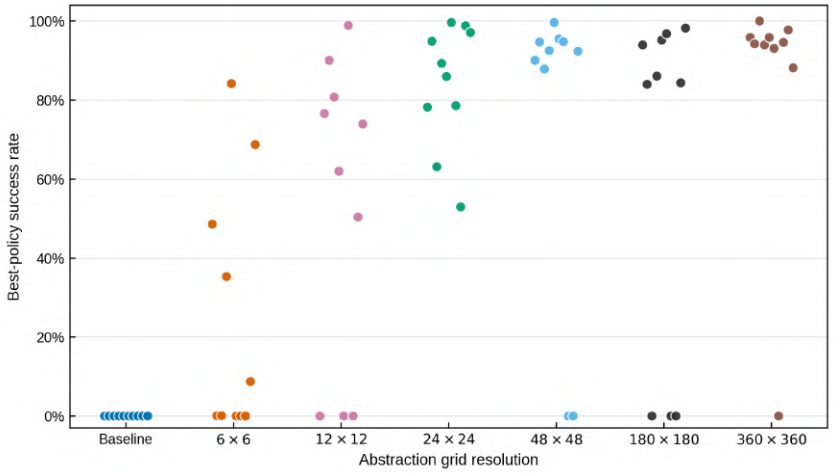


Figure 4.29. 4-phase trajectory. Distribution of post-training best-policy success rates for the baseline and the six abstraction grid resolutions. Each point represents one training seed, with each policy evaluated over 1000 episodes.

Figure 4.29 confirms that the observed high variance is strongly influenced by the presence of unsuccessful runs. It is also evident that increasing the grid resolution generally improves performance. In fact, for finer abstractions, successful policies are concentrated close to the upper part of the success rate range (excluding the failed runs). However, the lower performance obtained with a coarser abstraction grid cannot be explained only by the existence of failed runs. The plot shows that the policies associated with these grids span a wide range of success rates, suggesting

that their guidance is less informative and less consistent in fulfilling the more demanding task.
At this point, it is interesting to study the differences between successful and failed runs.

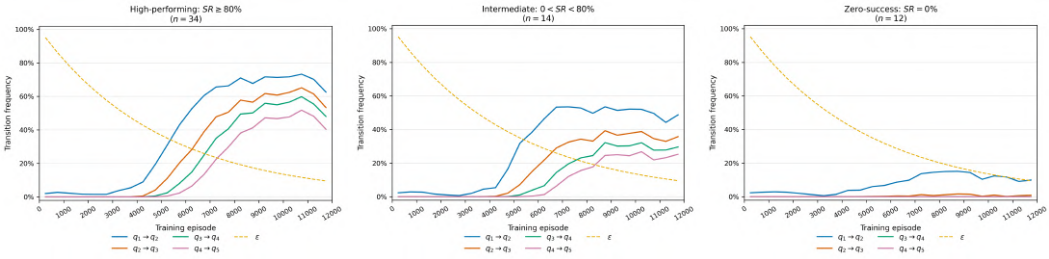


Figure 4.30. 4-phase trajectory. DFA-transition frequencies during training, grouped by the post-training success rate of the best policy: high-performing ($SR \geq 80\%$), intermediate ($0 < SR < 80\%$), and zero-success ($SR = 0\%$) runs. Each curve reports the fraction of episodes containing the corresponding transition, computed over non-overlapping windows of 500 episodes and averaged across the runs in each group.

From the plots of Figure 4.30 we notice a clear distinction among the three groups. The successful policies are characterized by a progressive increase in all the relevant DFA transitions. This indicates that successful policies gradually learn to complete increasingly longer prefixes of the required trajectory until the whole temporal task is reliably satisfied. This behavior is not unexpected because the task is sequential and each episode starts from the initial DFA state.
Intermediate policies show a similar but weaker pattern.
On the other hand, the failed runs exhibit a completely different behavior. In this case, only the first transition appears with a limited frequency, whereas the subsequent ones remain almost absent throughout training.

The abstraction itself does not appear to be the main explanation for failed runs. In fact, we have obtained counterexamples in which the same abstraction configuration is able to produce successful policies (we recall that without abstraction this task appears too hard to be solved).
A possible explanation is strictly related to exploration during the learning phase. In a purely sequential task, the agent always starts from the initial phase and can discover a subsequent DFA transition only after successfully completing all the preceding ones. Therefore, if a given phase of the required trajectory is not discovered sufficiently often during training, all the subsequent phases become unreachable.
This effect is particularly important when exploration decreases over time. It can be noticed also from the first plot of Figure 4.30. By the time the agent has consolidated the first phase of the task (between 6000 and 7000 episodes), the exploration rate has already decreased to approximately 30%. Therefore, only a limited amount of exploratory behavior is still available to discover and reinforce all the subsequent phases of the required trajectory.

Therefore, these considerations suggest that the interaction between the sequential task and exploration dynamics is a possible explanation for the unsuccessful runs.

However, this interpretation is still a hypothesis and requires further investigation.

4.4.4 5-Phase Trajectory

This task corresponds to a trajectory composed of five phases: reaching the first waypoint, then the second, third and fourth waypoints in sequence, and finally the target region. The LTL_f formula is:

$$\varphi = \Diamond(\text{wp}_1 \wedge \circ(\Diamond(\text{wp}_2 \wedge \circ(\Diamond(\text{wp}_3 \wedge \circ(\Diamond(\text{wp}_4 \wedge \circ(\Diamond g_1)))))))$$

where wp_1, wp_2, wp_3 and wp_4 are the waypoint predicates, and g_1 is the final target predicate.

The corresponding automaton is depicted in the following figure.

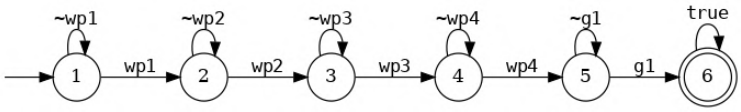


Figure 4.31. DFA for the 5-phase task.

The required trajectory is shown below.

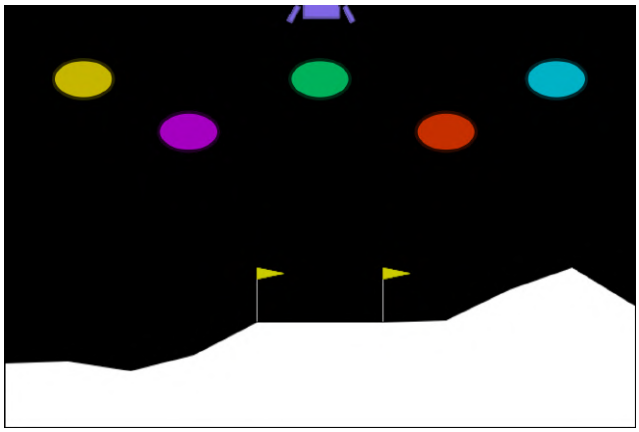


Figure 4.32. 5-phase task: yellow → purple → green → red → blue.

Hyperparameters

The hyperparameters adopted for these experiments are the same as those reported in Section 4.4.3.

Results and Interpretations

As in the previous experiment, we start by analyzing the learning curves and evaluation performance. Then, we move on to investigating the individual runs in order to better interpret the effects of high variance and unsuccessful seeds.

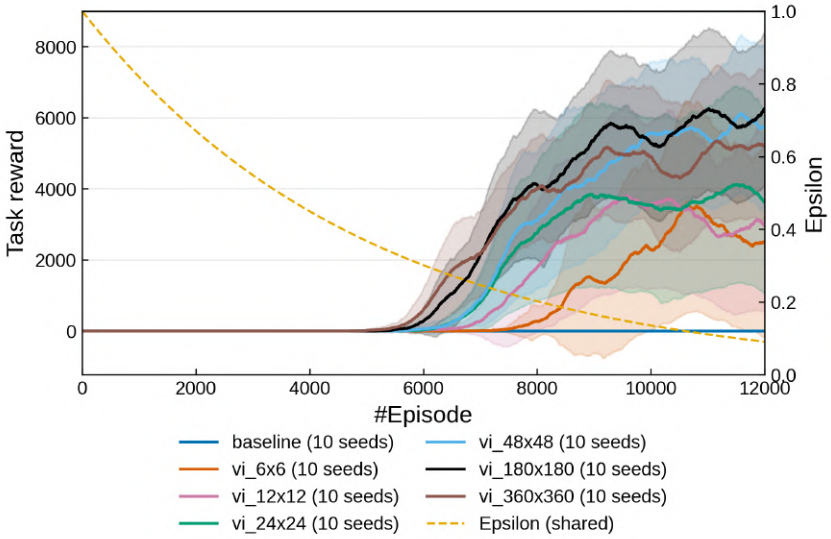


Figure 4.33. 5-phase trajectory. Training task reward for the baseline and the six abstraction grid resolutions, using ten seeds per configuration.

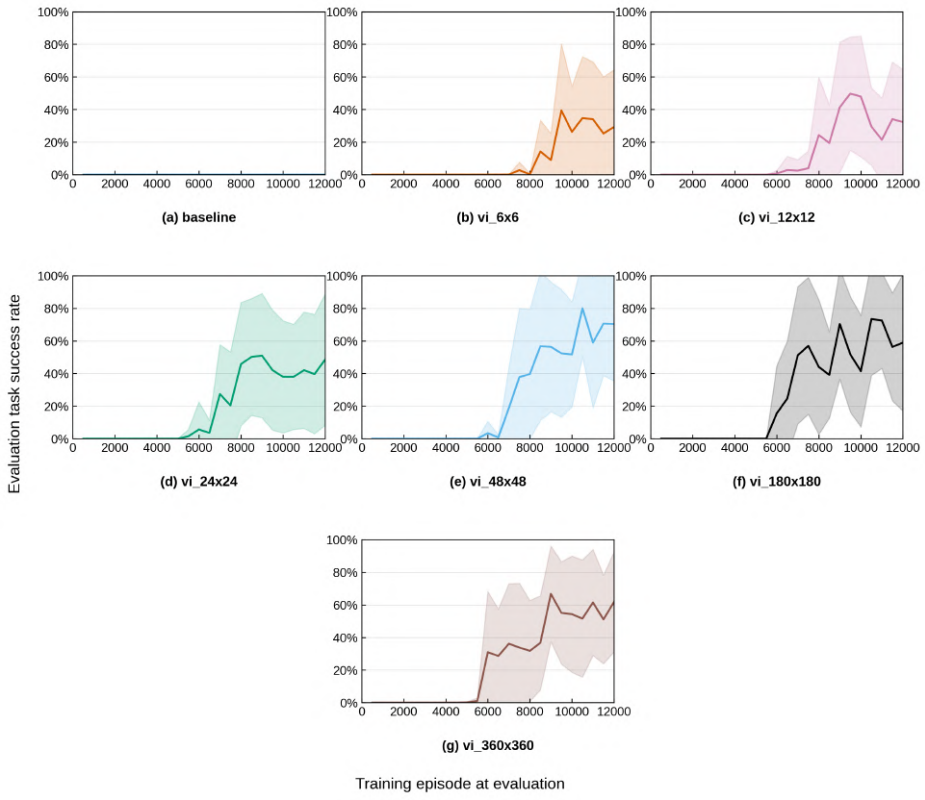


Figure 4.34. 5-phase trajectory. Periodic evaluation success rate during training for the baseline and the six abstraction grid resolutions.

Abstraction	Best policy success rate	Last policy success rate
Baseline	$0.0 \pm 0.0\,\%$	$0.0 \pm 0.0\,\%$
6×6	$52.3 \pm 43.6\,\%$	$29.2 \pm 35.1\,\%$
12×12	$62.3 \pm 42.3\,\%$	$32.3 \pm 32.2\,\%$
24×24	$65.2 \pm 43.2\,\%$	$48.5 \pm 40.2\,\%$
48×48	$87.3 \pm 29.4\,\%$	$70.4 \pm 34.9\,\%$
180×180	$88.5 \pm 29.5\,\%$	$58.9 \pm 41.9\,\%$
360×360	$85.6 \pm 28.7\,\%$	$61.8 \pm 30.7\,\%$

Table 4.13. Five-phase trajectory. Post-training evaluation results for the best and last policies over ten training seeds and 1000 evaluation episodes per policy.

From these results, the same initial considerations done for the previous task are still valid. However, the distinction between the policies obtained with a coarse or a fine abstraction appears to be more evident. In particular, the 48×48, 180×180, and 360×360 configurations achieve higher best-policy success rates, while the coarser grids remain characterized by lower performance and large variability. Moreover, unexpectedly, the additional temporal complexity for this task does not result in a degradation if we compare the performance obtained with the previous one.

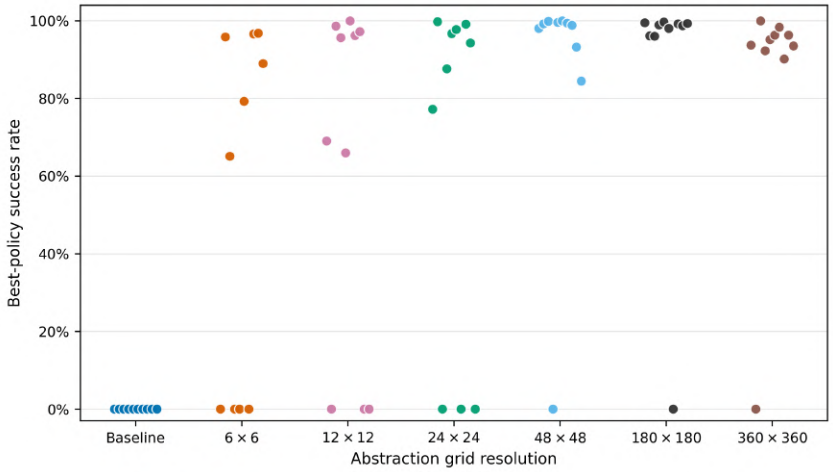


Figure 4.35. 5-phase trajectory. Distribution of post-training best-policy success rates for the baseline and the six abstraction grid resolutions. Each point represents one training seed, with each policy evaluated over 1000 episodes.

Once again, the analysis of single runs provides confirmation that the high variance is influenced by failed runs. We notice also the same trend described for the previous task, in which finer abstraction grids seem to be more reliable rather than the coarse ones.

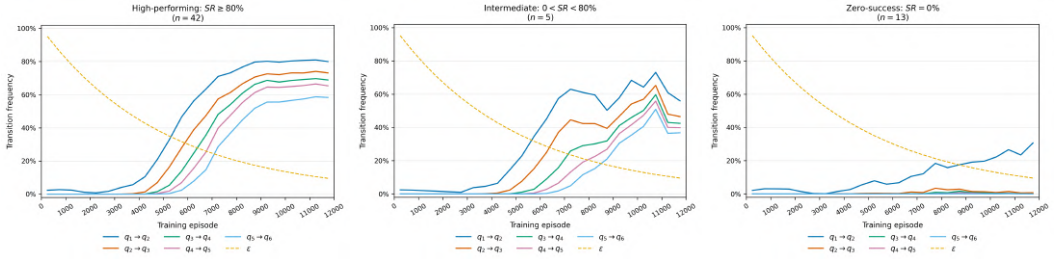


Figure 4.36. 5-phase trajectory. DFA-transition frequencies during training, grouped by the post-training success rate of the best policy: high-performing ($SR \geq 80\%$), intermediate ($0 < SR < 80\%$), and zero-success ($SR = 0\%$) runs. Each curve reports the fraction of episodes containing the corresponding transition, computed over non-overlapping windows of 500 episodes and averaged across the runs in each group.

The DFA transition analysis also exhibits the same pattern: in successful runs, policies are built by sequentially discovering the phases of the required task. Moreover, we notice some delay in the consolidation of the first phase, which now starts between 8000 and 9000 episodes. Despite the fact that the first phase required by this task is the same as in the previous tasks, this delay suggests that the consolidation of the first phase may not be completely independent of the subsequent task structure.

4.4.5 Discussion

Overall, the experiments shown in this section exhibit that the framework is able to support the learning of increasingly complex episodic non-Markovian tasks in the *Lunar Lander* domain. The abstraction-based shaping signal provides a substantial advantage over the baseline without guidance, which is not able to solve the required trajectories.

At the same time, the results highlight that increasing temporal complexity mainly affects the reliability of the learning process rather than imposing a limitation on the quality of the policies that can be obtained. In particular, for more demanding tasks, different training seeds may lead to very different results, ranging from highly successful policies to complete failures. The analysis of single runs and DFA progressions suggests that this behavior could be related to the sequential nature of the task considered, but also to the exploration strategy adopted during the learning phase. In the more complex settings, the full trajectory is not always discovered during exploration, since reaching a later phase requires the agent to have already discovered and sufficiently consolidated all the preceding ones.

Nevertheless, the results show that the abstraction built over the ground MDP was crucial to solve the proposed tasks. Starting with the simpler task, we notice that even coarse abstraction grids were sufficient to obtain a useful policy that fulfills the required objective. The relationship between grid precision and task feasibility is not monotonic. However, as the task becomes more demanding, finer abstractions lead to more reliable results. Increasing the precision of the abstraction also increases the complexity of the abstract decision problem. In this context, this issue was partially hidden by the fact that the abstract grids are solved through

Value Iteration, exploiting a fully-known MDP. This assumption is not general and may not be feasible when richer or higher-dimensional abstractions are considered, especially if we are trying to adopt this framework in a complex real-world scenario.

4.5 RQ3: Learning and Propagation of Abstract Guidance

The experiments presented in the previous sections exhibit that the abstraction-based guidance is useful for the learning on the ground MDP when non-Markovian tasks are required. However, all the abstract grids considered so far were solved through Value Iteration. This choice was done assuming that:

- the abstract MDP is fully-known;
- computing directly the optimal V-function is feasible.

These assumptions may become too restrictive when more complex abstractions are needed (especially when a multi-level abstraction stack is needed). In these scenarios, we are forced to adopt a learning algorithm to obtain a policy and then the associated V-function. This section aims to investigate the consequences of using a learning algorithm in the abstraction hierarchy. A secondary effect of learning at multiple levels is also the additional introduction of new hyperparameters to be configured in such a way in all levels we can obtain an useful policy.

4.5.1 Experimental Design

To investigate the behavior of the framework when learning is introduced at an abstract level, the experimental analysis is organized around three sub-questions:

- Can a large abstract MDP be effectively learned under sparse rewards, and can guidance from a higher abstraction facilitate this process?
- How does the V-function extracted from a learned abstract policy differ from the exact V-function obtained through planning?
- How does the choice of the abstract value representation affect learning on the ground MDP?

Throughout this section, we fix the *2-phase* trajectory as the reference task. This choice limits the variability introduced by different temporal complexities. Furthermore, we consider the 360×360 grid as the reference abstract level to be learned. This corresponds to the largest abstraction adopted in the previous experiments and the computational cost of executing Value Iteration over it is non-negligible. Moreover, we know in advance that this grid is also suitable to support more complex tasks.

Introducing learning over abstractions also requires the definition of some aspects related to the interaction in the environment. In particular, at this moment, it is not defined how to start an episode during learning on the grid. We have two main possibilities related to this aspect:

- the initial abstract state $s_0 = (x, y, q)$ is randomly sampled. This choice differs from the ground-level training setup, where the initial position of the agent is determined by the environment in a specific spatial region of the environment, and the initial q is always the initial state of the DFA. The purpose of this is to encourage the agent to explore a broader portion of the augmented abstract state space to obtain better guidance. When this option is adopted the temporal transition structure itself is unchanged; only the distribution of initial augmented states used during abstract-level training is modified. We refer to it as uniform choice of the initial space;

- the initial abstract state $s_0 = (x, y, q)$ is sampled adopting the same distribution of the ground environment. In particular, at each episode, we generate an initial ground state and we map it over the grid. Finally, we augment the spatial state representation always using the initial state of the task DFA. Adopting this choice is useful when we want to emulate precisely the task required over the grid environment. We refer to it as Gym-reset choice of the initial state (because we exploit the *reset* method which is always present in a *Gymnasium* environment implementation);

4.5.2 Learning on Abstract Levels

Although the underlying dynamics of the grid abstractions considered in this thesis are considerably simpler, in this section we want to investigate whether an optimal policy can be learned using Q-learning under the usual sparse-reward setting. In particular we test the learning over the 360×360 grid in two different configurations:

- standalone configuration, so a pure learning task over the grid;

- with the shaping guidance provided by a 90×90 grid solved with Value Iteration.

The objective of this comparison is twofold. First, it allows us to assess whether the learning task on the precise grid is feasible within a fixed episode budget. Second, it allows us to evaluate (not exhaustively) whether the same abstraction-guidance can also facilitate learning between two abstract levels.
Furthermore, with these experiments, we can test if the coupled biased-unbiased architecture learner works at least on the abstract environment (confirming the results already presented in [4]). The experiments are performed adopting all the variants of starting episode mechanisms described in the previous section.

Hyperparameters

The hyperparameters for these experiments are reported in the following table.

Parameter	Value
Episodes	1000000
Max. steps per episode	600
Learning rate α	0.1
Discount factor γ	0.99
ϵ decay	0.999995
ϵ minimum	0.05
Seeds	5
Abstract Algorithm	Tabular Q-learning
Eval. interval	10000
Eval. episodes during learning	1000
Eval. episodes after learning	1000
Shaping discount γ_s	1.0

Table 4.14. Hyperparameters adopted in the experiments.

Results and Interpretations

We evaluate the experimental outcomes considering the periodic evaluation performed on the unbiased greedy policy during the learning phase. More precisely, we adopt two evaluation protocols. In both cases, the DFA component of the initial abstract state is initialized to its initial state q_1. In the first protocol, the spatial component (x, y) is sampled uniformly on the abstract grid, while in the second case it is sampled as described in the Gym-reset variant. The adoption of these two different protocols allows us to evaluate the policy's ability to complete the task from an arbitrary grid position or under the initial-state distribution of the ground MDP.

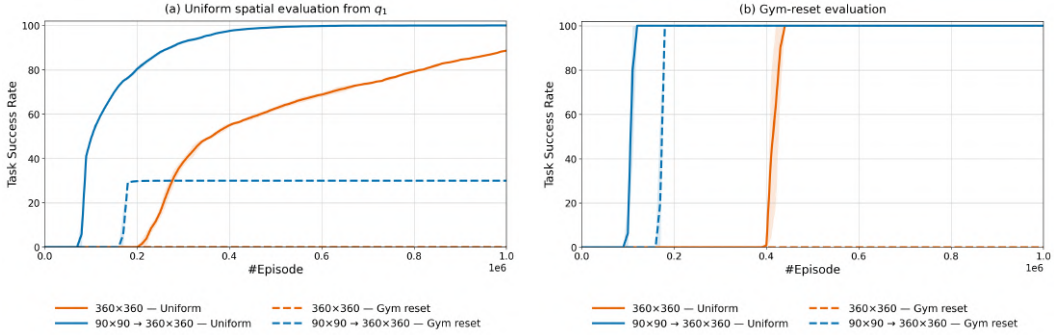


Figure 4.37. 2-phase trajectory. Periodic evaluation success rate of the unbiased greedy policy during Q-learning on the 360×360 abstract grid. The left panel uses initial positions sampled uniformly over the grid, whereas the right panel uses the Gym-reset strategy. Orange curves represent standalone learning, while blue curves represent learning guided by a 90×90 abstraction solved through Value Iteration. Solid curves correspond to training with uniformly sampled augmented initial states (x, y, q); dashed curves correspond to Gym-reset training. Curves report mean success rates across five training seeds, with evaluations performed every 10000 training episodes over 1000 episodes per policy.

Episode init.	Abstract learning	Uniform evaluation		Gym-reset evaluation	
		Final	AUC	Final	AUC
Uniform	360×360	88.62 ± 0.22	50.65 ± 0.80	100.00 ± 0.00	58.46 ± 0.99
Uniform	$90 \times 90 \rightarrow 360 \times 360$	99.98 ± 0.04	85.52 ± 0.24	100.00 ± 0.00	89.36 ± 0.50
Gym reset	360×360	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Gym reset	$90 \times 90 \rightarrow 360 \times 360$	29.94 ± 0.15	24.74 ± 0.17	100.00 ± 0.00	82.70 ± 0.45

Table 4.15. Final task success rate and normalized area under the learning curve (AUC), reported as mean ± standard deviation over five seeds. All values are percentages.

The results show that learning on the 360×360 abstract grid without shaping guidance is feasible when abstract episodes are initialized uniformly over the spatial domain. Nevertheless, if we add an abstraction level solved through Value Iteration over it we obtain a clear advantage. Under uniform training initialization, the shaped configuration reaches approximately 100% success using both evaluation protocols and exhibits a substantially larger AUC than standalone Q-learning. Thus, shaping improves both the final policy and the overall learning efficiency.

The initial-state distribution used during training also has a significant effect. As expected, policies trained with the uniform spatial initialization seem to cover a larger portion of the abstract state space. This helps exploration and reward propagation along different phases of the task and produces policies that perform well. On the other hand, when training starts from states generated by the LunarLander reset procedure (Gym-reset variant), the standalone Q-learning fails to discover a useful policy within the available episode budget.

Another important indication can be extracted from the outcome of $90 \times 90 \rightarrow 360 \times 360$ configuration trained with the Gym-reset variant. Considering the evaluation success rate with the uniform protocol, which is approximately 30%, we deduce that the learned policy is specialized in the subset of the grid reachable from the environment's initial-state distribution and probably it does not provide uniform coverage of the entire abstract domain.

A final observation concerns the biased–unbiased learning architecture. While the experiments in Section 4.3.1 showed that the unbiased learner fails to extract a policy, the same coupled mechanism works correctly on the abstract grid. This indicates that the architecture is not completely ineffective, but that its practical behavior depends on the features of the underlying problem.

4.5.3 Extraction and Analysis of the Guidance Signal

The analysis presented in the previous section exhibits that learning on the abstract level is feasible in the considered setting. In particular, we know that Q-learning successfully learns a policy on the 360×360 grid, while the additional guidance provided by the 90×90 abstraction improves the learning process.
However, learning a policy that fulfills the abstract task does not theoretically

guarantee that the corresponding value estimates constitute an effective potential function for ground-level shaping. In particular, they are not guaranteed to reproduce the V-function obtained through Value Iteration, which represents the optimal solution of the abstract MDP under the adopted model. In this section we address the second question presented in Section 4.5.1. More precisely, to answer this question, we analyze different methods for deriving a V-function from the learned unbiased Q-function and compare their outputs with the V-function computed through Value Iteration on the same 360×360 abstract grid. In particular, we consider the direct maximum over action values and the value obtained by applying Policy Evaluation on the learned unbiased Q-function, as described in Section 3.4.1. The starting Q-function is the one obtained from $90 \times 90 \rightarrow 360 \times 360$ abstract training with uniform spatial initialization. We select this configuration because it achieves high task success while providing broad coverage of the abstract grid, allowing the extraction methods to be compared without the additional confounding effect of a restricted training-state distribution.
This comparison allows us to distinguish between inaccuracies in the learned action-value estimates and the actual suboptimality of the policy encoded by them.

Analytical Evaluation Metrics

The analytical evaluation metrics considered to perform the comparison are:

- *Normalized Mean Absolute Error (nMAE):*

$$\mathrm{nMAE}(\hat{V}, V_{\mathrm{VI}}) = \frac{1}{R_{\mathrm{goal}}|\mathcal{S}'|} \sum_{s \in \mathcal{S}'} \left| \hat{V}(s) - V_{\mathrm{VI}}(s) \right|$$

  It measures the average absolute difference from the reference V-function, normalized by the abstract goal reward;

- *Normalized Root Mean Squared Error (nRMSE):*

$$\mathrm{nRMSE}(\hat{V}, V_{\mathrm{VI}}) = \frac{1}{R_{\mathrm{goal}}} \sqrt{\frac{1}{|\mathcal{S}'|} \sum_{s \in \mathcal{S}'} \left(\hat{V}(s) - V_{\mathrm{VI}}(s) \right)^2}$$

  It measures the magnitude of the numerical error while assigning greater importance to states with large discrepancies;

- *Spearman's rank correlation coefficient:*

$$\rho_{\mathrm{S}} = \mathrm{corr}\left(\mathrm{rank}(\hat{V}), \mathrm{rank}(V_{\mathrm{VI}}) \right)$$

  It evaluates whether the extracted value function preserves the ordering of states induced by the reference function;

- *Greedy-policy agreement:*

$$\mathrm{Agreement} = \frac{1}{|\mathcal{S}'|} \sum_{s \in \mathcal{S}'} \mathbb{I}\left[\pi_Q(s) \in \mathcal{A}^*_{\mathrm{VI}}(s) \right]$$

where

$$\pi_Q(s) = \arg \max_{a \in \mathcal{A}(s)} Q_{\text{unbiased}}(s, a)$$

and $\mathcal{A}^*_{\text{VI}}(s)$ is the set of optimal actions according to Value Iteration. It measures how frequently the policy learned through Q-learning selects an action that is also optimal under the planning solution.

Results and Interpretations

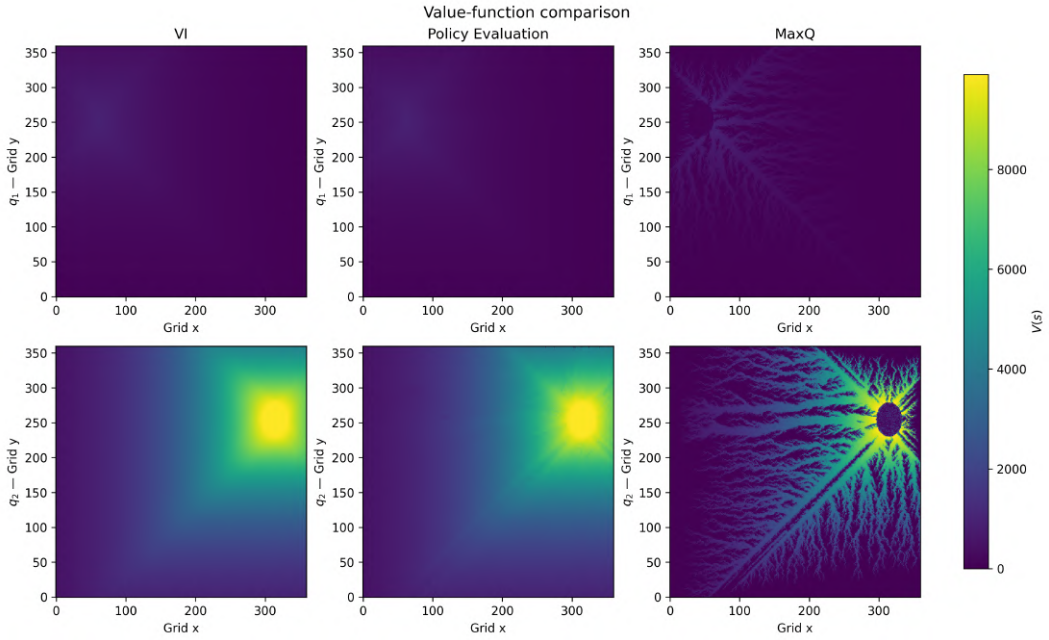


Figure 4.38. 2-phase trajectory. Comparison of value functions on the 360×360 abstract grid. Columns show, from left to right, the Value Iteration reference, the V-function obtained through Policy Evaluation of the learned unbiased greedy policy, and the direct estimate $\max_a Q(s, a)$. The top and bottom rows correspond to DFA states q_1 and q_2, respectively.

Extraction	nMAE	nRMSE	Spearman	Policy agreement
Policy Evaluation	0.41 ± 0.03	0.94 ± 0.04	0.999 ± 0.000	$91.06 \pm 0.10\,\%$
$\max_a Q(s, a)$	8.22 ± 0.08	16.83 ± 0.13	0.564 ± 0.003	$91.06 \pm 0.10\,\%$

Table 4.16. Comparison with the 360×360 value-iteration reference, reported as mean $\pm$ standard deviation over five seeds. Errors are normalized by the goal reward.

The results highlight a distinction between learning a successful abstract policy and obtaining an accurate V-function. Although the learned greedy policy agrees with an optimal action identified through Value Iteration in 91.06% of the non-accepting states, the methods produce substantially different results.

Visually, it is immediate to observe that the heatmaps associated with the MaxQ extraction method show that informative values are mainly concentrated along task-

relevant paths, while other regions remain characterized by sparsity and irregularity. If we also consider the analytical metrics, they highlight a significant difference in the numerical values. The combination of both considerations suggests that the Q-learner has learned an effective policy, but did not reproduce calibrated action values when compared with the optimal V-function.

On the other hand, when the Policy Evaluation extraction method is applied, the analytical metrics are significantly better. Therefore, we deduce that most of the observed MaxQ discrepancies appear to originate from inaccurate Q-value estimates. By propagating the returns of the learned policy over the complete abstract model, Policy Evaluation recovers a V-function that closely approximates the optimal reference.

The improvement in accuracy achieved by adopting the Policy Evaluation method is not free, in terms of computational cost. Instead, MaxQ can be computed directly from the Q-function, also online during the learning process. This aspect is crucial because it highlights a significant trade-off between computational cost and achieved accuracy in the extracted V-function.

4.5.4 Impact on Ground-Level Learning

The previous experiments exhibit that there are differences between the V-functions obtained by Policy Evaluation and the MaxQ method. Although the final policy substantially agrees everywhere with the reference provided by the Value Iteration algorithm, we have no guarantee that the ones under study imply equivalent usefulness when adopted for the shaping-guidance during the training phase on the ground MDP.
Hence, we test this aspect investigating whether notably differences, in terms of performance, appears after the training on the ground environment. More precisely, we adopt these configurations:

- learning with shaping guidance extracted through Policy Evaluation;
- learning with shaping guidance extracted through MaxQ method;
- learning with shaping guidance provided by the optimal V-function. These results are already available from Section 4.4.1.

Hyperparameters

The hyperparameters adopted for these experiments are the same as those reported in Section 4.4.1.

Results and Interpretations

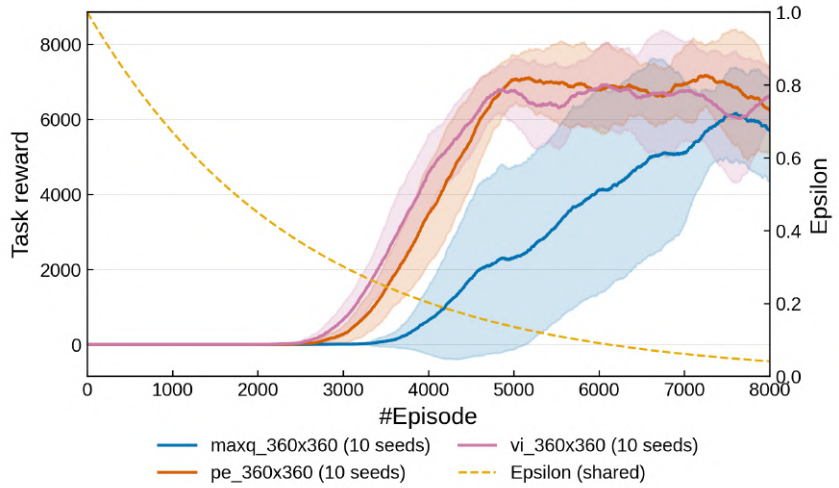


Figure 4.39. 2-phase trajectory. Training task reward for the MaxQ, Policy Evaluation, and Value Iteration guidance configurations, using ten seeds per configuration.

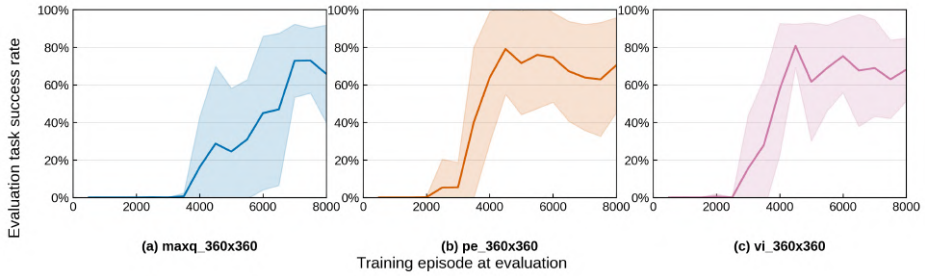


Figure 4.40. 2-phase trajectory. Periodic evaluation success rate during training for the MaxQ, Policy Evaluation, and Value Iteration guidance configurations.

Method	Best policy success rate	Last policy success rate
VI	$97.01 \pm 2.25\,\%$	$69.38 \pm 17.28\,\%$
PE	$96.40 \pm 3.98\,\%$	$70.51 \pm 25.11\,\%$
MaxQ	$90.41 \pm 8.93\,\%$	$65.66 \pm 25.99\,\%$

Table 4.17. Post-learning evaluation for Trajectory 2. Success rates of the best and last policies are reported as mean ± standard deviation across 10 seeds, with each policy evaluated over 1000 episodes.

The outcomes of these experiments confirm the suspects emerged from the previous section analysis. Learning curves exhibit that Policy Evaluation and Value Iteration guidance achieve comparable performance also in terms of sample efficiency.
The MaxQ method also enables the agent to learn the task, however, its large variability and the delay experienced in terms of training convergence suggest that the guidance provided is not effective as the other methods. Nevertheless, the

evaluation rates of MaxQ are acceptable. This means that the guidance provided with the latter method is still useful, especially if we recall that the baseline (without shaping) poorly fails in learning the task. Probably, it would be sufficient to augment the training episode budget to reach the plateau.

4.5.5 Discussion

The results shown suggest that abstract-level learning is a valid alternative when planning over an abstract MDP becomes impractical. However, learning a successful abstract policy is not, by itself, sufficient to obtain an effective shaping potential. Even when the task is synchronized across levels, the quality of the guidance depends on how the learned policy is converted into a V-function. The `computeValue` step in Algorithm 1 is therefore an important design choice, rather than a merely mechanical consequence of abstract learning. Furthermore, this choice must be made at each abstract level and it could not be uniform across the whole hierarchy considered.

The available extraction methods involve a trade-off between computational cost and V-function quality. MaxQ requires a single pass over the state-action values, with cost $\mathcal{O}(|\mathcal{S}||\mathcal{A}|)$, and does not require an abstract transition model. Policy Evaluation can produce a more coherent V-function, but requires access to that model and repeated sweeps over the state space. In our settings, its cost is $\mathcal{O}(|\mathcal{S}||\mathcal{A}| + K|\mathcal{S}|)$, where K is the number of sweeps required to reach the chosen tolerance. Consequently, the risk is that its use may become costly or unavailable for more complex abstraction models.

4.6 RQ4: Effectiveness on Cyclic Temporal Tasks

The experimental analysis carried out so far has focused on episodic temporal tasks, expressed through the LTL_f language. Now, we move on to investigating whether the same abstraction-based guidance can also support cyclic temporal behaviors. In these tasks, the completion of one sequence does not terminate it, but instead starts a new repetition of the same temporal pattern.

4.6.1 Experimental Design

To evaluate this new setting, we consider four different cyclic tasks: vertical, horizontal, triangular and rectangular. In all cases, the agent is required to visit a sequence of spatial regions according to a predefined order.

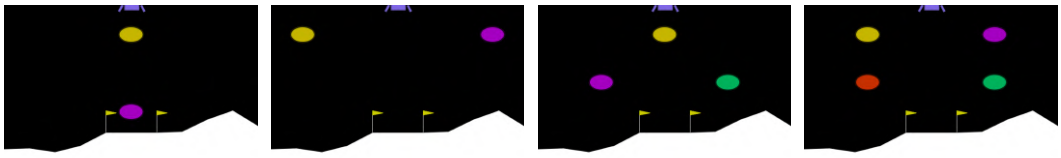


Figure 4.41. Cyclic temporal tasks considered in RQ4. From left to right: vertical, horizontal, triangular, and rectangular. In each task, the spatial regions must be visited following the colour sequence yellow → purple → green → red, when the red region is present.

This section does not aim to provide another extensive comparison of abstract grid resolutions. The effect produced by different grids has already been investigated in Section 4.4. Therefore, for all cyclic tasks we fix a single abstraction grid that provides sufficiently informative guidance. In particular, we have chosen this grid adopting the following criteria: the coarsest grid useful to fulfill the task.

Another important assumption is related to the maximum number of cycle repetitions allowed within each episode. We fix this limit to three completed cycles:

- the first one verifies whether the agent is capable of performing the task at least once within the episode, starting from a state s_0 sampled from the initial-state distribution (controlled by the environment itself);
- the second one requires the policy to continue correctly after terminating the previous iteration. Therefore, the agent is required to perform another iteration starting from the final state of the previous one;
- the third and last one is an additional confirmation that the learner behavior can be repeated consistently over time.

Consequently, the adopted evaluation protocol accounts for this choice, allowing us to assess not only whether the cyclic sequence can be discovered, but also whether the learned policy effectively supports the required cyclic task.

Hyperparameters

The common hyperparameters adopted for the cyclic-task experiments are reported in the following table.

Parameter	Value
Episodes	8000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	10
Grid dimension	12x12
Abstract Algorithm	Value Iteration
Maximum completed cycles	3
Eval. interval	500
Eval. episodes during learning	1000
Eval. episodes after learning	1000

Table 4.18. Hyperparameters adopted in the cyclic-task experiments.

4.6.2 Results

We evaluate the experimental results considering the task-reward evolution during training, in order to analyze how the cyclic behaviors are progressively learned. Then we focus on the post-training performance of both the best and last policies, giving also a more detailed analysis about the distribution of cycles completed in each episode.

Training performance

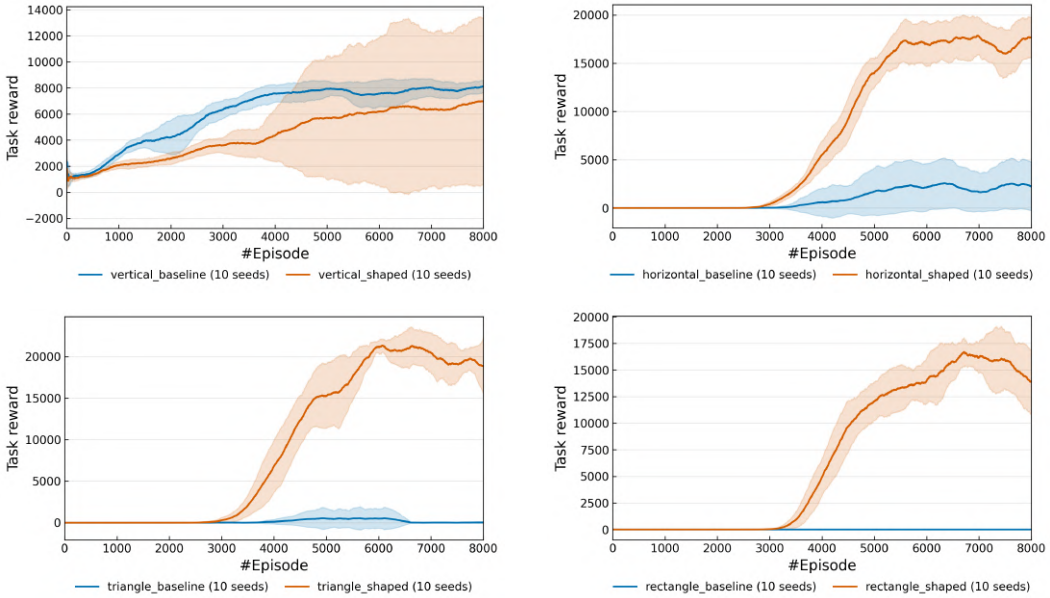


Figure 4.42. Task-reward evolution during training for the baseline and shaped configurations on the four cyclic tasks. From top-left to bottom-right: vertical, horizontal, triangular, and rectangular cyclic tasks.

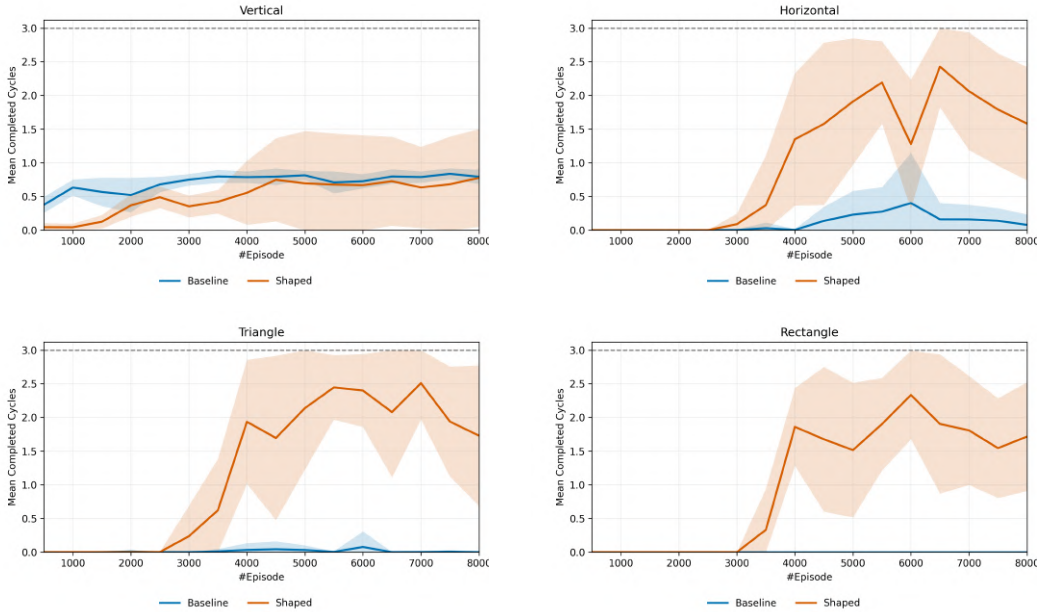


Figure 4.43. Mean number of completed cycles during the periodic greedy-policy evaluations for the baseline and shaped configurations. From top-left to bottom-right: vertical, horizontal, triangular, and rectangular cyclic tasks.

Post-training performance

Task	Policy	P(C = 3)		Mean completed cycles	
		Baseline	Shaped	Baseline	Shaped
Horizontal	Best	8.04 ± 24.12 %	92.16 ±10.95 %	0.564 ± 0.766	2.892 ±0.153
Horizontal	Last	0.00 ± 0.00 %	31.99 ± 29.34 %	0.081 ± 0.158	1.600 ± 0.842
Vertical	Best	0.01 ± 0.03 %	10.81 ±29.19 %	0.910 ± 0.036	0.966 ±0.675
Vertical	Last	0.00 ± 0.00 %	10.31 ± 27.70 %	0.797 ± 0.098	0.782 ± 0.725
Triangle	Best	0.00 ± 0.00 %	96.94 ±2.91 %	0.096 ± 0.227	2.955 ±0.036
Triangle	Last	0.00 ± 0.00 %	45.86 ± 39.87 %	0.000 ± 0.000	1.743 ± 1.043
Rectangle	Best	0.00 ± 0.00 %	92.72 ±7.90 %	0.000 ± 0.000	2.880 ±0.144
Rectangle	Last	0.00 ± 0.00 %	33.50 ± 28.84 %	0.000 ± 0.000	1.727 ± 0.794

Table 4.19. Post-training performance of the best and last policies, aggregated across ten training seeds. Each policy is evaluated over 1000 episodes. Values report mean ± standard deviation across training seeds, and C denotes the number of completed cycles in an episode.

Cycle completion distribution

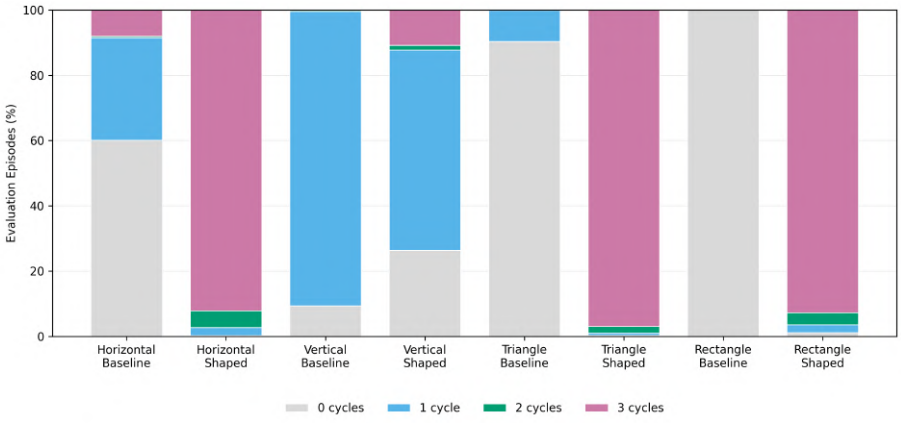


Figure 4.44. Distribution of the number of cycles completed by the post-training best policies. Each bar aggregates the evaluation outcomes of ten independently trained policies, with each policy evaluated over 1000 episodes.

Shaped Performance without limit

Task	Policy	Mean C	P(C ≥ 3)	P(C > 10)	Timeout
Horizontal	Best	12.97 ±3.71	94.08 ±9.07 %	75.64 ±28.15 %	72.48 ±28.69 %
Horizontal	Last	2.95 ± 2.62	36.78 ± 30.97 %	6.34 ± 11.69 %	12.33 ± 21.03 %
Vertical	Best	2.82 ±6.19	10.82 ±29.19 %	9.79 ±29.37 %	9.84 ±29.35 %
Vertical	Last	2.17 ± 4.84	10.32 ± 27.70 %	8.50 ± 25.50 %	8.16 ± 23.92 %
Triangle	Best	14.80 ±1.86	98.57 ±1.26 %	92.59 ±6.35 %	92.05 ±8.44 %
Triangle	Last	6.86 ± 5.69	54.51 ± 38.27 %	37.21 ± 38.66 %	52.34 ± 39.32 %
Rectangle	Best	13.00 ±1.93	95.37 ±6.08 %	83.99 ±13.98 %	84.55 ±10.94 %
Rectangle	Last	4.88 ± 4.35	51.26 ± 34.09 %	15.57 ± 30.59 %	52.03 ± 34.27 %

Table 4.20. Extended post-training evaluation of the shaped policies for the cyclic tasks. Each policy is evaluated over 1000 episodes with a limit of 5000 steps and without imposing a practical limit on the number of cycles. Values report mean ± standard deviation across ten training seeds, and C denotes the number of completed cycles in an episode.

4.6.3 Discussion

The results exhibit that the framework, and in particular the shaping guidance, can be very effective in learning cyclic temporal tasks. More precisely, the outcomes show that for horizontal, triangular and rectangular configurations the abstraction-based shaping guidance allows the agent to learn policies that repeatedly satisfy the temporal sequence. Conversely, the corresponding baseline configurations fail to acquire the same cyclic behavior.

Nevertheless, the distinction between completing the sequence once and learning to repeat it is particularly important. This aspect emerges from the outcomes related to the vertical configuration. We notice that the vertical baseline was able to discover the required sequential pattern, but almost never completed three cycles. This is not completely unexpected because the experiment was designed deliberately. The configuration is particularly easy for the agent because it only requires the agent to fall without acting. Furthermore, in this case study, the shaping guidance was effective in only one out of ten seeds.

The last consideration highlights a possible and known limitation of the abstraction adopted so far. The grid built over the ground MDP lacks physical information, and therefore the agent is only attracted toward the goal region without any support in repeating the pattern. Instead, the abstraction is useful for the other task, where

the task specification forces the agent to control the lander carefully so that it can complete the temporal pattern.

Overall, the experiments provide a positive but task-dependent answer to RQ4. The framework can support cyclic temporal behaviors, and abstraction-based shaping provides a substantial advantage over unguided learning. However, its effectiveness depends on whether the spatial guidance is compatible with the ground-level dynamics required to restart the sequence.

4.7 Additional Case Studies

The experiments presented so far evaluated the framework on temporal tasks mainly described by ordered sequences of spatial regions, possibly including cyclic requirements. In this section we want to extend those results with two additional case studies designed to examine more demanding behaviors required of the agent.

The first one introduces explicit spatial constraints that must remain satisfied while the agent moves between the various regions of the required trajectory. Its purpose is to investigate whether abstraction-based shaping remains effective when task completion depends not only on reaching a sequence of waypoints, but also on following a prescribed spatial route.

The second considers landing-oriented tasks. The latter corresponds to the original *Lunar Lander* task, but in this scenario we are trying to solve it without exploiting the dense reward function provided by the *Gymnasium framework*.
In this setting, reaching the final spatial region is not sufficient to satisfy the specification: the accepting transition is validated only when the episode terminates with a successful landing in the ground environment. These experiments will also be useful to examine the limitations of a position-based abstraction when the final task condition also depends on physical variables that are not represented at the abstract level.

4.7.1 Trajectory with Spatial Constraints

The first additional case study extends the *2-phase trajectory* formulation by introducing predicates associated with entire portions of the state space. More precisely, the agent must first reach the region wp_1, move toward the lower part of the environment while remaining within the left-hand side, and subsequently approach the goal region g_1 from the right-hand side.
The corresponding LTL_f formula is

$$\varphi = \Diamond wp_1 \ \wedge \ \Box\Big(wp_1 \rightarrow \bigcirc(\mathit{left}\ \mathcal{U}\ (\mathit{low} \wedge (\mathit{low}\ \mathcal{U}\ (\mathit{right} \wedge \bigcirc(\mathit{right}\ \mathcal{U}\ g_1))))))\Big)$$

The predicates *left*, *right* and *low* identify half-planes of the spatial domain. Therefore, unlike the previous waypoint trajectories, the DFA also monitors whether the agent remains inside the required areas while progressing toward the final target. The following figure shows the resulting spatial specification.

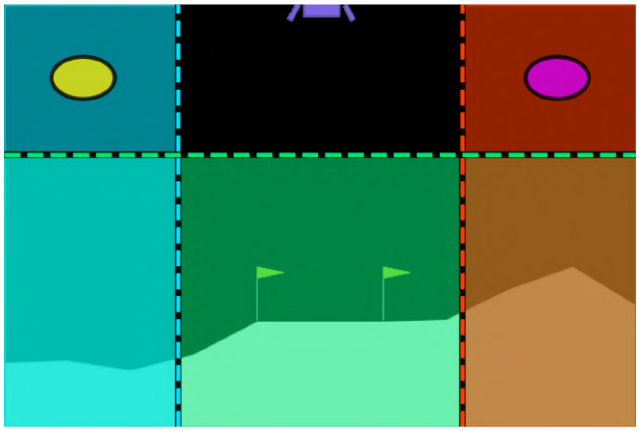


Figure 4.45. 2-phase trajectory with spatial constraints.

Hyperparameters

Parameter	Value
Episodes	8000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	10
Grid dimension	48x48
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes during learning	1000
Eval. episodes after learning	1000

Table 4.21. Hyperparameters adopted in the current experiment.

Results and Interpretations

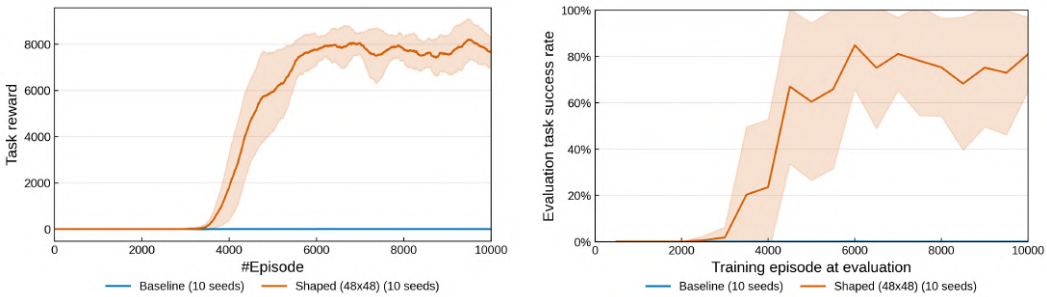


Figure 4.46. 2-phase trajectory with spatial constraints. Training performance for the baseline and the configuration guided by the 48×48 abstraction grid, using ten seeds per configuration. The left panel shows the training task reward, while the right panel reports the periodic evaluation success rate.

Policy	Baseline success rate	Shaped success rate
Best	$0.10 \pm 0.30\,\%$	$98.68 \pm 1.18\,\%$
Last	$0.00 \pm 0.00\,\%$	$80.70 \pm 17.00\,\%$

Table 4.22. Post-training evaluation for the trajectory with spatial constraints. Success rates are reported as mean ± standard deviation across ten training seeds, with each policy evaluated over 1000 episodes. The shaped configuration employs a 48×48 abstraction grid.

The results show that the abstraction-based shaping is highly effective for the task with spatial constraints. The baseline does not acquire any useful policy.
More precisely, the high mean and the limited variability across seeds show that the framework consistently learns to satisfy both the temporal ordering of the target regions and the spatial constraints imposed along the trajectory. Hence, the abstraction-guidance can also represent extended spatial predicates such as half-planes and guide the agent through the corresponding required route.
As always, the performance of the last shaped policies are worse than the best policy ones.

4.7.2 Landing-Oriented Tasks

The second additional case study investigates whether abstraction-based shaping can support tasks whose final objective is not determined exclusively by the spatial position of the agent. More precisely, we consider specifications in which the required behavior is an ordered sequence of regions followed by a successful landing on the ground.
This setting introduces an important distinction between reaching the landing region and actually landing correctly. The positional abstraction can represent an approximation of the landing area, but it does not include any information about physical quantities like lander velocity, orientation, angular velocity or leg contacts. Consequently, in these new scenarios, reaching the final region at the abstract level does not guarantee that the corresponding ground trajectory terminates with a successful landing.

In a sparse-reward setting, a new problem arises: how to define a successful landing operation properly. To represent this aspect, the framework supports an additional ground-level acceptance condition named `successful_landing`. When this option is enabled through the task specification, a transition that would normally move the DFA into its accepting state is validated only if the same ground transition satisfies

$$\mathit{terminated} \wedge \neg \mathit{truncated} \wedge (r_{\text{env}} = 100)$$

where r_{env} denotes the original reward returned by *LunarLander*. In *Lunar Lander* a terminal environment reward equal to 100 identifies a successful landing [6]. If the condition is not satisfied, the transition toward the accepting DFA state is suppressed and the task is not considered completed. It is worth specifying that the environment reward is used exclusively to classify the terminal transition. It is not included in any way in the reward used to train the agent, which remains composed of the sparse synthetic task reward and, in the shaped configuration, the

potential-based shaping contribution. Therefore, the experiments preserve the Goal MDP formulation adopted throughout this thesis.
The additional condition is applied only during interaction with the ground environment. At the abstract level, task completion retains its purely spatial interpretation and corresponds to reaching the landing region (the landing concept exists only in the ground domain). Thus, the abstract V-function guides the lander toward the appropriate spatial area, whereas the ground learner remains responsible for discovering the physical control strategy required to land successfully (only through exploration, without any reward feedback, unlike in the classical *Lunar Lander* setting).

In this section, two landing-oriented tasks are considered. The first one defines a descending trajectory composed of two intermediate regions followed by the landing area. Its LTL$_f$ formula is:

$$\varphi = \Diamond\,(wp_1 \wedge \bigcirc\,(\Diamond\,(g_1 \wedge \bigcirc\,(\Diamond\,\mathit{landing}))))$$

The intermediate regions are positioned so as to progressively direct the lander toward the landing pad, forcing the agent to guide the lander without falling. Instead, the second task extends the original *2-phase trajectory* by introducing two additional approach regions before the landing area. The corresponding formula is:

$$\varphi = \Diamond\Big(wp_1 \wedge \bigcirc\Diamond(g_1 \wedge \bigcirc\Diamond(\mathit{approach}_1 \wedge \bigcirc\Diamond(\mathit{approach}_2 \wedge \bigcirc\Diamond\,\mathit{landing})))\Big)$$

The approach regions are intended to guide the lander toward the pad after completing the original trajectory. For both specifications, entering the landing region is insufficient by itself: the final accepting transition is rewarded only when the ground environment simultaneously reports a successful landing.

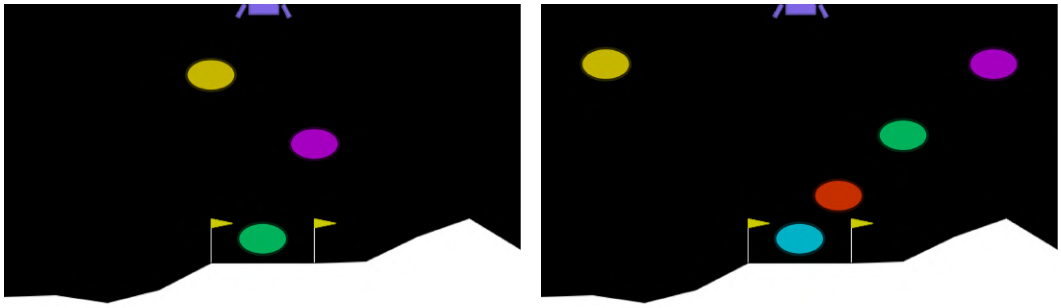


Figure 4.47. Landing-oriented tasks considered in the additional case study. The left panel shows the descending trajectory composed of two intermediate regions followed by the landing area. The right panel shows the 2-phase landing task, obtained by extending the 2-phase trajectory with two additional approach regions.

Hyperparameters

The hyperparameters adopted for the two landing-oriented tasks are reported in the following tables.

Parameter	Value
Episodes	8000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	10
Grid dimension	24×24
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes during learning	1000
Eval. episodes after learning	1000

Table 4.23. Hyperparameters adopted for the landing trajectory.

Parameter	Value
Episodes	8000
ϵ decay	0.9996
ϵ minimum	0.01
Seeds	10
Grid dimension	48×48
Abstract Algorithm	Value Iteration
Eval. interval	500
Eval. episodes during learning	1000
Eval. episodes after learning	1000

Table 4.24. Hyperparameters adopted for the 2-phase landing trajectory.

Results and Interpretations

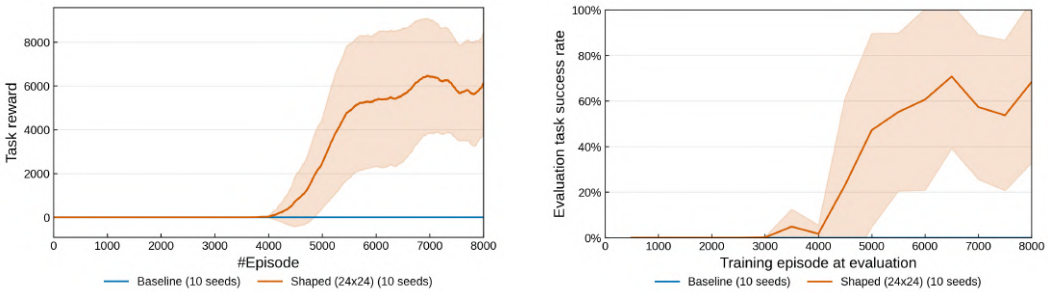


Figure 4.48. Landing trajectory. Training performance for the baseline and shaped configurations, using ten seeds per configuration. The left panel reports the training task reward, while the right panel shows the periodic evaluation success rate.

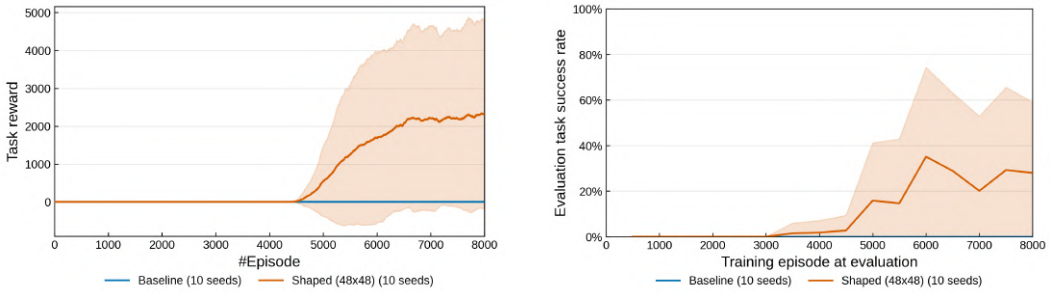


Figure 4.49. 2-phase landing trajectory. Training performance for the baseline and shaped configurations, using ten seeds per configuration. The left panel reports the training task reward, while the right panel shows the periodic evaluation success rate.

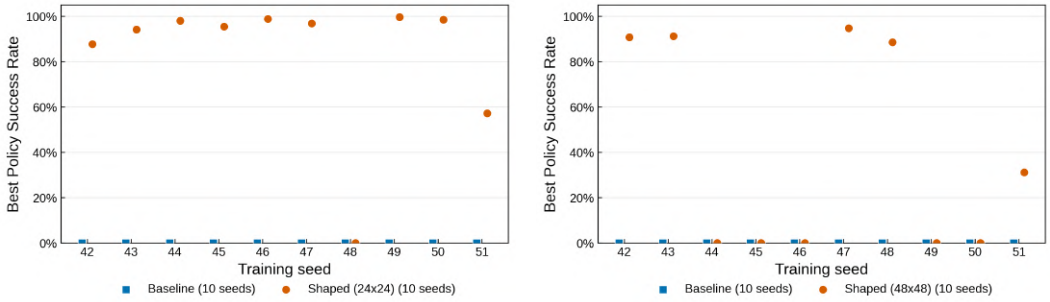


Figure 4.50. Success rate of the post-training best policy for each training seed in the baseline and shaped configurations. The left and right panels correspond to the landing and 2-phase landing trajectories, respectively. Each policy is evaluated over 1000 episodes.

The outcomes of these experiments show that shaping guidance can make the landing task learnable, although the abstraction involves only the spatial position of the agent. On the other hand, neither baseline configuration obtains a successful policy, whereas the shaped configuration solves the landing task in most training seeds. Nevertheless, the two specifications achieve different levels of robustness. In the first landing task, 8/10 seeds obtain best policies with success rates close to or above 90 %, while only one seed fails completely. Conversely, the *2-phase* landing task exhibits a strongly bimodal distribution: some seeds learn highly successful policies, whereas several runs never complete the task.
In some sense, this behavior is not completely unexpected. Approaching regions have been introduced to force the agent to carefully drive the lander toward the landing pad, but this choice does not provide any guarantee about the correct behavior that the agent must adopt to land successfully.

4.7.3 Discussion

Overall, the results show that the framework can support both spatial specifications and task acceptance conditions that depend on the outcome of the ground environment, like the landing one. In both settings, the baselines fail completely to acquire useful policies, confirming the difficulty of discovering these behaviors in a sparse-reward setting.

The trajectory with spatial constraints represents the setting in which the task specification and the abstraction are aligned. The propositions involved depend only on the ground spatial coordinates abstracted by the grid. Consequently, the abstract model can distinguish the states that are relevant to satisfying the constraints and produce an informative and complete guidance toward the goal.
The landing-oriented tasks evaluate a different situation. The abstract guidance can represent the target regions and guide the agent toward the landing area, but successful task completion also depends on velocities, orientation, angular velocity and contacts with the ground. Since these quantities are absent from the abstract state, different ground configurations can receive the same shaping guidance despite requiring substantially different control actions.
Therefore, shaping can increase the probability of reaching useful configurations and observing successful landing transitions, but it cannot completely influence the final obtained behavior.
Furthermore, the comparison between the two landing specifications highlights the lack of information at the abstraction level. In the future, if we want to execute a more complex trajectory that also includes the landing phase, we will need a richer level of abstraction.

Chapter 5

Conclusions and Future Works

5.1 Overall Findings

This thesis has investigated the practical extension of a multi-layer abstraction framework to Reinforcement Learning problems characterized by sparse rewards and temporally extended non-Markovian objectives. More precisely, this work addressed both the representation of such objectives and the practical use of abstract models to generate additional guidance useful within the learning process.
One of the main contributions of the thesis is related to the way non-Markovian tasks are specified and translated into learning guidance. In classical Reward Machine approaches, the designer explicitly specifies the finite-state reward structure associated with the task [16] and this is strictly related to the specific required behavior. Due to this task-dependence, this process can be cumbersome, especially when we consider complex task specifications. Furthermore, it requires substantial design modifications whenever the temporal specification changes. The framework developed in this thesis adopts a pure declarative approach. Episodic temporal requirements are expressed through LTL_f formulas, which describe in a logical way the behavior that must be satisfied rather than specifying how the reward signal should be engineered. The logical specification is then automatically translated into a DFA, whose state is incorporated into the state representation of every abstraction level. This provides a consistent representation of the task's temporal progress throughout the hierarchy.
This mechanism allows the temporal specification to be completely separated from the construction of a dense learning signal. Therefore, the user will be able to define a task through a logical formula and a set of predicates, while the framework automatically derives the finite-state structure needed to monitor its execution and the dense reward function useful to learn it. In this way, changing the required temporal objective depends primarily on updating the logical formula.
The experimental analysis carried out throughout this thesis provides evidence that the automatically generated guidance can be effective in practice. In the considered *Lunar Lander* domain, abstraction-based shaping enabled the learning of temporally extended objectives that were difficult to discover in a complete sparse-reward setting. This result was observed across tasks with different temporal structures, including increasingly long episodic trajectories, cyclic behaviors, and

also specifications involving spatial constraints that had to remain satisfied during task execution. Furthermore, the additional case studies support this observation. In the trajectory with spatial constraints, the framework was able to take advantage of the extended spatial predicates and guide the agent along a prescribed route, while the corresponding baseline failed to acquire a useful policy. Moreover, the landing-oriented experiments considered a more demanding scenario in which reaching the correct spatial region was not sufficient for task completion, since the final accepting transition also depended on a successful landing. Even in this setting, the abstract guidance made the task learnable in several runs, although the results also exposed the limitations of using a purely positional abstraction for behaviors that depend on physical quantities not included at the abstract level.
Another result concerns the use of learning inside the abstraction hierarchy. The experiments showed that guidance from a higher-level abstraction can facilitate the learning of another abstract level. This provides evidence that the hierarchical mechanism can remain useful when exact planning must be replaced by learning at intermediate levels. At the same time, the experiments also highlight that obtaining a successful abstract policy and obtaining appropriate shaping guidance are related but distinct problems. In fact, when abstract levels are learned, the mechanism used to extract and propagate the additional information through the different levels becomes an important design choice of the framework.

5.2 Limitations

The experimental evaluation exposed some limitations that become relevant when the framework is adopted to a concrete learning problem. In particular, some of them concern mechanisms that are intrinsic to the architecture and may arise independently of the particular abstraction adopted, whereas others are related to the experimental choices made in this thesis.
A first important issue is related to the interaction between the biased and unbiased learners. The framework architecture relies on the shaped learner to generate experience used by the unbiased learner, whose objective remains the optimization of the original task reward (without considering the additional shaping signal). More precisely, the experiments showed that a successful biased policy does not provide any guarantee that the same collected experience is sufficient and useful for the unbiased learner to acquire a successful policy. This problem was observed on the ground domain, while the same coupled architecture was effective for learning on the abstract grids. Therefore, the issue should not be considered as an intrinsic failure of the architecture proposed in [4], but rather as a criticism about the distribution and coverage of the experience generated by the behavior policy.
A second limitation concerns the temporal relationship between consecutive levels of the hierarchy. In the considered setting, with discrete grids as abstract levels, several ground transitions could occur while the corresponding abstract state remains unchanged. When we adopt the classical shaping formulation (with $\gamma_s < 1$), these transitions may generate harmful shaping contributions, even in the absence of abstract progress. This aspect shows a temporal misalignment between levels evolving at different temporal resolutions. Although the problem has emerged from

the specific ground–grid relationship considered in this thesis, the underlying issue may not be restricted to this setting. In this thesis, to mitigate this problem, we have set the shaping discount factor $\gamma_s = 1$ to neutralize this side effect in the experiments. Nevertheless, this choice should be considered as a practical adaptation rather than a general solution, such as an alternative heuristic function.
Another important limitation concerns exploration when long temporal specifications are required. The experiments showed that successful policies tend to discover and consolidate the different logical phases progressively within the learning episode budget. In a temporally extended task, especially when the specifications are purely sequential like trajectories, later phases are more difficult to explore, because reaching them requires a successful execution of all the preceding portions of the task. Consequently, the experience collected during the training phase becomes increasingly unbalanced across automaton states as the temporal horizon grows. A single exploration process (like the one adopted with the $\epsilon-$greedy policy) applied uniformly to the whole task may become limiting, even when the shaping-guidance is available.
A further methodological challenge emerges when abstract levels are solved through learning rather than planning. The experiments demonstrate that learning a successful abstract policy is not equivalent to obtaining a reliable shaping signal. An abstract policy may select effective actions (and therefore be equivalent to the optimal one) while its numerical values could remain poorly calibrated over parts of the abstract state space. This thesis presents a comparison between two different techniques of V-function extraction: MaxQ and Policy Evaluation. Policy Evaluation substantially reduced the discrepancy between the V-function extracted and the optimal one, but requires an explicit transition model and additional computation. This requirement may become too restrictive, especially when learning is introduced to address a complex abstract problem.
Other observed limitations are more closely related to the experimental choices intentionally adopted in this thesis. All evaluated abstractions were deterministic 2D grids based only on the spatial coordinates of the ground environment. The observed relationship between abstraction resolution and learning performance should therefore be interpreted within this specific family of representations. The same consideration applies to the physical information discarded by the grids. They do not explicitly represent velocity, orientation, angular velocity, or contact information. This choice was sufficient for many of the considered temporal tasks, but the cyclic and landing-oriented experiments showed situations in which positional guidance alone can provide only partial information about the behavior required at the ground level.

5.3 Future Research Directions

The limitations identified in the previous section suggest three main directions for future research.
A first direction concerns exploration and experience coverage in long temporally extended tasks. Future work could investigate exploration mechanisms that exploit the logical structure of the task. For instance, possible solutions can include

phase-dependent exploration strategies, custom replay buffers that balance transitions associated with different automaton states, prioritized sampling of rarely observed transitions, reuse of successful prefixes, or curriculum-like training procedures. Another possibility could be a replica-based mechanism to artificially generate meaningful experience for the agent. Furthermore, when supported by the environment, training could also be resumed from previously encountered intermediate states, provided that consistency between the physical configuration and the corresponding logical state is preserved.
A second direction concerns the adoption of richer and possibly learned abstraction hierarchies. The experimental evaluation in this thesis intentionally focused on deterministic 2D grids, but the framework is not restricted to this type of representation. Future work could investigate qualitatively different abstractions that preserve selected physical quantities, interaction modes, semantically meaningful regions, or other task-dependent information. Moreover, different abstraction techniques could also coexist within the same hierarchy, with different state representations, action spaces, and solution methods at different levels. This possibility is particularly relevant for tasks like landing, where positional guidance can be useful but does not represent all the physical conditions required for successful runs.
A third direction concerns increasing the level of automation and coverage of the experimental validation of the framework. The current implementation already mitigates task-specific engineering by allowing non-Markovian objectives to be expressed through LTL_f. However, the selection and construction of suitable abstractions still remain manual. A further extension could involve the adoption of richer specification formalisms, like Signal Temporal Logic (STL) [11], which would allow temporal requirements to be expressed directly considering real-valued signals and physical quantities. Furthermore, evaluating the framework on additional environments, learning algorithms and abstraction structures would help distinguish limitations that are specific to the *Lunar Lander* setting from those that reflect more general properties of the adopted framework.

5.4 Final Remarks

The whole work presented in this thesis shows that combining logical task specifications with abstraction-based guidance is a promising direction for addressing sparse-reward non-Markovian Reinforcement Learning problems. At the same time, the experimental analysis highlights that the effectiveness of this approach depends on different design choices that remain open to further investigation. The proposed implementation and the results obtained provide a concrete basis for extending this line of research toward more general, expressive, and automated frameworks.

Appendix A

Framework User Manual

This appendix aims to provide a practical description of the software framework developed and used for the experimental evaluation presented throughout this thesis. The framework has been designed to separate the definition of the experimental problem from the underlying implementation. In particular, the abstraction hierarchy and the temporal task are specified through independent configuration files, allowing different experiments to be defined without modifying the core learning procedure. More precisely, starting from the provided configuration, the framework builds the required abstraction hierarchy and processes its levels in a top-down direction. The resulting information is then used to generate the shaping guidance for the level immediately below. This procedure is repeated until the ground environment is reached, where the final learning process is performed.

A.1 Project Structure

Path	Description
`config/`	Directory of files describing the abstraction hierarchy and the temporal task.
`src/`	Source directory where the core implementation of the framework is located.
`others/`	Auxiliary scripts used for analysis, diagnostics, and post-processing of experimental results.
`results/`	Output directory containing training logs, policy checkpoints, evaluation results, and other experiment-specific data.
`docker/`	Docker files required to reproduce the software environment.
`run_experiment.sh`	Main entry point used to launch training experiments in a containerized environment.
`run_evaluation.sh`	Script used to evaluate previously trained policies in a containerized environment.

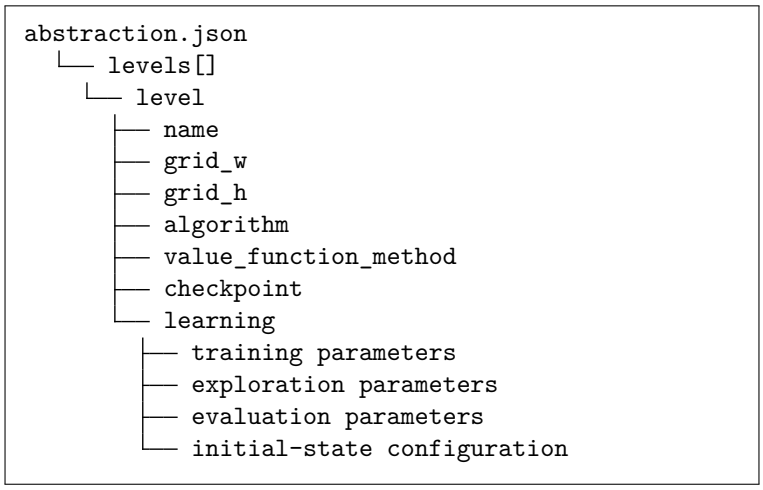
Table A.1. Main components of the framework implementation.

A.2 Configuration Files

Two main configuration files are used: one describes the abstraction hierarchy while the other defines the temporal task, its logical specification, and the predicates used to interpret the environment.
The following sections provide a detailed reference for the parameters supported by the current implementation.

A.2.1 Abstraction Configuration

The `abstraction.json` file defines the abstraction hierarchy used in an experiment. It specifies the number and characteristics of the abstract levels, the method used to solve each level, and the parameters required by the corresponding learning procedure when it is adopted.
Its structure can be summarized as follows:



Field	Type	Required	Accepted values / Default
`name`	String	No	`levelN`
`grid_w`	Integer	Yes	Positive integer; alias: `width`
`grid_h`	Integer	Yes	Positive integer; alias: `height`
`algorithm`	String	Top level only	`value_iteration`, `learning`; default: `value_iteration`
`value_function_method`	String	No	`max`, `policy_evaluation`; default: `max`
`checkpoint`	String	No	Non-empty path; default: `null`
`learning`	Object	Conditional	See Table A.3

Table A.2. Configuration fields supported by each abstraction level.

All non-top abstraction levels are learned and therefore must not explicitly define the `algorithm` field. Conversely, the top level may select either value iteration or tabular learning. The learning object must be specified only if the selected algorithm corresponds to tabular learning. The parameters of this sub-object are described in the following table:

Field	Type	Accepted values / Default
`episodes`	Integer	Positive; default: `10000`
`max_steps`	Integer	Positive; default: `100`
`max_cycles_per_episode`	Integer	Positive; default: `3`
`alpha`	Number	$0 < \alpha \leq 1$; default: `0.1`
`epsilon_start`	Number	$0 \leq \varepsilon \leq 1$; default: `1.0`
`epsilon_min`	Number	$0 \leq \varepsilon_{\min} \leq \varepsilon_{\text{start}}$; default: `0.05`
`epsilon_decay`	Number	$0 < d \leq 1$; default: `0.999`
`gamma_shaping`	Number	$0 < \gamma_{\text{s}} \leq 1$; default: $\gamma_s = 1$
`seed`	Integer	Any integer; default: `0`
`log_interval`	Integer	Positive; default: `1000`
`eval_interval`	Integer	Positive; default: `10000`
`eval_episodes`	Integer	Positive; default: `500`
`eval_seed`	Integer	Any integer; default: `100000`
`initial_state_distribution`	String	`uniform_product`, `gym_reset`; default: `uniform_product`

Table A.3. Configuration fields supported by the `learning` object.

A.2.2 Task Configuration

The `trajectory.json` file defines the temporal objective that the agent must satisfy. It contains the logical specification of the task together with the atomic propositions and the predicates used to evaluate them in the environment.

```
trajectory.json
├── task_type
├── formula                                   [LTL_f only]
├── waypoint_cycle[]                          [cyclic only]
├── goal_reward
├── gamma
├── ground_acceptance_condition               [LTL_f only]
├── regions
│   └── <proposition_name>
│       ├── center[]
│       │   ├── x
│       │   └── y
│       └── radius
└── predicates
    └── <proposition_name>
        ├── type
        ├── center[]                              [circle]
        ├── radius                                [circle]
        ├── x_min                                    [box]
        ├── x_max                                    [box]
        ├── y_min                                    [box]
        ├── y_max                                    [box]
        ├── axis                               [half_plane]
        ├── operator                           [half_plane]
        └── threshold                          [half_plane]
```

The configuration separates the temporal specification from its spatial interpretation. The fields `formula` and `waypoint_cycle` define how propositions must evolve over time, whereas `regions` and `predicates` define when those propositions are true in the *LunarLander* space.

Field	Type	Required	Accepted values / Default
`task_type`	String	No	`ltlf`, `cyclic_waypoints`; inferred when omitted
`formula`	String	No	Valid LTL_f formula; default: `F(goal)`
`waypoint_cycle`	Array	Cyclic tasks	Non-empty array of unique proposition names
`goal_reward`	Number	No	Default: `10000`
`gamma`	Number	No	Default: `0.99`
`ground_acceptance_condition`	String or null	No	`successful_landing`; default: `null`
`regions`	Object	Yes	Non-empty object; see Table A.5
`predicates`	Object	No	Default: empty object

Table A.4. Top-level fields supported by `trajectory.json`.

Circular Regions and Circle Predicates

Field	Type	Required	Accepted values
`type`	String	Predicates only	`circle`
`center`	Array	Yes	Two finite numbers: $[x, y]$
`radius`	Number	Yes	Finite number greater than zero

Table A.5. Fields supported by circular regions and circle predicates.

Box Predicates

Field	Type	Required	Accepted values
`type`	String	Yes	`box`
`x_min`	Number	Yes	Finite number
`x_max`	Number	Yes	Finite number
`y_min`	Number	Yes	Finite number
`y_max`	Number	Yes	Finite number

Table A.6. Fields supported by box predicates.

Half-Plane Predicates

Field	Type	Required	Accepted values
`type`	String	Yes	`half_plane`
`axis`	String	Yes	`x`, `y`
`operator`	String	Yes	`<`, `<=`, `>`, `>=`
`threshold`	Number	Yes	Finite number

Table A.7. Fields supported by half-plane predicates.

A.3 Source Code Organization

The core framework is contained in the `src/` directory of the `multilevel_framework` package. Its implementation is organized in modules which separate configuration management, spatial abstraction, temporal-task processing, RL algorithms, experiment orchestration, validation, and result visualization of the framework. The directory has the following structure:

```
src/
├── trainer.py
├── agent.py
├── abstraction.py
├── abstract_mdps.py
├── spatial_regions.py
├── automaton_validator.py
└── utils.py
```

A.3.1 Experiment Orchestration

The `trainer.py` module corresponds to the entry point of the framework and coordinates the complete lifecycle of an experiment. Before training begins, the module performs the following operations:

1. it loads and validates the spatial and temporal task definitions;
2. it builds the task automaton;
3. it builds the configured hierarchy of abstract MDPs;
4. it computes or restores their V-functions;
5. it initializes the selected ground-level learner; and
6. it augments *Lunar Lander* observations with the current automaton state.

Once the initialization process has been completed, the module starts the ground-level training procedure. During its execution, it manages periodic evaluations, logging, policy checkpoints, and the collection of experimental results. Furthermore, `trainer.py` supports a post-processing mode useful to restore previously saved configurations, V-functions, and numerical metrics to regenerate plots without repeating training.

A.3.2 Learning Agents

The `agent.py` module implements the learning agents available to perform the ground-level training.
The module provides the following components:

- `QNetwork`: standard FFN network used by the DDQN agent;
- `DuelingQNetwork`: alternative neural architecture;
- `ReplayBuffer`: fixed-capacity buffer used to store and sample training transitions. It also tracks the distribution of transitions among automaton states for diagnostic purposes;
- `HierarchicalDQNLearner`: neural learner responsible for action selection, replay-buffer sampling, network optimization, target-network updates, and policy serialization;
- `TabularQLearner`: tabular Q-learning implementation.

A.3.3 Abstraction Configuration

The `abstraction.py` module loads and validates the spatial abstraction hierarchy defined in `abstraction.json`. It also implements the mapping operations required to transfer states and spatial regions between grids with different resolutions.
Three main configuration classes are defined inside it:

- `LearningConfig`: stores the training and evaluation parameters associated with a learned abstraction level;
- `GridLevel`: represents a single abstraction level, including its name, grid dimensions, solution algorithm, V-function method, learning configuration, and optional checkpoint;
- `AbstractionConfig`: represents the complete ordered hierarchy and validates the relationships between its levels.

The method responsible for loading the configuration is `AbstractionConfig.load()`, which validates the field types, accepted values, level names, grid dimensions, and hierarchy constraints. Relative checkpoint paths are resolved with respect to the directory containing `abstraction.json`.
The order of the levels is significant. The first element of the `levels` array defines the abstraction used by the ground-level training procedure. Each following element represents an upper abstraction that can provide a potential to the preceding level. All non-top levels use learning, while the top level can use either learning or value iteration. The module also exposes the following grid-mapping utilities:

- `map_cell`: maps a cell from one rectangular grid to another using its normalized center;
- `map_state`: maps the spatial coordinates of an abstract state while preserving its automaton state;

- `overlapping_cells`: returns the destination-grid cells that overlap a given source-grid cell;
- `map_waypoints`: maps a collection of waypoint cells between grids with different dimensions.

A.3.4 Task Automata and Abstract MDPs

The `abstract_mdps.py` module constructs the task automaton and the abstract MDP hierarchy used by the framework. It supports both episodic LTL_f and cyclic tasks. The module implements the following components:

- `AutomatonStep`: stores the result of an automaton transition, including the next state and any acceptance, failure, or cycle-completion event;
- `LTLfAutomaton`: converts an LTL_f formula into a finite-state automaton and exposes its states, transitions, accepting states, and failure states;
- `CyclicWaypointsAutomaton`: constructs a specialized automaton from the ordered waypoint sequence specified by `waypoint_cycle`;
- `LTLfWaypointMDP`: represents one abstraction level and it is responsible for abstract transitions, task rewards, policy computation, V-function computation, and inter-level shaping;
- `MultiLevelWaypointMDP`: it creates one `LTLfWaypointMDP` instance for each entry in `abstraction.json` and processes the levels from the top of the hierarchy to the bottom.

A.3.5 Spatial Propositions

The `spatial_regions.py` module is responsible for loading and evaluating the spatial propositions declared in `trajectory.json`. These propositions are the predicates that compose the logical task specification.
The module supports three types of spatial objects:

- `CircularRegion`: a circular area defined by a center and a positive radius;
- `BoxPredicate`: an axis-aligned rectangular area defined by minimum and maximum bounds;
- `HalfPlanePredicate`: an area defined by a threshold comparison on either the x or y coordinate.

Spatial propositions are evaluated in two different contexts:

- `truth_assignment_from_observation` evaluates them directly from the continuous coordinates of a *Lunar Lander* observation;
- `truth_assignment_from_cell` evaluates them from a cell of an abstract grid.

A.3.6 Other Supporting Modules

The `src/` directory also contains the following supporting modules:

- `automaton_validator.py`: validates the structure of the generated task automaton, including its states, transitions, propositions, and reachability properties;
- `utils.py`: provides shared utilities for coordinate mapping, data persistence, post-processing, and the generation of plots and diagnostic visualizations.

A.4 Running an Experiment

This final section describes the steps required to configure and launch a training experiment. The complete workflow is summarized in the following diagram.

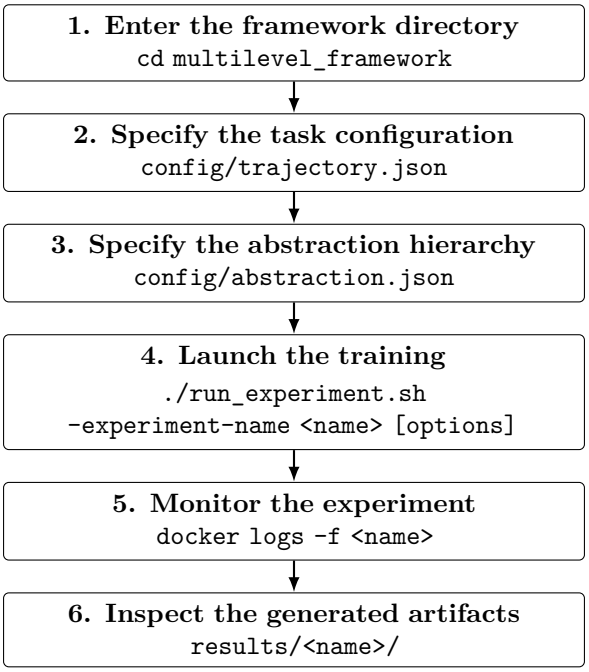


Figure A.1. Steps required to configure and launch a training experiment.

A.4.1 Experiment Output Directory Structure

Each experiment is associated with an output directory whose name is specified through the `-experiment-name` option. Experiment directories are created under `multilevel_framework/results/` and contain all the artifacts related to and generated by the training procedure. The general structure of an experiment directory is the following:

```
results/<experiment-name>/
├── trajectory.json
├── abstraction.json
├── logs/
│   ├── single_epsilon_training_seed_<seed>.log
│   └── abstract_learning/
├── policy/
│   ├── best/
│   ├── last/
│   └── unbiased/                                    [optional]
│       ├── best/
│       └── last/
├── results/
│   ├── single_epsilon_data.npz
│   ├── single_epsilon_data_seed_<seed>.npz
│   ├── abstract_learning/
│   └── abstract_value_functions/
└── img/
    ├── <task>_automaton.png
    ├── abstract_learning/
    ├── heatmaps/
    ├── seed_<seed>/
    ├── evaluation_performance.png
    ├── training_variance_single_epsilon.png
    └── buffer_variance_single_epsilon.png
```

The directories have the following purposes:

- `logs/` contains the ground-level training logs and the logs generated while learning the abstract levels;
- `policy/best/` contains the policies that achieved the best periodic evaluation results;
- `policy/last/` contains the policies saved at the end of training;
- `policy/unbiased/` contains the optional checkpoints produced when the ground unbiased learning is enabled;
- `results/` contains the numerical training metrics and the computed abstract V-functions;
- `img/` contains the automaton representation, abstract V-function heatmaps, learning curves, evaluation results, and seed-specific diagnostic plots.

Bibliography

[1] Giorgio Ausiello, Fabrizio D'Amore, and Giorgio Gambosi. *Linguaggi, modelli, complessità*. Franco Angeli, 2003.

[2] Fahiem Bacchus, Craig Boutilier, and Adam Grove. Rewarding behaviors. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence*, AAAI'96, page 1160–1167. AAAI Press, 1996.

[3] Ronen I. Brafman, Giuseppe De Giacomo, and Fabio Patrizi. Ltlf/ldlf non-markovian rewards. In *AAAI Conference on Artificial Intelligence*, 2018.

[4] Roberto Cipollone, Marco Favorito, Flavio Maiorana, Giuseppe De Giacomo, Luca Iocchi, and Fabio Patrizi. Exploiting robot abstractions in episodic rl via reward shaping and heuristics. *Robotics and Autonomous Systems*, 193:105116, 2025.

[5] Giuseppe De Giacomo and Moshe Vardi. Linear temporal logic and linear dynamic logic on finite traces. *IJCAI International Joint Conference on Artificial Intelligence*, pages 854–860, 01 2013.

[6] Farama Foundation. Lunar lander. Online documentation. Accessed October 8, 2026. Available at `https://gymnasium.farama.org/environments/box2d/lunar_lander/`.

[7] Francesco Fuggitti. Ltlf2dfa. Software, version 1.0.3, Zenodo, March 2019. DOI: `https://doi.org/10.5281/zenodo.3888410`. Source code: `https://github.com/whitemech/LTLf2DFA`. Project website: `http://ltlf2dfa.diag.uniroma1.it`.

[8] Giuseppe De Giacomo, Luca Iocchi, Marco Favorito, and Fabio Patrizi. Restraining bolts for reinforcement learning agents. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 13659–13662. AAAI Press, 2020.

[9] Marek Grześ. Reward shaping in episodic reinforcement learning. In *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '17, page 565–573, Richland, SC, 2017. International Foundation for Autonomous Agents and Multiagent Systems.

[10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.

[11] Oded Maler and Dejan Nickovic. Monitoring temporal properties of continuous signals. In Yassine Lakhnech and Sergio Yovine, editors, *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems*, pages 152–166, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.

[12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[13] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, ICML '99, page 278–287, San Francisco, CA, USA, 1999. Morgan Kaufmann Publishers Inc.

[14] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*, pages 46–57, 1977.

[15] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.

[16] Rodrigo Toro Icarte, Toryn Q. Klassen, Richard Valenzano, and Sheila A. McIlraith. Reward machines: Exploiting reward function structure in reinforcement learning. *J. Artif. Int. Res.*, 73, May 2022.

[17] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2025.

[18] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.

[19] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning, 2016.

[20] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.